

Generative Specification: A Pragmatic Programming Paradigm for the Stateless Reader

Author: Juan Carlos Ghiringhelli (Pragmaworks)

Version: 4.0

Date: June 2026

Status: Preprint

Document type: *Compendium* — the canonical, comprehensive treatment and single source of truth for the Generative Specification research program. The focused **white paper**, the *Onwards* essay, and individual **conference submissions** are derived from this document and kept consistent with it — the cascade pattern GS itself prescribes, applied to its own literature. Read a derived artifact for the argument in brief; read the compendium for the complete treatment, evidence, and apparatus.

Changes in V4 (vs V3 published 2026-04-27): §4.1.f extended with the *Generative-X lexicon* — a parallel naming for what the AI executor performs at each tier (authoring + verification), with explicit notes on hardening as a cross-cutting capability and dynamic testing as a primitive realized at three tiers. The obligation-removal framing in V3 named what the practitioner *stops* doing; V4 adds the symmetric naming for what the executor *starts* doing. §4.1 extended with the *schema-activated derivation* framing — a cognitive-science analog (Bransford & Johnson, 1972 schema-fit effect) naming the mechanism by which a bounded, self-describing specification activates the AI’s domain knowledge subset rather than its full prior distribution; explains why Bounded and Self-describing have disproportionate weight in the rubric. §4.4 extended with the *failure-mode catalog* note — twenty-nine named pathologies derived from six production projects, each mapping to one or more absent properties; published as a companion diagnostic instrument at pragmaworks.dev/diagnostico. §4.1.b extended with the *token economics of authored structure* — the per-session cost argument complementing cost inversion: tokens-per-correct-output, not tokens-generated, is the binding metric, and the sentinel is an authored compact structure of the same family as a *compact knowledge graph* (CKG; Yarmoluk & McCreary, 2026) that the reader navigates instead of re-deriving, generalizing the CKG’s single prerequisite relation to the AI executor’s operating path. §5 disciplines table extended with four structural disciplines named for their industry origin — *Type-Driven Design* (make-illegal-states-unrepresentable; the compiler as a verification layer), *Design by Contract* (Meyer; named as the direct lineage of the F-NNN specification format), *Convention over Configuration / Screaming Architecture* (predictable structure as a self-describing aid), and an enriched *Functional Programming* row naming immutability and the functional-core/imperative-shell split — each mapped to the GS properties it feeds and how GS subsumes it. §4.1 opens with *the bridge* — the positive premise naming why externalizing intent succeeds: every structural discipline is a bidirectional bridge between human conceptual language and executable code, and the transformer is the first reader trained on both banks, able to cross it in both directions; bridge-construction is identified as the mechanism by which a stateless reader’s derivation matches human intent (the CKG retrieval result is its empirical signature). §4.1.b token-economics passage upgraded from “planned” to *measured* — the ForgeCraft KX replication ran the CKG benchmark method on a GS software harness (routed CNT 0.808 F1 / 78.6k tok vs RAG-dump 0.611 / 100k vs no-structure 0.431 / 233k), confirming routed retrieval wins on accuracy and cost and the CKG divergence replicates on software (T4 0.909 vs 0.006); §7.8.E placement noted. §4.1.b further extended with the *retrieve-generate-verify* decomposition — naming the read side of a GS session as a retrieval problem (RAG-grade scanning vs. CKG-grade lookup over authored structure) attacked from both ends (authoring the structure + shaping the code to be retrievable, with a code-search engine as traversal), and the harness as the categorically distinct verification step; GS framed as retrieval-augmented *and*

verified generation with the retrieval authored rather than inferred. §4.1 *bridge* sharpened with the **asymmetry**: the two banks are unequal in the model's competence (training corpus overwhelmingly natural language; code an exact, fragmented slice), so the bridge's leverage is that encoding intent in human-conceptual terms routes the hard half (exact code) through the model's strong half (human meaning) — load moved from the weak bank to the strong one. §4.1.b adds the **discipline-role taxonomy**: one *verify* role (secure correctness — TDD/BDD/contracts/types/gates → Verifiable/Defended/Executable) and three *retrieve* roles on the bridge — *legibility* (read the what → Self-describing/Composable), *bounding* (less to read; a second token-cost lever — DRY/dedup/dead-code/YAGNI → Bounded), and *decision-memory* (read the why → Auditable, via ADRs/commits/EDRs) — each with its enforcement mechanism (CodeSeeker's dedup/dead-code detection is the bounding enforcer), explaining why the seven properties partition as they do. **New §7.8.E (Experiment V)** presents the measured evidence: AX Treatment-v8 *construction invariance* (a ForgeCraft-generated harness reaches the same 14/14 / 12-of-12-blind-audit ceiling as the best hand-built arm, 11/11 live use-case probes) and the **KX knowledge-retrieval replication** (routed navigation tree 0.808 F1 / 78.6k tok beats RAG-dump 0.611/100k and no-structure 0.431/233k; CKG divergence replicates on software, T4 0.909 vs 0.006; learning-graph.csv emission makes every ForgeCraft project a benchmarkable CKG domain). The remaining controlled experiments — **EX** (production T1–T3 proof: 13/13 Hurl probes/1,013 assertions, 3/3 env probes, 6/6 SLO gates on a live Railway deployment), **ALX** (formal-tier self-applicability: Loom compiler derived from its own spec, S_realized 0→1.0, 386/386), and **RX** (independent reproducibility: 104 passing tests from a committed GS document) — are given their own dedicated subsections (§7.8.D, §7.8.F, §7.8.G), so all seven controlled experiments now present evidence in one place. No theoretical claims revised; sections deepened, the bridge premise named, disciplines table extended, the experimental record reorganized so all seven controlled experiments hold dedicated §7.8 subsections (KX added; EX, ALX, and RX promoted from inline references to full subsections), one companion resource linked. This version also **repositions the document as the Compendium** — the canonical source from which the white paper, the *Onwards* essay, and conference submissions are derived — and adds **§7.8.A**, observational field corroboration from a paying four-day practitioner workshop (June 2026), fenced as observational rather than controlled evidence.

Addendum (June 2026) — two pilot experiments added. §7.8.H **MX** (model cost & tiering): once a task is GS-specified, a mid-tier model (Sonnet 4.6) matches a strong model (Opus 4.8) at 149/149 verified on the full Conduit for $\approx 6\times$ lower cost, and model-tiering is unjustified when the mid model one-shots the task — this moves the cost claim (§4.1.b, §8.2.1) from reasoned to *measured* for this task class. §7.8.I **RND-1** (specification / verification / bounded-context under delivery pressure): the **prescriptive-specification** arm is confirmed (a descriptive spec floors the model to the literal minimum at equal cost; a prescriptive spec recovers full intent), while the independent-verification and bounded-context arms return honest, *bounding* nulls (their value surfaces against weaker/adversarial agents and at large scale, not on medium tasks with a capable, honest model). Both are **pilot-scale** (single-shot, small n) and are weighted accordingly — distinct from the pre-registered controlled experiments (§7.8.B–G). Both are committed publicly at `experiments/mx/` and `experiments/rnd-1/`.

Prologue: The \$327 Million Contract That Was Never Written

On September 23, 1999, the Mars Climate Orbiter completed a nine-month, 416-million-mile crossing of interplanetary space. It arrived within 26 kilometers of its intended trajectory — a feat of precision that represented the combined work of thousands of engineers, two years of mission planning, and \$327.6

million in public investment. Then it entered the Martian atmosphere at the wrong angle and was destroyed in 57 seconds.

The cause was not a bug in any individual system. Lockheed Martin's navigation software reported thruster force in pound-force seconds. NASA's flight computer expected newton-seconds. Both teams had implemented their components correctly, according to their own assumptions. The interface between the two systems had no explicit unit specification. The contract had been assumed, not written. *(NASA MCO Mishap Investigation Board, Final Report, November 1999; analysis confirmed in the Columbia Accident Investigation Board framework as a category of organizational interface failure.)*

No test caught it. The code compiled cleanly. Individual modules passed validation. The failure was invisible everywhere except at the boundary — at the seam where two systems, each internally coherent, had to agree on a shared language. They did not agree, because the agreement had never been formalized.

That was 1999. That contract failure took two years and \$327.6 million to produce.

Today, in current practice, an AI assistant produces an equivalent interface in thirty seconds. It generates clean code. Types check. Unit tests pass. And embedded in that output, invisible, is a set of implicit assumptions — about units, about field ordering, about what a null means, about which layer owns which concern — that the generating model resolved silently, drawing on everything it has ever read about how such systems are usually built. The code is correct in isolation. At the boundary, when two AI-generated systems meet across a session boundary or a team boundary or a service boundary, the implicit assumptions do not automatically agree.

The Orbiter problem did not go away. The velocity of producing it increased by orders of magnitude.

This is not an argument against AI-assisted development. It is an argument for what that development requires. The Jacquard loom, introduced in 1804, did not eliminate weaving craft — it relocated it. The weaver stopped managing the shuttle. The weaver started managing the card. The complexity moved upstream, into the specification, where it could be expressed once and executed at machine speed. The loom was capable. The card made that capability directed.

Software engineering is undergoing the same relocation, at a speed the loom's inventors could not have imagined. An AI agent with CLI access can read your codebase, write tests, execute migrations, commit to git, and iterate on a running system, all within a single session, starting from nothing. The executor is extraordinarily capable. The question is what governs it across the session boundaries it cannot see, the team context it was never given, and the architectural decisions that were made in conversations it was not part of.

The answer is the same as it was in 1804. The answer is the same as it was in 1999. A specification precise enough that a stateless reader — one with no memory of your intentions, no shared context, no ability to ask a clarifying question — can derive correct output from it alone.

That discipline is what this work defines.

This work requires an AI agent with direct CLI access — not a chat assistant, but an agent that can read files, write files, run tests, commit to git, and execute commands. Claude Code, Cursor, and VS Code agents in agentic mode meet this requirement. The methodology is not applicable to chat-based interfaces. §8.6 develops the runtime requirements in full.

Abstract

The dominant failure mode of AI-assisted development is not incorrect code. It is **architectural drift**: structurally incoherent output produced at generation speed across session boundaries. The Orbiter problem did not go away. The velocity of producing it increased by orders of magnitude.

Generative Specification (GS) is the programming discipline of the pragmatic tier: the tier at which *derivability* — what a stateless reader can correctly determine from artifacts alone — becomes a binding constraint. In the sense Robert C. Martin characterized structured programming, OOP, and functional programming, GS is a discipline defined by what it removes from programmer freedom. What GS removes is implicit context.

Seven specification properties operationalize the discipline — **Self-describing, Bounded, Verifiable, Defended, Auditable, Composable, Executable** — each named for a specific failure mode observed across six production projects. The economic consequence: **cost inversion** (§4.1.b). When regeneration is near-free, specification becomes the scarce resource.

Empirical evidence spans six production projects, a controlled adversarial study (14/14 rubric, 109 runner-verified tests, 106 passing on independent re-run), a self-applicability experiment at the formal specification tier (ALX: Loom compiler derived from its own formal specification, $S_{\text{realized}} = 1.0$, 386/386 acceptance tests), a replication experiment reproducible by any reader with an API key (RX), a production deployment proof against a live runtime (EX, §7.8.D: 13/13 behavioral probes, 6/6 SLO gates), and a knowledge-retrieval replication (§7.8.E) that measures the harness's CKG-class token economics — routed retrieval over the authored navigation tree reaching higher accuracy at lower cost than both everything-in-context and code-search baselines, with the advantage replicating the compact-knowledge-graph result on a software substrate. Observational field corroboration from a paying practitioner cohort (§7.8.A) shows the discipline taking hold on a real team and codebase within a single engagement; it is fenced as observational, and a controlled human-participant study is noted as future work.

The discipline is replicable: §7.8.G (RX) reproduces a scoped implementation from a fresh GS document with 104 passing tests, starting from nothing. The specification is the mold. The AI is the foundry.

1. Introduction

The dominant failure mode of AI-assisted development is architectural drift. This work names its structural cause, defines the discipline that resolves it, and provides evidence across six production projects, a controlled adversarial study, and observational corroboration from a paying practitioner cohort.

Hoare logic (1969), design by contract (Meyer, 1992), REST (Fielding, 2000), and the semantic web — every one of these formal disciplines for correct computing was provably right. None was fully implemented at scale. The formal theories were complete. The executor was not.

The missing executor was not intelligent — it was *consistent*. These disciplines failed not because their proofs were wrong but because sustaining them required what humans reliably cannot provide across every person, every commit, every deadline. That executor has changed.

Programming languages were never designed for you to think in. They were designed as messengers — protocols for translating human intent into what a context-free parser could deterministically read. The

new reader — a large language model — is not a context-free parser. Trained on the full corpus of human-written text and code, it interprets each token relative to everything surrounding it in the context window: the import statements, the class hierarchy, the architectural rules in the session-loaded documentation. The constraint that made programming languages rigid is no longer operative. The specification can now operate at a higher abstraction level than the implementation language, stated in the language of the domain, derived into implementation by a reader that understands the surrounding context.

But this requires discipline. The new reader is stateless: no memory of prior sessions, no institutional context, no tolerance for the implicit. Incomplete descriptions are completed at generation speed, consistently, incorrectly. Architectural drift is the observable result — multi-session accumulation of locally valid but architecturally incoherent output. The cure must persist across session boundaries.

The most common objection — that better prompting solves drift — is addressed in §8.9. A prompt is a session artifact. The cure for a multi-session problem cannot be a single-session tool.

Structured programming constrains *form* (syntactic tier). The semantic disciplines — SOLID, TDD, DDD — constrain *meaning for a human reader* (semantic tier). Generative Specification operates at the pragmatic tier: derivability by a *stateless* reader becomes the binding constraint.

The four claims this work makes differ in epistemic weight: Claim 1 is definitional (internally consistent or not); Claim 3 is the strongest empirical result (external implementations, blind scoring, independent metrics); Claim 2 is directional evidence from a single-practitioner experiment with a documented non-monotonic trajectory; Claim 4 is observational field corroboration of single-session practitioner transfer (a paying client cohort), with a controlled human-participant study noted as future work.

This work makes the following contributions: (1) *derivability obligation* — the naming of implicit context removal as the defining structural constraint of AI-assisted development, occupying the pragmatic tier prior disciplines left vacant; (2) *pragmatic tier identification* — the first formal placement of AI-assisted development discipline in the syntactic/semantic/pragmatic tripartition, distinct from completion tools that operate within the existing paradigm; (3) *seven-property rubric* — a teachable, scored instrument (Self-describing, Bounded, Verifiable, Defended, Auditable, Composable, Executable) operationalized as a 14-point scale and validated across the six production projects; (4) *phase collapse* — a named phenomenon in which specification, implementation, and verification collapse into a single derivation step when the specification is complete and the executor is a capable AI reader; (5) *domain dimensional expansion* — an observed cross-domain effect in which a GS-governed specification exposes latent structural errors in adjacent system layers not originally targeted by the specification; (6) *six-tier obligation cascade* — a maturity model mapping the derivability obligation across development, staging, production, evolution, synthesis, and meta-telos, each tier removing a distinct class of practitioner obligation; (7) *layered validation methodology* — the AX/BX experimental structure in which solo-practitioner and peer-implementation layers each close a distinct source of circularity in the evidence chain, with observational field corroboration on a real team as the practice-side check.

2. The Abstraction Ladder

The ladder has been moving in one direction since the first compiler freed the engineer from machine code. Each step produced a more capable reader. Each more capable reader demanded a richer specification. The engineer stopped managing registers when compilers could derive machine code from

expressions. Stopped wiring object graphs when frameworks could derive them from configuration. Stopped writing route handlers when annotations could declare them. The pattern — specify *what*, not *how*; let the reader derive *how* — has been the field’s intuition for sixty years.

The reader changed again in 2017. A large language model reads context-sensitively: its interpretation of any token depends on everything surrounding it in the context window — the import statements, the class hierarchy, the architectural rules in the loaded documentation. These are not optional enrichments. They are the grammar the model uses to determine what a valid sentence in this system looks like. The specification can now operate at a higher abstraction level than the implementation language, stated in the language of the domain, derived into implementation by a reader that understands the surrounding context.

The recurring pattern the field has been executing — identify a layer where intent is still being prescribed as execution, find or build a reader capable of deriving execution from a richer specification, and remove the prescription — continues at the lifecycle layer. Architecture, decisions, conventions, rationale: the layer that human teams carried implicitly in shared memory. Generative Specification names the pattern, applies it to that layer, and derives the discipline that follows from the reader now available.

3. The Theoretical Gap: From Context-Free to Context-Sensitive Practice

The Introduction described why the reader changed. This section describes what that change costs when the specification does not change with it.

An AI coding assistant starts from the artifacts present. The context window resets at the session boundary, carries no institutional memory of prior sessions (Tulving, 1972; Squire, 1987), and has no mechanism for deterministic judgment — every output is a probability distribution over possible continuations. What the channel carries is itself a specification act: incoherent artifacts amplify incoherence at generation speed. Where a human engineer *interprets* an underspecified requirement — compensating across the gap with memory, inference, and accumulated context — the AI *processes* what is present. The human compensation layer does not exist.

The missing context is not ambiguity in a present signal but absence of institutional memory. The gap that prosody and paralanguage bridge in human dialogue — “that doesn’t feel right” carries a precise technical concern through everything the room provides — is a gap the specification must close explicitly. The cost of ambiguity is not misunderstanding but **drift** (analogous to architectural erosion; De Silva and Balasubramaniam, 2012): implementation that is locally valid and tests-passing but architecturally incoherent, propagated across every subsequent session that inherits the corrupted context. Lehman’s laws (1980) establish that complexity increases unless active work is done to reduce it. Drift is that law operating in the absence of a specification constraint.

The specification does not merely describe the system. Processing it *causes* the system to be what it becomes. That is why GS must be designed for a **stateless reader**: an executor that begins each session with no memory of prior sessions, no institutional context, no accumulated conventions, and no ability to ask clarifying questions. Everything not in the artifacts is absent.

The expanding context window does not escape this conclusion. An infinite window over an underspecified codebase is not infinite derivability — it is an infinite drift surface. The model reads more

of the implicit record; it cannot derive intent that was never externalized. The structural enforcement layer — commit hooks, CI gates, phase guards — is also independent of context size: a model with unlimited memory cannot prevent incorrect output from entering the codebase unless the discipline makes that output architecturally unreachable.

Concurrent infrastructure-layer work addresses an adjacent problem. Anthropic’s Auto Memory feature for Claude Code (2026) writes session notes from agent corrections across sessions; Auto Dream, a background consolidation process, prunes stale or contradictory memories and indexes them after every five sessions. This is a bottom-up approach: it learns from observed behavior and consolidates. GS is a top-down approach: the architectural constitution is authored intentionally before generation begins. Both address the stateless reader problem at different layers — Auto Dream reduces preference drift across sessions; GS constrains architectural intent within and across sessions. They are complementary, not competing. The distinction matters: reactive consolidation learns what the practitioner does; preventive specification states what the system must be. A well-maintained GS artifact set is the stable substrate that makes memory consolidation coherent rather than arbitrary.

3.5 Related Work

Other researchers arrived at this problem independently, from different directions. It belongs here — after the problem has been stated, before the solution is defined — so readers can evaluate the distance between what others found and what this work claims.

Two independent research threads validate the problem formulation, neither arriving at the paradigm claim or the full methodology.

Gordon (2024, ACM Onward!) argues in *The Linguistics of Programming* that linguistic research (including formal grammar theory and the Chomsky hierarchy) offers substantially underused conceptual tools for programming language and software engineering research. Gordon establishes structural parallels between linguistics and PL/SE without focusing on LLMs as the reader who changes the design requirements. This work takes a specific step within the direction Gordon identifies: grounding the Chomsky hierarchy in the architectural shift produced by deploying LLMs as primary consumers of software specifications, and deriving the restriction discipline that follows. Gordon independently identifies the territory this work’s core framework inhabits, without arriving at the stateless-reader consequence, the restriction discipline, or the paradigm claim. The distinction is between analytical and prescriptive work: Gordon maps the domain; this work derives a discipline from standing in it.

Thirolf (2025, KIT/KASTEL) independently identifies, in *Analysis of Project-Intrinsic Context for Automated Traceability Between Documentation and Code*, the exact failure mode described in §3: architectural drift caused by implicit context that AI tools cannot access, observed empirically through documentation-code traceability gaps in AI-assisted development sessions. The problem statement matches without coordination. Thirolf proposes automated traceability tooling as a structural response — complementary to but narrower than the full generative specification methodology.

Orlanski et al. (2026) introduce **SlopCodeBench** (arXiv:2603.24755), a language-agnostic benchmark of 20 problems and 93 checkpoints in which agents repeatedly extend their own prior solutions. Findings: no agent solves any problem end-to-end across 11 evaluated models; structural erosion rises in 80% of trajectories, verbosity in 89.8%; agent code is 2.2× more verbose than matched human-authored code and deteriorates with each iteration while human code stays flat. A prompt-

intervention study shows prompting improves initial quality but does not halt degradation. The authors conclude “current agents lack the design discipline iterative software development demands.” This is independent empirical measurement of the failure mode §3 names, using trajectory-level instrumentation the GS experiments do not employ. The mechanistic explanation follows from tooling: each context-window compaction retains recently active files but discards prior architectural decisions — the agent continues from a lossy summary, applying the same token-pressure pattern from a degraded baseline. GS artifacts are structurally resistant because the architectural constitution and ADRs are short, structured, and loaded at session start; they fit the compaction routine’s file budget. Session memory does not fit; it is what the summary replaces.

These works establish that the failure mode this work addresses is not an artifact of a single practitioner’s context. SlopCodeBench independently measures the problem GS is designed to solve but does not test GS as a solution and is not cited as solution validation.

Anthropic (2026) provides a fourth validation thread through Auto Memory and Auto Dream, shipped in Claude Code (March 2026). Auto Dream is a background consolidation sub-agent that prunes stale session notes, resolves contradictions, and reindexes the memory corpus between sessions — addressing the same failure mode §3 describes: accumulated session notes become noise that degrades output across multi-session workflows. This is a production feature shipped because the failure mode was observed at scale; it constitutes a category of evidence distinct from benchmark studies. The relationship to GS is architectural, not competitive: Auto Dream is reactive (it consolidates after drift has accumulated); GS is preventive (the architectural constitution and ADRs make drift structurally unreachable before any session begins). Both address the stateless reader problem at different points in the causal chain.

The convergence of four independent lines of evidence — two academic (Gordon, Thirolf), one benchmark (Orlanski et al.), one product engineering decision (Anthropic Auto Dream) — is precisely the finding §3’s theoretical claim predicts.

Spec-driven tooling — the artifact of rigor versus the predicate of guarantee. A solution-side neighbor is worth distinguishing directly, because it is the nearest practice and the easiest to mistake GS for. Spec-driven development toolkits — GitHub’s *Spec Kit* and the phased-gate workflows it popularizes (Constitution → Analyze → Plan → Tasks → Execute) — bring real discipline: a written constitution, ADRs, a traceability matrix, quality reports. A field examination of a production-grade enterprise system built this way — a strong engineer, a full governance harness, genuinely well-made — isolates the variable GS exists to name. The *shape* of rigor was fully present while the *guarantee* behind it was, in places, absent: a verification-phase report that described a validation procedure never executed; a flagship capability read as “done” in the narrative while the traceability matrix recorded its core output as never implemented; a governance gate that lived as fitness tests but whose CI had never run; functional requirements that cited *source files* as their origin — specifications reverse-engineered from the code they were meant to generate. None of this was concealment; the harness was admirably honest about its own gaps. The lesson is structural: **spec-driven tooling produces the artifact of rigor; GS’s seven properties are the predicates that decide whether the artifact is backed by a guarantee.** Three predicates carry most of that weight, and are exactly where the specimen scored lowest — **Verifiable** (status must be computed from an execution record, never authored), **Defended** (a control counts only if it actually runs — enforced, not aspirational), and **Executable** (the directionality test: could a stateless reader regenerate the system from the specification alone, or does the spec only cohere with the code open beside it?). Used this way, spec-driven tooling *documents* a system; GS *specifies* one into existence. Its genuine strengths — an executable policy layer, an ADR

dependency graph, a per-decision constraint ledger, an honest gap register — are imported as positive patterns; its failure modes become negative test cases for the scoring rubric.

The full treatment of GS’s relationship to SOLID, clean architecture, TDD, DDD, the prior paradigm’s anomalies, and the LLM code generation research literature is in §5.

Status of companion work (current as of April 2026)

Work	Status
Loom — Formal language layer	Active development (M1–M23 complete). Companion paper in preparation. All references in this work route to the companion.
BIOISO — Biological Isomorphisms	Companion paper in preparation. Colony run in progress.
Controlled practitioner study	A pre-registered, powered, multi-cohort human-participant study is planned to move practitioner-transfer evidence from observational corroboration to controlled confirmation.

This work stands on its own evidence. The companion works extend the claim; they are not required to evaluate it.

4. Generative Specification: The Principle

The preceding sections established why a new discipline is necessary: AI readers are stateless, specifications designed for human readers are insufficient, and the gap compounds at generation speed. This section defines what the discipline consists of.

Paradigm throughout this work carries Robert C. Martin’s precise sense: a discipline defined by what it removes from programmer freedom. Structured programming removed `goto`. Object-oriented programming removed unconstrained access to internal data. Functional programming removed variable reassignment. GS removes the freedom to leave architectural intent implicit. This removal differs in kind from its predecessors. Structured programming, OOP, and functional programming each constrained freedoms for human readers who could still compensate for gaps through memory, collaboration, and institutional context. GS’s removal is absolute for its reader: a stateless executor that begins each session with none of those recovery mechanisms. What prior paradigms made inconvenient, GS makes structurally absent — because the reader that would have compensated does not exist. The semiotic tripartition — *syntactics*, *semantics*, *pragmatics* — follows Charles W. Morris (1938 *Foundations of the Theory of Signs*): the pragmatic tier is the relation of signs to their interpreters in context of use. GS occupies the pragmatic tier because it governs derivability for a reader who carries no interpretive context — the tier prior disciplines left vacant.

The most common alternative framing — that AI coding assistants already achieve what GS claims — conflates acceleration with direction change. Copilot, Cursor, and completion-based tools write code in the paradigm the practitioner already uses, faster. GS removes the obligation to write code at all. The remaining obligation — writing the grammar from which correct implementations are derived — is categorically different: different artifact set, different workflow, different failure mode. A faster completion tool is a Popperian refinement. GS is a Martin-sense paradigm: it removes a degree of

programmer freedom in the same structural move structured programming removed `goto`. The practitioner who adopts GS does not speed up. They change direction.

Martin's three examples establish the mechanism; they are not its ceiling. Every formally proved theory of correct computing is a potential restriction layer: Hoare's precondition-postcondition logic (1969), Milner's type polymorphism (1978), Meyer's design by contract (1992), Girard's linear types (1987), Honda's session types (1998), Fielding's hypermedia constraints (2000). Each was abandoned not because its mathematics failed but because sustaining it required consistency no human team could maintain at scale. The AI executor holds all of them. The specification opens the door. The full argument for why these theories failed under human practice and succeed under GS is developed in *Onwards!*

On enforcement. A reviewer familiar with Martin's criterion — that paradigms are defined by the restrictions they *enforce*, not merely recommend — may object that GS's restriction is a document the compiler knows nothing about. The objection has a structural answer. GS's enforcement layer is not the specification file; it is the quality gate stack the specification mandates. The pre-commit hook that rejects a file exceeding the Bounded property's line limit is enforced — it blocks the commit. The CI pipeline that fails on a coverage drop is enforced — it blocks the merge. The MCP tool boundary that prevents a Bounded violation from being committed is enforced — it prevents the write. The spec is the grammar; the toolchain is the compiler. The architectural constitution is not advisory in a GS-governed project any more than a type annotation is advisory in TypeScript: both can be suppressed with deliberate override, and both are automatically checked without override. What GS restricts — implicit context, unbounded scope, unverified output, unrecorded decisions — is structurally unreachable in a correctly governed project, not merely discouraged. The enforcement is distributed across commit hooks, CI gates, and MCP boundaries rather than concentrated in a single language parser, but enforcement is enforcement. What changes with each new model generation is not the paradigm's restriction claim; it is the precision with which the restriction can be stated. As models become more capable readers, some current directives become redundant and fall away. The restriction class itself — implicit context is not permitted in the lifecycle layer — does not change.

On verification: cross-cutting, not separate. The obligation cascade developed below in §4.1.f is organized by *lifecycle stage* — development, staging, production, evolution, synthesis, meta-telos — not by activity type. Verification is not one of those stages; it is a capability that recurs *at* every stage with stage-appropriate tests. A dev-time harness certifies the derivation that lets the practitioner stop reading generated code (T1); a staging-stage harness certifies the deployed environment (T2); a production-stage harness certifies runtime behavior (T3); an evolution-stage harness certifies governed mutation (T4); a colony-level harness certifies that each synthesized entity satisfies its own telos (T5). Each tier therefore removes two obligations simultaneously — an *authoring obligation* and a *verification obligation* — because the verification at that stage is what makes the authoring removal safe. Treating the harness as a cross-cutting capability rather than a separate tier resolves a structural ambiguity in earlier formulations of GS that placed “harness” alongside “spec” as a peer rung on the ladder; it is neither, it is what binds every rung to the spec.

4.1 The Mechanism

A programming discipline of the pragmatic tier consists of making derivable from artifacts what was previously accessible only through interpretive context. That is the obligation a stateless reader makes structurally necessary.

The bridge — why this works at all. The stateless reader explains *why* intent must be externalized; the seven properties and the navigation tree explain *how*. The bridge explains why externalizing it *succeeds*. Every structural discipline — intentional naming, the specification itself, SOLID, domain-driven design’s ubiquitous language, type-driven design, design by contract — is a bridge between human conceptual language and executable code: it encodes human meaning in a form the machine can act on, and machine behavior in a form the human can verify. These disciplines were built to carry intent across that gap for the *next human reader*. The transformer is the first reader trained on both banks of the gap — the corpus of human language and the corpus of code — and therefore the first that can cross the bridge in both directions: read intent encoded as structure, and emit code that encodes intent. Generative Specification works because it makes building and maintaining that bridge the *primary act* of development rather than a byproduct of it. The disciplines were never only about machine legibility or only about human legibility; their power is that the *same* structure serves both readers, and a reader now exists that is fluent on both sides. But the two banks are not equal in that reader’s competence, and the asymmetry is the source of the bridge’s leverage. The training corpus is overwhelmingly natural language; code is a small, exact, fragmented slice of it — every language its own precise grammar, several of them low-resource, every token load-bearing in a way prose is not. The model is therefore far stronger on human conceptual meaning than on any specific language’s exact syntax. Encoding intent in human-conceptual terms — names, ubiquitous domain language, the specification, contracts — routes the hard half of the problem (producing exact code) through the half the model is fluent in (understanding what is meant). The bridge moves the load from the model’s weak bank to its strong one, collapsing roughly half the problem to a derivation anchored in meaning the model already holds: read human comprehension, emit code. The Yarmoluk–McCreary result (§4.1.b) is the empirical signature of the same principle in retrieval: a bridge authored once is crossed an order of magnitude more cheaply, and more accurately, than one re-derived from prose at every query. This is why the derivation step is disproportionately determined by naming, specification, and the binding of tests to acceptance criteria — nearly all of that is bridge-building, and the bridge is what makes a stateless reader’s output match human intent.

The two translations — the bridge in a practitioner’s terms. The same asymmetry can be stated the way an engineer lives it daily. Building software is two translations. The *first* turns a clear specification into working code, tests, configuration, and infrastructure — the **syntactic** translation, taught in courses and mastered early in a career. The *second* turns a mix of business ideas, customer demands, and half-formed feature requests into that specification — the **semantic** translation, the true craft of an experienced engineer who has seen enough mature systems to know what can be built, what holds, and what people actually use. Historically juniors carried the first and seniors the second, though the division was never clean. The stateless reader changes the economics of precisely the first: given a sufficient specification it derives the code, the tests, the pipelines, and the configuration without fatigue — the syntactic translation automated. What it cannot do is the second — reading intent, expanding a vague wish into a concrete contract, ratifying what *correct* means — because that is the judgment layer (T6, §4.1.f), which this work holds to be structurally irreducible. The discipline therefore does not remove the engineer; it moves the engineer up a floor, from the translation a machine now masters to the one it cannot. And the convergence is toward *fewer engineers doing more*, not zero: the second translation contracts as models read intent better, but the human ratifier at its terminus does not disappear. This is the accessible on-ramp to the bridge, and it is the framing that has consistently landed with practitioners and non-technical audiences alike.

4.1.a The Grammar Mechanism

A **Generative Specification** is a finite, coherent set of system artifacts sufficient to generate any valid implementation state of the system without requiring external human context.

A description records what was built. A specification is what a correct implementation must continuously fit. When the implementation drifts, you fix the specification and regenerate. The specification is the program.

The central structural property is *derivability*: a system's lifecycle layer is derivable when a stateless reader, given its artifact set alone, can correctly determine what should be built, where, why, and to what contracts, without requiring external human context.

Valid carries a broader obligation than Chomsky's *grammatical*: a valid implementation state is both structurally well-formed under the specification's rules and conformant to its behavioral and acceptance-test obligations. A wrongly-specified grammar is possible — correct by its own rules while failing the system's actual obligations. This is the methodology's primary failure mode, and the reason the specification faces the same verification discipline as the implementation it governs (§8.10).

Specification is the act of ruling things out. Without constraints, any output from the AI's vast default distribution is valid — which means every output is arbitrary. As constraints accumulate — naming conventions, architectural boundaries, ADRs closing open decisions — the set of valid sentences shrinks. But the AI's ability to derive *the correct sentence for a given requirement* grows. Restriction is the activation mechanism. The output-space formulation is precise: every constraint removes a degree of freedom from the space of valid programs. The programs that remain after all constraints are applied are exactly the correct ones — a smaller space where every reachable point is right. The restriction is the expansion mechanism.

The AI's training corpus contains the full formal tradition of computer science. Without specification, the model defaults to what human practice historically permitted: the convenient shortcut, the informal approximation, the discipline abandoned under deadline pressure. The practitioner who names Hoare contracts or session types is not teaching the AI anything new — they are opening the door to knowledge the model already holds. The depth of the specification determines the depth of the formal tradition activated.

Schema-activated derivation. This mechanism has a structural analog in cognitive science. Bransford and Johnson (1972) demonstrated that a context label supplied before reading an ambiguous passage — “laundry,” “music,” “construction” — made it immediately comprehensible: the label activated the reader's pre-existing schema, supplying the interpretive framework that filled every ambiguous phrase with correct meaning. The same mechanism operates in generative development, with the direction inverted. Where schema fit aids *comprehension* (context cue → human reader fills in meaning from prior knowledge), spec-activated derivation aids *generation* (spec cue → AI executor fills in implementation from trained domain knowledge). The practitioner does not supply the domain knowledge; the model already holds it. The specification supplies the activation cue that selects the correct knowledge subset and suppresses adjacent patterns from the model's prior distribution. A bounded, self-describing specification produces a single tightly-constrained derivation; an incomplete specification produces a population of derivations — architecturally coherent by accident, if at all. This is why the Bounded and Self-describing properties have disproportionate weight in the seven-property rubric: they are the primary schema-selection mechanism. A specification that fails to bound its scope or describe its identity activates the wrong schema, or no schema at all, and the downstream derivation is arbitrary regardless of how carefully the remaining properties are satisfied.

Positional placement matters. Liu et al. (2023) demonstrate systematic accuracy degradation for information positioned in the middle of a long context window. The architectural constitution — placed at the leading position of every AI session — responds directly to this failure mode. The spec is the first thing the model reads.

The Chomsky hierarchy provides the structural analogy: as readers gain expressive power, the specification required to govern them correctly must become richer. Finite rules generating infinite valid outputs is the structural intuition. The practical implication is a single imperative: **assume nothing**. Every assumption is a gap the agent will fill arbitrarily, at generation speed, across every session that inherits the result.

Normative keywords (RFC 2119 / RFC 8174). Obligations are phrased with the standard normative vocabulary — **MUST / MUST NOT / REQUIRED / SHALL** (hard obligation), **SHOULD / SHOULD NOT / RECOMMENDED** (defeasible: deviation requires a recorded reason), **MAY / OPTIONAL** (permitted) — and, per RFC 8174, the words carry normative force only when capitalized, leaving ordinary prose unaffected. This is the lexical instrument of the prescriptive-versus-descriptive distinction the RND-1 pilot (§7.8.I) measures: a descriptive obligation (“the actor needs to see certain data”) leaves the stateless reader to resolve ambiguity toward the literal minimum; a normative one (“the dashboard **MUST** return counts aggregated by type, **MUST** compute active-members-in-7-days, **MAY** cache”) removes the degree of freedom the reader would otherwise fill arbitrarily. The vocabulary is closed — it serves Bounded and Self-describing, so obligation level is parsed deterministically rather than inferred — and it binds each clause to verification: every **MUST** is an acceptance criterion and therefore a probe (Verifiable), while the keyword sets the gate’s severity — **MUST** → **blocking gate**, **SHOULD** → **warning**, **MAY** → **ungated** (Defended). The discipline is selective, not total: keyword the load-bearing obligations — acceptance criteria, constraints, prohibitions — not every sentence; over-marking is the same harness excess that degrades any bounded artifact. It is the F-NNN format’s acceptance-criteria line (§5) made lexically exact.

4.1.b The Economic Consequence

The economic consequence of this structure is **cost inversion**: the cost of iteration approaches zero.

In a traditional development cycle, wrong output costs a sprint: read the code, identify the error, write a correction, review, wait for CI, merge. Coordination overhead accumulates at every boundary. Under GS, wrong output costs one sentence: identify the missing constraint, add it to the specification, re-run. The AI is stateless — it re-reads the complete specification and regenerates from scratch. No codebase navigation, no branch management, no review cycle for the specification change itself.

The residual is real: identifying *which* constraint is absent requires domain fluency. That cost is bounded by the practitioner’s own competence, not organizational friction. The process is self-correcting: each iteration makes the grammar more precise, revealing the next gap.

At portfolio scale, cost inversion shifts the binding constraint from execution capacity to *specification bandwidth* (coined here: the rate at which intent can be correctly externalized into a durable specification). Multiple projects cycle concurrently — a project in a waiting state (deploy running, output under review) requires no execution from the practitioner. Portfolio size is bounded by status management, not execution load.

Once a grammar is complete, derivation is mechanical — not because the model is intelligent but because completeness closes the space of valid outputs to those that are correct. The adoption dynamic follows every prior software abstraction: friction on entry, then dominance once the productivity margin

exceeds the residual cost. What differs is magnitude: the cost being settled is not a layer of implementation but the act of converting intent into implementation itself.

Cost inversion describes the price of *iteration*; a second economic effect governs the price of *each session*. The objection most often raised against specification-first development is token cost: authoring a specification, cascade documents, and a navigation tree appears to spend tokens that ad-hoc prompting does not. The objection measures the wrong quantity — not tokens generated, but tokens per correct, accepted output. A session without authored structure re-reads the codebase and re-derives its architecture on every invocation; the stateless reader pays the discovery cost repeatedly. A session with a bounded, self-describing specification reads the authored structure once and navigates it directly.

This is the same principle, independently quantified outside our domain. Yarmoluk and McCreary (2026) benchmark three knowledge-retrieval architectures and find that reading a pre-authored *compact knowledge graph* (CKG) — a small, enumerable DAG of concepts and dependencies — costs roughly an order of magnitude fewer tokens ($\approx 11\times$) at substantially higher accuracy ($\approx 3.8\times$ token-level F1) than retrieving from chunked prose (RAG) or re-deriving graph structure from text at query time (GraphRAG); they conclude that “when expert structure is available, the dynamic extraction step is wasted computation.” The GS sentinel is a member of the same family — an authored, compact, closed-vocabulary structure the reader consults instead of rediscovering — but it generalizes the CKG’s single relation. Where a CKG encodes one edge type, prerequisite order, to mark a human learner’s path, the sentinel encodes the AI executor’s operating path: navigation across the cascade documents, the dependency direction, the disciplines applicable at each location, and the constraints the executor must never violate — described for the assistant that will traverse it. The token expenditure practitioners attribute to GS is, more precisely, the expenditure of working *without* such authored structure, paid again at every session — and we have now measured this directly. The ForgeCraft KX replication (§7.8.E) ran the CKG benchmark method on a GS software harness across three retrieval conditions: a routed navigation tree (CKG-analog) reached macro-F1 0.808 at 78.6k tokens/query, beating both everything-in-context (RAG-dump analog: 0.611 F1 at 100k tokens) and code-search-at-query-time (no-structure analog: 0.431 F1 at 233k tokens) on accuracy *and* cost — up to $3.0\times$ cheaper per query than the unstructured condition (233k $\rightarrow$ 78.6k tokens), and cheaper than the in-context dump as well. The absence of structure was the single most expensive condition: the structureless agent burned up to 492k tokens on one query type, searching for conventions that did not exist, to score ≈ 0 . The CKG architectural divergence replicates on this new substrate — aggregate-enumeration queries scored 0.909 (routed) versus 0.006 (no-structure), mirroring the original benchmark’s 0.964 versus 0.054 — confirming that the advantage is a property of pre-structured retrieval itself, transferring from textbooks and pharmacology to software harnesses. As the benchmark’s own caveat applies (ground truth for structural queries derives from the same structure the navigation tree reads), the claim is the bounded one: *explicit structure beats inferred structure on structural queries*.

This locates the cost precisely. A GS session decomposes into three steps — *retrieve, generate, verify* — and only the second is the generation that prompting optimizes. The first, retrieval, is the assembly of the right context for the stateless reader, and it is a retrieval problem in the literal sense: without authored structure, the reader scans, embeds, or re-derives the architecture from the code on every session, the codebase-equivalent of RAG. Generative Specification attacks that retrieval from both ends. It *authors the structure* — the sentinel and navigation tree are the codebase’s compact knowledge graph (the bridge of §4.1), traversed by direct lookup rather than similarity search — and it *shapes the code to be retrievable*, since the structural disciplines (§5) make predictable placement, typed contracts, and intentional naming the conditions under which the reader’s first search succeeds. A code-search tool (e.g., a semantic/structural index) is the engine that traverses this structure; the disciplines and the

sentinel are what make the traversal cheap and exact. The third step, *verify*, is the harness — the test suite, the use-case walk, the NFR gates — and it is categorically distinct from retrieval: it does not assemble context, it checks generated output against the specification. Conflating the two obscures the economics. The retrieval upgrade — from RAG-grade scanning to CKG-grade lookup — is where the token differential of the preceding paragraphs is paid or saved; the harness is where correctness is established. GS is, in this framing, retrieval-augmented *and verified* generation, with the retrieval authored rather than inferred.

This split sorts the structural disciplines (§5) by function as load-bearing roles. One role serves the *verify* step — disciplines that **secure correctness and the system’s attributes**: test-driven and behavior-driven development, design by contract, type-driven design, and the enforcement gates, feeding the Verifiable, Defended, and Executable properties. The other three serve the *retrieve* step — the bridge — and they are distinct because retrieval is the harder, multi-faceted half. **Legibility** disciplines make the *what* readable (current intent): intentional naming, the SOLID interface boundaries, hexagonal layering, convention over configuration, and ubiquitous domain language, feeding Self-describing and Composable. **Bounding** disciplines leave *less* to read at all — a second, independent lever on token cost: not better navigation but a smaller surface to navigate. DRY and deduplication, dead-code elimination, the minimization of scope (YAGNI), and small units feed Bounded; minimal code is at once cleaner, more maintainable, and cheaper to read. **Decision-memory** disciplines make the *why* retrievable across time: architecture decision records, conventional commits, and engineering decision records, feeding Auditable. Each role has its enforcement mechanism — verification by hooks, CI, and the harness; legibility by the navigation tree and a structural code index; bounding by automated deduplication and dead-code detection over the dependency graph; decision-memory by the cascade documents. The KX experiment (§7.8.E) measures the payoff of the retrieve side; the AX-T8 run (§7.8.E) measures the payoff of the verify side. A discipline can serve more than one role (a type both verifies a constraint and documents it; Single Responsibility serves both legibility and bounding), so the grouping here is by *dominant* function, not exclusive membership — the §5 table carries the finer per-discipline property assignments. Naming the dominant role explains why the seven properties partition as they do, and why a codebase strong on verification yet weak on legibility and bounding still reads expensively.

Formally correct specifications previously failed at scale — REST’s hypermedia constraints, semantic web annotations, session types — because their annotation burden exceeded what human teams would sustain. The failure had six identifiable causes at two levels. *System-level*: (1) annotation fatigue — correct invariants existed but no team could maintain them under production pressure; (2) single-target economics — one annotation per language never paid for itself at the tool layer; (3) tooling fragmentation — type checkers, security auditors, dashboards, and configuration surfaces never unified into one reviewable artifact. *Practitioner-level*: (4) learning cost — no career was long enough to master the intersection of all relevant formal disciplines simultaneously; (5) maintenance erosion — disciplines known at hire degraded under deadline rotation and team turnover; (6) transfer loss — knowledge lived in people, not artifacts; when people left, so did the discipline. The AI executor eliminates all six simultaneously: it has no incentive to skip annotations (1), its training corpus covers every annotation for every target (2), it reads the unified discipline surface without fatigue (3–4), it applies every constraint consistently across sessions without erosion (5), and the specification artifact is the transfer mechanism, not the practitioner (6). Correct implementation becomes recursive: each correctly implemented standard makes the system more legible to the executor that built it, raising the quality floor for every subsequent generation.

The philosophical and civilizational consequences — why the formal tradition is now achievable at every scale, for every project — are developed in the companion essays *Onwards!* and *The New Golden Century*.

4.1.c The Convergent Principle

The grammar mechanism is specific to the relationship between three elements: a declared specification, an executor capable of producing outputs from it, and an observation mechanism capable of measuring the gap and triggering correction. Any system where all three elements are present operates under the same discipline: *the correctness of the outcome is a function of the completeness of the specification*. Its convergent form across other executor domains — industrial systems, autonomous vehicles, medical devices — is developed in §10 and *The New Golden Century*.

One consequence compounds forward: a correctly implemented GS system is more legible to the next stateless reader session that encounters it. The executor that generates correct output against a complete specification has, in doing so, made the system's formal properties visible in its own artifacts — correct naming, enforced boundaries, emitted decision records. Every subsequent session starts from a higher floor, because the reader of that output is the same kind of executor that produced it. Correct output is a recursive investment: it raises the quality of the next generation without any additional practitioner act.

4.1.d The Pedagogical Structure

GS does not instruct by exhaustion. The specification states what to build and why — intent and constraint, not procedure. The spec orients; the model derives. That division is the mechanism, not a limitation.

Correction arrives by exclusion. Quality gates name what is not acceptable and return the output — the cognitive apprenticeship model (Collins, Brown & Newman, 1989): the expert makes tacit knowledge visible through targeted correction, not prescription. Each gate is a named failure mode. As practitioners contribute gates derived from their own project failures, those failure modes propagate to every project that adopts the shared template. The floor rises (Dreyfus & Dreyfus, 1986: internalized failure modes become invisible rules; community-contributed gates encode that internalization structurally).

The validation strategy maps the same sequence. AX (Author-Executed): solo practitioner, unobserved. BX (Benchmark Cross-validation) and RX (Replication): calibration against independent implementations not shaped by the same assumptions, scored against criteria that predate GS. Observational field corroboration (§7.8.A) adds a practice-side check on a real team; a controlled human-participant study is noted as future work.

4.1.e Illustration: The Art Generation Pipeline

A strategy game concept given to the system as a narrative idea demonstrates the mechanism at a domain that has no prior GS tooling. The precision of the idea is the ceiling of everything that follows: a vague concept produces a generic game; a precise one — factions with named ideological conflicts, unit archetypes tied to faction doctrine, a color palette grounded in environmental lore — produces a specification the AI executes with fidelity. The AI derives toolchain selection, model configuration, quality constraints, and every generation step without human direction at any intermediate stage. The quality constraints *are* the specification: once they close the surface, every reachable output is a correct one.

A calibration phase is required before the pipeline reaches steady state: human review of sample outputs to determine whether the constraint set is sufficient. This phase can itself be automated — a vision-

capable model given the stated quality rules and a sample can evaluate conformance and return a structured gap analysis. The human’s final role is to decide the gap analysis is empty.

One surface this constraint vocabulary cannot close is aesthetic judgment: visual weight, compositional tension, emotional resonance. A constraint that closes symmetry and palette conformance produces a *technically correct* asset; whether it is compelling is a judgment the specification cannot make. The pipeline produces materials, not final deliverables — it collapses the distance between idea and revisable first form, which is precisely what an artist needs to begin. A failure mode specific to this structure — the AI building the sample artifact instead of the generative mechanism — is named and addressed in §8.15.

4.1.f The Six-Tier Obligation Cascade

Generative Specification does not improve how code is written. Applied fully, it removes six categories of work from the practitioner’s responsibility entirely. Each tier operates on two axes simultaneously: it adds a restriction to the system and removes an obligation from the practitioner. These are the same move stated from two directions. The formal constraint added to the system is precisely the mechanism by which the practitioner’s obligation dissolves — the restriction is the liberation. Both statements are required to characterize a tier fully: what the system now must satisfy, and what the practitioner is therefore no longer required to provide.

The cascade is organized by *lifecycle stage*: development, staging, production, evolution, synthesis, meta-telos. Each tier removes simultaneously an *authoring obligation* (what the practitioner no longer writes) and a *verification obligation* (what the practitioner no longer has to check by hand). The pairing is the load-bearing structural claim — the verification obligation can be removed because a stage-appropriate harness now certifies the derivation was faithful at that stage, and the authoring removal is safe only because that certification holds. In earlier formulations the verification harness was treated as its own tier, which obscured this symmetry; verification is not a phase, it is a cross-cutting capability that recurs at every stage with stage-appropriate tests.

The six tiers form an obligation cascade — what GS removes from the practitioner, step by step:

Empirically demonstrated tiers (T1–T4): empirically demonstrated across production deployments, the AX adversarial series, and the ALX self-application.

Tier	Stage	Obligations removed (authoring + verification)	Primary mechanism	Status
T1	Development	Write code; read or review generated code	Spec authoring + dev-time harness in one cycle: ForgeCraft gates, unit/integration/E2E tests, mutation testing, AI-as-QA visual confirmation, automated scoring	Demonstrated (production cases §7.1–§7.7; AX series §7.8.B; ALX: S_realized=1.0, 386/386, evidence)
T2	Staging / Pre-prod	Touch deployment; manually validate the staged system	Config/CLI-driven CI/CD across CAE/LTE/PRD environments; staging-stage harness — NFR thresholds, integration smoke tests, load/security/gateway automation against the real environment	Demonstrated (Chronicle/Railway, §7.7)
T3	Production	Monitor; diagnose bugs	Chronicle signals + CodeSeeker; production-stage harness — drift detection, runtime contract verification, automatic anomaly detection and correction	Demonstrated (COMPASS ETL, §7.7)
T4	Evolution	Manually maintain and extend the living system	Bio Iso reads telos and signals; evolution-stage harness — the governed mutation gauntlet, senescence, self-improvement	Demonstrated (Loom colony, github.com/jghiringhelli/loom)

Research-frontier tiers (T5–T6): architecturally specified; empirical demonstration is the next validation phase.

Tier	Stage	Obligations removed (authoring + verification)	Primary mechanism	Status
T5	Synthesis	Design the system architecture; review the colony	Axon derives a colony of T4-governed systems from a single problem statement; colony-level harness — each entity validates its own telos against typed inter-entity channels	Designed (Axon)
T6	Meta-telos	Initiate the process	Meta-telos observes the practitioner across history and surfaces candidate intents before they are stated	Research agenda

T1–T4 are empirically demonstrated across production deployments and the Loom colony simulation. T5 is architecturally complete; empirical demonstration is the next validation phase — it is not a near-term availability claim. T6 is the logical terminus — stated as a research agenda rather than an implementation claim; governance must precede capability.

What is not a tier: Loom is a language layer that cuts across T1–T4. It is not a rung on the cascade; it is the medium through which the cascade eventually operates at the compiler layer. Full treatment is in the companion Loom paper.

Philosophical and civilizational frames (not tiers): Nous/Logos (developed in the companion essay *Onwards!*) grounds why T1–T4 are structurally achievable now — the Logos that can hold the Nous without degrading it across sessions. The Golden Century (§8.8) names the civilizational consequence when T4–T6 complete. *Attention is All You Have* frames what remains uniquely human when T1–T6 are operational: the capacity to direct attention, not the burden of execution. Ambient Engineering (§9.2) names the interface mode at T5–T6 — the environment is the computer; intention accumulates without session cost. These frames are not tiers. They ground, contextualize, and project the cascade.

The Generative-X lexicon: naming what the executor performs at each tier

The tables above name what each tier *removes* from the practitioner. A parallel naming is required for what the executor *performs* at each tier, because the obligation-removal framing is silent on the operation that replaces the practitioner’s hand. The shared root is *generative*: at every tier the executor derives an artifact (code, deployment, alert, mutation, ecosystem) from a specification rather than executing a procedure under human direction. The suffix names the kind of artifact derived at that stage, and the verification side names how its faithfulness is certified.

Tier	Authoring operation	Verification operation	Reference implementation
T1	Generative Specification — code, tests, documentation, AI-behavior files derived from spec	Generative Execution — pyramid of tests (unit, integration, E2E, mutation, contract, NFR) plus multimodal-AI-as-QA against the live app, plus a pre-merge hardening pass	ForgeCraft gates + Hurl + multimodal AI scoring (§7.7)
T2	Generative Deployment — CI/CD pipelines, environment configuration, infra-as-spec, deployment manifests	Generative Validation — NFR contracts (latency, throughput, error budgets), staging smoke + load + security, compliance-policy gate as type-level constraint	Railway/Chronicle deployment chain (§7.7.1)
T3	Generative Monitoring — alerts, dashboards, thresholds, structured logging derived from spec	Generative Diagnostics — drift detection against spec-derived runtime contract, automatic root-cause analysis, signal-to-spec feedback	Chronicle + CodeSeeker + forgecraft-eye (§7.7.2)
T4	Generative Evolution — governed self-mutation: proposal of spec amendments from observed production behavior	Generative Gauntlet — the full T1–T3 harness chain admits only mutations that pass at every prior tier; telomere bounds enforced	Loom colony, public repository [[GS T4] - tagged commits]
T5	Generative Synthesis — derivation of an interacting ecosystem of T4-governed entities from a problem statement	Generative Colony Validation — each entity’s own T1–T4 harness chain plus cross-entity contract enforcement at typed channels	Axon (designed)
T6	Generative Cognition (<i>research horizon</i>) — intent inference before statement	<i>Irreducible — judgment layer</i> — practitioner ratification is structurally required	Research agenda

Three properties of this lexicon are worth noting because they are load-bearing for the rest of this work:

1. **The verification operation at each tier is what makes the authoring operation safe at that tier.** Generative Deployment without Generative Validation is the most dangerous state in the cascade — automated deployment past the ability to verify it. The pairing is a structural condition, not a tooling preference.
2. **Hardening is not a separate generative operation.** It is the load-bearing capability that recurs at every tier’s verification side under stage-appropriate scope: T1 hardening is the pre-merge pass; T2 hardening is the pre-promotion gate; T3 hardening is continuous; T4 hardening is the mutation gauntlet itself. Naming “Generative Hardening” as a peer of Generative Deployment would over-fragment the lexicon. Each tier *has* a hardening capability.
3. **Dynamic testing is realized under three different generative-X names** — Generative Execution at T1, Generative Validation at T2, Generative Diagnostics at T3 — because the same primitive (derive expected behavior from spec, observe actual, compare, signal) runs at three different timescales with three different consequences-of-mismatch. The shared structure is what

makes the cascade a cascade; the different names are what allow the verification surface at each tier to be implemented and reasoned about independently.

The per-tier prose that follows uses these names interchangeably with the obligation-removal framing — the obligations are what the practitioner notices when GS arrives; the generative operations are what an external observer measures.

A note on convergent external-verification work. Shumer’s *Gauntlet Loop* (2026) arrives independently at the verify side of this cascade from the agent-orchestration direction: a *builder* agent produces work, a *critic* agent with fresh context compares it against a concrete reference and names the single largest remaining gap, and the loop repeats until a quality criterion — not a fixed round count — is met. The builder/critic separation, the fresh-context critic, the insistence on inspecting the actual artifact (pixels, code, test output) rather than a summary of it, and the loop-until-satisfied stopping rule are the same discipline this work applies as *generative execution* and its multimodal external judges (Tier 1 above). The instructive difference is the *reference standard*. The Gauntlet critic compares against “the bar” — an exemplar of what excellent looks like — whereas GS verification compares against the *specification*: the contract that defines *correct*, not merely *better*. In GS terms “the bar” is a few-shot exemplar, and by the bridge’s account (§4.1) an exemplar earns its keep only where a property has not yet been — or cannot be — stated. That is precisely its domain: subjective, aesthetic goals that resist formal specification, where the GS contract is weakest and an inspectable exemplar is strongest. The two are therefore complementary rather than competing — the Gauntlet supplies a disciplined verify loop for the quality gradient a spec cannot close; GS supplies the correctness contract an exemplar cannot state — and a recognized practitioner converging on the same external-judge loop is third-party corroboration that the operative mechanism is *verification*, not *proof*. The terminological echo is not accidental: T4’s governed *mutation gauntlet* (above) is the same loop turned inward on a self-mutating system, admitting only changes that survive every prior tier’s judge.

Tier 1 — Development: you do not write code, and you do not read what was generated.

The specification is the industrial blueprint — the source from which every implementation is derived and re-derived on demand — and the dev-time harness is what makes the second removal safe. The testing disciplines (TDD, integration, E2E, mutation, contract, non-functional) each specify a distinct class of behavioral obligation: TDD per unit, integration at boundaries, E2E for telos fulfillment, mutation for detection completeness, contract for distributed agreements, NFRs as executable thresholds. A spec including only TDD has left every other class implicit — which is where the failures are. The executor holds every formal discipline in its training corpus without fatigue, erosion, or transfer cost; the structural files are the mechanism by which every new session starts from the same enforced position. The dev-time harness — ForgeCraft gates, Hurl, multimodal AI-as-QA, automated scoring — verifies every executable obligation against the running system without a human reading the generated code. If validation fails, the specification is tightened and T1 regenerates. Multimodal extension closes the last category previously requiring a human reviewer: a screenshot captured by Playwright is compared against the use-case visual postcondition by a vision model; pass/fail is automated. The MVC walkthrough (companion essay: `docs/essays/the-flea-game.md`) and the COMPASS data platform demonstrate the full T1 harness: pre-execution database state capture, Playwright drives the UI, service-layer log inspection, post-execution state diff, multimodal UI confirmation, independent API endpoint tests via curl/Postman.

Tier 2 — Staging / Pre-prod: you do not touch deployment, and you do not manually validate the staged system. CI/CD pipelines, deployment manifests, environment configurations — governed by the same specification as the code. Compliance policy (HIPAA, PCI-DSS, GDPR, SOC2) is

enforced as a deployment-gate type-level constraint, not a manual checklist reviewed at launch. The practitioner never issues a CLI command or edits an infrastructure file. The staging-stage harness extends the harness pattern from T1 into the deployed environment: NFR contracts (latency, throughput, memory, security) become executable thresholds; integration smoke tests fire against the real environment with real services; gateway, load, and security tests execute automatically before any promotion to production. Validation that previously required a human walking through the staged build is now an automated certification.

Tier 3 — Production: you do not monitor, and you do not diagnose bugs. Logs, signals, and runtime observations are evaluated against the same formal properties that drove construction. Drift from the specification is a specification violation, detectable and correctable by the same derivation mechanism that built the system. The production-stage harness extends the pattern again: drift detection runs continuously, runtime contract verification compares observed behavior against the spec’s behavioral promises, and discrepancies trigger structured root-cause analysis rather than human paging. Demonstrated in COMPASS/The Eye (§7.7).

Tier 4 — Evolution: you do not manually maintain or extend the living system. Self-maintaining, self-renewing formal systems whose lifecycle is governed by the specification, with manual maintenance removed because the evolution-stage harness — the governed mutation gauntlet — admits only mutations that pass the harness chain at every prior tier. The Loom colony demonstrates T4 in operation: governed genome mutation, senescence, and self-improvement are running against the Loom codebase, with auto-applied mutations committed under the `[GS T4]` tag in the public repository (github.com/jghiringhelli/loom). The theoretical framework is at bioiso.dev; the formal treatment of telos as fitness grammar, mutation triggers, and reproduction protocols is in the companion paper *Biological Isomorphisms in Formal Self-Maintaining Systems*.

Tier 5 — Synthesis: you do not design the system architecture. The practitioner states a problem. A stateless reader derives an interacting set of T4-governed programs — each with its own derived telos, interacting through typed channels, dying when their telos is fulfilled. The colony-level harness is each entity’s own T1–T4 harness chain applied to itself, with cross-entity contracts enforced at the channels: the system as a whole admits only configurations where every member can certify its own derivation. The practitioner is no longer the system architect; they are the problem-holder. Formal prerequisites and colony simulation are specified in Loom’s milestone roadmap and the *Bio Iso* companion paper.

Tier 6 — Meta-telos: you do not initiate the process. A system that has observed the practitioner across their full history infers what is needed before it is asked, and the meta-telos verification is the practitioner’s own ratification — observation surfaced as a candidate intent that must be accepted before any action is taken. T6 is noted as the logical terminus — stated as a research agenda, not an implementation claim. Governance must precede capability; what T6 requires is not more formal theory but a careful answer to who decides what the practitioner needs before any autonomous inference becomes action.

On the role of the harness. The specification establishes intent; it does not certify that the derivation was faithful. That certification is the harness’s function. The harness is not a tier — it is a cross-cutting capability that recurs at every tier with stage-appropriate tests: dev-time at T1, staging at T2, production runtime at T3, evolution-time (the mutation gauntlet) at T4, colony-level at T5. A specification without the verification harness for the relevant tier is an assertion, not a guarantee: the claim “spec is the

program” holds structurally only when the behavioral contracts from the specification are continuously verified against the running system at every stage that matters. The harness is what closes the derivation loop — it is as constitutive of the GS guarantee as the specification itself. This is why each tier’s harness is not optional scaffolding but a structural requirement of that tier; removing it degrades the paradigm claim from a guarantee to a discipline preference.

The six tiers are not independent. T1’s harness requires the spec’s use cases be formal enough to test against. T2’s compliance gates and NFR thresholds require T1 data flow labels and contracts to compare against. T3 drift detection requires the T2 staging harness to have established the runtime contract baseline. T5 synthesis requires T4 self-maintenance. Specification quality is the only constraint determining how far the cascade runs without the practitioner.

On cascade refinement. The tier hierarchy is conceptual; the implementation loop is recursive. A failure detected at T2 or T3 does not necessarily indicate a T2 or T3 defect: it frequently indicates a T1 gap — an NFR not stated, a data flow not labeled, a contract left implicit — whose downstream consequence became visible only at the higher-stage harness. The correct response is to return to T1, close the gap, and re-run the cascade from that point. This non-linearity is not a weakness of the method; it is the expected diagnostic behavior of a system in which all higher tiers derive from the specification. T2 and T3 failures are detectors; T1 is almost always the site of correction. A complete NFR register at T1 is therefore not a documentation exercise but a prerequisite for T2 and T3 enforceability.

On the judgment layer. The cascade removes obligations the practitioner previously executed; it does not remove the obligations that depend irreducibly on human judgment. Domain expert validation (does the AI’s interpretation of finance, law, medicine, game design, music match what a real expert would do?), edge-case discovery from lived experience, aesthetic and quality judgment, strategic and business decisions about what should exist at all, compliance and legal sign-off, real user research with real humans, and performance tuning at production scale — these constitute the **judgment layer**. They are not a tier within the cascade because they are not a tool layer; they are the irreducible human work that runs at human pace because it must. Practitioners working under GS report that the judgment layer feels slow only by contrast to the AI-paced work that precedes it. This is a perception artifact, not a methodological gap: 90% of work that previously consumed weeks now resolves in hours, leaving the 10% that has always required judgment to occupy proportionally more of the practitioner’s attention. GS is honest about this: the discipline does not claim to replace human judgment. It claims to ensure everything before the judgment layer is correct, so that judgment is spent on what it alone can decide.

4.2 The Closed-Loop Cascade

The six tiers describe what GS dissolves; the closed-loop cascade describes how it stays dissolved. The tier hierarchy is the obligation map; the cascade is the enforcement substrate that prevents the map from drifting away from the territory between sessions. Without the cascade, the seven properties are aspirations. With it, they are admissibility conditions on every change that enters the repository.

A working GS project is not a sequence of separate practices stitched together; it is a single closed loop in which every layer derives from the layer above and feeds observations back into the layer above. The substrate the loop runs on is the cascade, and the cascade runs on a single operative rule:

A change is admissible if and only if the layer immediately above it in the cascade has been amended to explain it.

The rule is biconditional. A code change unaccompanied by a corresponding upper-layer amendment is inadmissible; an upper-layer amendment without an immediate downstream change is permitted, because the spec is the only artifact whose change requires no further upstream explanation — it is the cascade’s apex, governed by judgment rather than by another layer. This produces the asymmetry the paradigm depends on: specification leads, implementation follows, and the loop closes only when judgment ratifies the result.

Conventional commits as the trigger surface

The cascade requires a parsing surface — a place at which the system can mechanically determine which subset of the rule applies to a given change. Conventional Commits provides it. Each commit message of the form `<type>(<scope>): <description>` carries a type that maps deterministically to a layer obligation:

Commit type	Required upper-layer touch	Encouraged additional touch
feat:	spec amendment	use case, schema, ADR if architectural
fix:	regression test	decision record if behavior was intentionally redefined
refactor:	—	ADR or decision if an architectural choice was made
perf:	—	decision and benchmark
revert:	—	decision
docs:, test:, chore:, ci:	—	—

The mapping is enforced at two boundaries: at commit time by a hook that inspects the staged diff against the declared type, and at integration time by a continuous-integration gate that re-runs the same logic against the full pull-request diff against base. Severity ramps from advisory (warning) on day one of brownfield adoption to blocking (error) once the project’s baseline is clean. The progression is structural — the same rule, applied at increasing rigor as the specification stabilizes.

The commit type is therefore not an annotation on history; it is a declaration the cascade reads to determine which subset of the rule to apply. A `feat:` commit declares “this change has spec implications,” and the cascade gate verifies that declaration by inspecting whether spec artifacts are part of the same commit. A misclassified commit — a feature labeled `chore:` to evade the rule — is detectable at the public-surface layer (below) regardless of the type the practitioner chose.

The three-layer recording architecture

The cascade operates on a substrate of three independent recording layers, each with declared ownership and formal propagation rules:

Layer	Owner	Holds	Persists across
Project	the project-layer record-keeper (repository state)	specs, ADRs, decisions, use cases, roadmaps, schemas, contracts, hooks, gates	sessions, developer turnover, calendar
Individual	the individual-layer memory (per-developer store)	session-local prompts, findings, work patterns, personal preferences	sessions for one practitioner
Team	the team-layer aggregator (shared store)	shared findings, ticket-to-spec mapping, cross-project patterns, prompt analytics	the whole organisation, all projects, all practitioners

The layers are independent in implementation — no SDK couples them — but they propagate at every boundary. A finding at the individual layer can promote to the project layer (as an ADR or decision in

the repository); a project-layer pattern observed across multiple projects can promote to the team layer; a structural change at the team layer cascades downward to every project that inherits the policy. The propagation is asymmetric: useful patterns surface upward through judgment one at a time; team-layer policy descends uniformly.

The integration contract between the three layers is a single per-project file: the **manifest** (`docs/manifest.yaml`). It declares which document types live in which paths, which commit types require which layer touches, what the public-surface detection rules are, and which tool owns which recording layer. Every other tool — gates, hooks, dashboards, analytics — reads the manifest. The manifest is the only required contract; the rest is implementation. The narrative form of the three-layer architecture, with its biological framing, is developed in the companion book; the compendium's version is the formal one above.

The public-surface diff rule

A change to exports, public types, command-line flags, or tool schemas exposed across an inter-process boundary requires a spec or ADR touch *regardless of commit type*. This rule closes a loophole the type-driven cascade alone leaves open: a `chore:` or `refactor:` commit can carry a behavior change in disguise, smuggled across the type-to-layer map by mislabeling. The public-surface rule is a second classifier — a structural one — that operates on the diff itself rather than on the practitioner's declared type. It triggers on a small set of declared globs (the project's API-surface block in the manifest) and demands an upper-layer touch whenever any path in that block is modified.

The result is a redundant gate: the commit type names what the practitioner intends; the public-surface rule names what the change actually does. When the two disagree, the cascade flags it. The loophole becomes structurally unreachable, not merely discouraged.

The judgment layer as the cascade's terminus

The cascade is the substrate the AI executor operates within; the terminus where it discharges is the judgment layer named in the previous note. The cascade preserves judgment by routing every change through it: no merge to a protected branch is admissible without (a) the cascade firing cleanly, (b) all gates green, and (c) explicit human ratification recorded as a review, comment, or approval. Branch protection enforces this at the version-control layer; a manifest-driven gate validates it at the project layer. Together they form a redundant checkpoint that an AI cannot route around — an AI cannot create approvals on its own pull requests, and the protected branch refuses merge without them.

The cascade's design is to ensure human judgment enters at this terminus and only here. Everything before — specification reading, use-case decomposition, schema authoring, code generation, test execution, harness measurement, observation structuring — runs at AI speed. The decision is what cannot be automated, and the cascade is the mechanism by which everything else gets out of the way of the decision.

Anti-drift formalized

The anti-drift property the cascade enforces can be stated as a single formal claim:

A published specification is admissible if and only if every code path it claims to govern is reachable through the cascade from the specification itself.

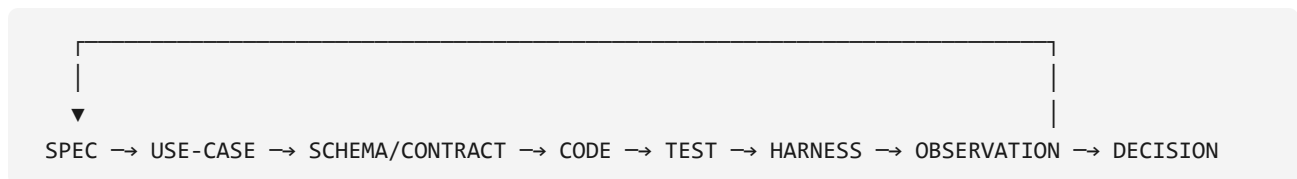
This is a stronger statement than “the specification governs the code.” It requires that the path from spec to code is *closed*: every executing module is traceable to a spec artifact through the cascade chain

(spec → use case → schema/contract → code → test → harness), and every spec assertion has a downstream realization the cascade can reach. A specification that claims to govern a module unreachable through the cascade is, to that extent, fictional — its claim is unverifiable by the substrate that would have enforced it. The closure property is what distinguishes a generative specification from documentation: documentation can describe code it has no path to; a generative specification cannot, by construction.

The reachability test is implementable in both directions. Walk the cascade forward from each spec assertion and verify the chain terminates at executing code; walk it backward from each module and verify the chain terminates at a spec assertion. Gaps in either direction are drift made visible. The cascade does not eliminate drift — it converts drift into the kind of failure a check can catch, rather than the kind that surfaces as an incident months later.

The closed-loop diagram

The shape of the loop, stated minimally:



Each downward arrow is a derivation governed by the admissibility rule above; the closing arrow from decision to spec is judgment ratifying direction. The diagram is not a sequence of stages to be executed once; it is the topology the cascade enforces on every change. A change enters at the layer its commit type names and propagates downward; the observation feeds back into the next decision, which becomes the next spec amendment.

The operational details — the precise hook chain, the manifest schema fields, the severity-ramp policy, the brownfield-override mechanism — are matters of implementation. They are documented in the **Practitioner Protocol** companion document, not in this compendium. The compendium’s claim is the structure above: that GS is enforced as a closed loop, that the loop’s substrate is a cascade, that the cascade’s substrate is a three-layer recording architecture with a manifest as integration contract, and that judgment is the terminus the cascade is designed to preserve attention for.

4.3 The Three-Tier Taxonomy

Throughout this section, syntactic and semantic are used in the programming-language sense: syntactic = pertaining to the form and structure of source artifacts; semantic = pertaining to the meaning those artifacts communicate to a reader who brings interpretive context. This is consistent with the Morris semiotic tripartition cited in §4 and with standard usage in programming language theory, but differs from the technical senses these terms carry in formal linguistics.

The Syntactic and Semantic Tiers

Syntactic disciplines (Martin’s three paradigms — structured programming, object-oriented programming, and functional programming, described above — and structural schema such as clean architecture, a layered design pattern that separates concerns into concentric rings: domain entities at the center, application logic surrounding them, infrastructure at the outermost edge) constrain the *form* of source artifacts: what constructs are permitted, what dependency directions are allowed. Whether

every principle widely discussed as a paradigm falls neatly into this tier is a taxonomy debate this work notes for completeness and takes no part in; the productive claim is directional: these disciplines constrain *what is permitted in the artifact*.

Semantic disciplines (SOLID — five principles for structuring object-oriented code: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion — test-driven development, domain-driven design, behavior-driven development, conventional commits) constrain the *meaning* that structure communicates to a human reader who brings context to the interpretation. A SOLID-violating codebase compiles; its cost is paid by engineers who recognize the deficit. TDD (Test-Driven Development — the discipline of writing a failing test before writing the code that satisfies it) makes a codebase *certifiable*: it removes the option of shipping unproven code, and the enforcement layer that makes it a discipline rather than a preference is real (CI — Continuous Integration — gates that automatically run tests on every commit, coverage requirements, deployment blocks). These disciplines assume a reader with state: colleagues, institutional memory, interpretive context built over shared history. TDD occupies the boundary between this tier and the pragmatic: a test suite is the closest prior art to a stateless, machine-readable behavioral contract, and TDD’s verification posture is carried directly into GS as its Verifiable property (§4.4). What places TDD in the semantic tier is its incompleteness as a derivation grammar, tests certify behavior but leave architecture, naming, decision history, and rationale implicit. GS subsumes TDD rather than extending it.

The Pragmatic Tier

Generative Specification is a programming discipline of the pragmatic dimension, the first to name the obligation to make the lifecycle layer derivable for a context-sensitive stateless reader. It constrains not what is constructed and not what communicates to a reader with context, but what is *derivable by a reader with zero context*: no colleagues to ask, no institutional memory persisting across sessions, no informal channels through which intent can travel. Every intent that would previously have been resolved through shared knowledge must be externalized as formal artifact, because the channel through which shared knowledge travels does not exist for the stateless reader. The pragmatic tier had no prior occupant not because the distinction was unrecognized, and not because stateless readers did not exist, IDLs and formal specification languages are both stateless by design, and they predate LLMs by decades, but because no widely-deployed stateless reader *of the pragmatic kind* existed: one deployed to read lifecycle intent and derive from it what should be built, where, and why, without requiring a human to navigate the gap between the specification and the implementation. IDLs (Interface Definition Languages — specifications like OpenAPI that define what messages a system boundary accepts, without addressing how or why the system is built as it is) read interface contracts. Formal specification languages verify property invariants. Neither reads the lifecycle layer. The transformer architecture produced the first widely-deployed reader that does, and with its deployment, leaving the lifecycle layer implicit changed from a recoverable cost paid by skilled humans to a structural failure propagated at generation speed.

The lifecycle layer is the subject of the derivability obligation (coined here; the nearest established concept is specification completeness, as formalized by Parnas in 1972) GS states. It comprises: *architectural identity*, what the system is, how it is structured, and why; *evolutionary intent*, which directions of change are valid and which violate structural invariants; *quality contracts*, the behavioral, performance, security, and compliance obligations the system must satisfy; and *decision history*, what alternatives were considered and why they were rejected. The lifecycle layer excludes the type system, the test suite, and the source code, those belong to the syntactic and semantic tiers respectively. A codebase that satisfies SOLID and has full test coverage is syntactically and semantically specified; it is

not lifecycle-specified if an executor without institutional memory cannot determine from its artifacts alone whether a proposed change is architecturally valid.

On the Semiotic Origin of the Taxonomy

The syntactic/semantic/pragmatic trichotomy originates in Charles Morris's 1938 *Foundations of the Theory of Signs* — settled semiotic theory. Applying it to classify the obligation structure of programming discipline is this work's proposal: a theoretical frame, not a claim within semiotic theory itself. The assertion that GS opens the pragmatic tier rests on the structural observation above — that no prior discipline stated the obligation to make the lifecycle layer derivable for an executor carrying no prior session context — not on the taxonomy alone. The classification provides orienting vocabulary; the taxonomy debate over which existing disciplines fall in which tier is noted and left open. (Jim Gray's 2009 *The Fourth Paradigm: Data-Intensive Scientific Discovery* uses a paradigm count for scientific methodology — a distinct domain with no overlap in argument.)

Prior Occupants of the Pragmatic Surface

Interface definition languages (IDLs — OpenAPI, wire protocol schemas, RPC definitions) are stateless specifications designed for machine readers, but they operate at the interface layer: they define what crosses a boundary. A stateless IDL consumer can call an endpoint; it cannot determine whether that endpoint *should* exist or whether adding it violates an intentional boundary. Formal specification languages (TLA+, Alloy, Z notation) are stateless and system-level, but they are property verifiers, not derivation grammars: they establish that a design satisfies a stated invariant; they do not generate the naming convention, the module boundary, or the decision record the AI reads before implementing. More fundamentally, they were designed for a deterministic, rule-bound verifier — TLC, the Alloy Analyzer — that checks whether an explicitly modeled finite state system satisfies a stated logical property. That reader cannot reason about evolutionary intent or the direction of valid change, not because it is underpowered but because those questions are not expressible in the language it reads. The formal verifier and the context-sensitive natural language reader differ in kind. GS is designed for the second. No prior discipline was.

The Derivability Obligation

This discipline becomes visible when its failure mode becomes undeniable: The accumulated cost of leaving intent implicit in AI-assisted development (architectural drift produced at generation speed, propagating silently across every session that inherits a corrupted context) has been independently measured. Orlanski et al. (2026) find that structural erosion rises in 80% of AI agent trajectories and that prompt-intervention improves initial quality but does not halt degradation — “current agents lack the design discipline iterative software development demands” (§3.5).

The concept has roots in classical software engineering. Parnas (1972) established that a well-decomposed system should make every design decision locatable by inspection: a reader with access to the specification should be able to derive the intended behavior without consulting the implementation. Jackson (2001) extended this with the Problem Frames approach: the specification must be sufficient to bound the problem, or the implementation will fill the gap arbitrarily. The derivability obligation is the GS instantiation of both principles, applied to the AI generation context.

The pragmatic tier has a specific failure mode that names it. A system that leaves context implicit is not merely poorly documented: it is *underspecifiable*: an agent with no persistent context cannot derive correct output because the grammar is incomplete. The consequence (architectural drift at generation speed) is structural, not stylistic. The distinction between the semantic and pragmatic tiers is therefore

not one of intensity but of kind: semantic disciplines produce worse systems when violated; a pragmatic violation produces a grammar the context-free executor cannot parse, the failure is not a quality deficit at higher intensity but a derivability collapse.

A system has achieved generative specification when its artifact set is designed so that any AI coding assistant, given access to those artifacts alone, has what it needs to: correctly identify what should and should not change for any given requirement; produce output that conforms to the system’s architectural, quality, and behavioral contracts; and detect when any existing artifact violates those contracts.

Whether a given AI model succeeds in practice is an empirical question about the model. Whether a given artifact set satisfies this design criterion is a structural question about the specification, answerable by inspection against the seven specification properties below.

Generative specification is a stronger property than “well-documented code.” Documentation can be narrative and passive: it can exist in a README that three people have read and that the AI session will never be given. Generative specification is active: the artifacts are themselves executable, verifiable, and self-correcting. The distinction is operational: a system cannot violate a generative specification without a mechanism triggering.

4.4 The Seven Specification Properties

The seven properties below are to Generative Specification what SOLID is to object-oriented programming: a named, teachable set of obligations that makes the discipline concrete, inspectable, and transferable. SOLID tells a developer how to structure objects for a human reader who brings context and judgment. These seven properties tell a practitioner how to structure specifications for a stateless reader who brings neither. Each property names a specific failure mode observed in production across six projects — not a taxonomy constructed in advance, but a record of what breaks and why. Together they operationalize the derivability obligation (§4.3): make each class of failure structurally unreachable. The rubric derived from them — 0/1/2 per property, 14 points total — is the primary measurement instrument of the validation experiments in §7. The rubric’s reproducibility — that two independent assessors arrive at the same score given the same artifact set — is grounded by **calibration anchors**: per-property reference exemplars at each scoring level (0, 1, 2) that fix the rubric’s interpretation through shared cases rather than abstract definitions. A score is admissible only when the assessor can name the anchor it is closest to. The anchors are the mechanism that makes the rubric transferable across projects, evaluators, and time. The full per-property anchors at each level (0/1/2), each property’s **lifecycle and organizational manifestation** (how it shows up across dev → staging → production → evolution and toward the world, the company, and the team — the dimension on which this rubric differs from a static design rubric such as SOLID), and the pathology→remedy mappings are developed in the companion **Rubric Scoring Guide** (`docs/white-paper/GS_Rubric_ScoringGuide.md`); it also defines the closed-key YAML frontmatter schema by which harness documents declare their rubric-relevant metadata for machine consumption (Self-describing made machine-readable).

Self-describing. The system explains its own architecture, decisions, and conventions from its own artifacts. No external knowledge is required. Self-describing addresses the *rationale layer*: not just what the structure is, but why it is that way and what rules govern it. It extends the Single Responsibility Principle (Martin, 2002) from runtime modules to the full artifact surface: specification, tests, and architecture documents are each responsible for one concern and contain what a reader carrying no prior session history needs to understand that concern. Automatable checks: (1) presence of an explicit

intent statement; (2) presence of a scope boundary statement. A specification lacking either fails regardless of prose quality.

Bounded. Every unit of work has explicit scope and seams. Functions do one thing. Modules own one concern. The line limit carries a mechanical justification: AI tool read operations are capped at a fixed line budget — a file that exceeds it is silently truncated, and the agent edits against an incomplete view. A specification artifact that exceeds the tool’s read budget is, from the executor’s perspective, equivalent to one that does not exist. Anthropic’s MEMORY.md index is hard-capped at 200 lines for exactly the same reason: the startup context cannot load what exceeds it. The 300-line specification limit and the 200-line memory index cutoff are the same constraint at two layers. Bounded addresses the *structural layer* — the distinction from Self-describing: a well-annotated system with blurry module boundaries fails Bounded; a structured system with no architectural constitution fails Self-describing. The property operationalizes Parnas’s information hiding principle (1972) at the specification level.

Bounded: The Sentinel Navigational Tree. At scale, Bounded applies to the session context itself. Loading all specification artifacts into a single context window produces the same pathology the property prevents at the file level — and, per Liu et al. (2023), degrades accuracy for information not near the leading position while consuming token budget on context the current task does not need. The structural solution is a **sentinel navigational tree**: a hierarchy of specification files where each node declares its own scope and routes to children. The root is always loaded (must stay within the bounded line limit); the AI descends only the path relevant to the current task. The tree is lossless — joining all leaf nodes yields the full specification — but each session receives only the slice it needs, eliminating both degradation and unnecessary token cost. Every well-formed tree must collectively contain five categories:

Category	What it covers
Architectural identity	What the system is, scope boundary, ADR index
Standards	Naming, commit discipline, quality gate thresholds
Constraints and prohibitions	What must not happen; boundary violations the AI must refuse
Tool sequencing	When to use which tool, in what order — not “these tools exist” but “use X before Y when C”
Routing	What each child covers and when to descend

Tool sequencing is the most commonly absent and most consequential gap. A spec that lists tools without stating when to prefer one over another forces unreliable inference.

Each node’s declaration — its scope, load policy (`always` for the root, `on-demand` for the rest), which of the five categories it contributes, and its routing targets — can be carried as **closed-key YAML frontmatter** (the `sentinel-node` schema in the companion Rubric Scoring Guide) rather than asserted in prose. This turns the tree from a prose convention into a machine-navigable, *verifiable* structure: a gate can confirm the root stays within the bounded line budget, that the five categories are collectively present across nodes (failing a tree with no tool-sequencing node), and that every `routes_to` resolves — the Bounded property made checkable. ForgeCraft’s sentinel renderer is the natural emitter and validator; hand-authored trees adopt the frontmatter once a consumer reads it (the minimal-sufficient rule — metadata no tool consumes is itself harness excess).

Verifiable. The correctness of any output can be checked without human judgment. Types, tests, lint rules, coverage and cyclomatic-complexity gates, mutation testing, dependency-vulnerability scans, and schema contracts — the standard static-analysis toolchain, from `tsc` and ESLint to quality-gate platforms such as SonarQube — form a continuous verification layer. The structural necessity of this

property is visible at the tooling layer: AI coding agents report file-write success when bytes reach disk, not when the resulting code compiles. The success signal is write-completion, not semantic validity. Without an explicit verification layer, the agent’s “done” and the developer’s “done” refer to different states. Verifiable closes this gap structurally: the verification layer is not optional post-work, it is the definition of completion. Verification is automatic, fast, and blocking; not aspirational. In a GS context, the test suite carries an adversarial role: tests are written against interfaces, not implementations — to detect violations of the contract, not to confirm the current implementation. A test that verifies internal state fails on correct refactors and passes on behavioral violations that preserve internal structure. Load tests, penetration tests, and chaos probes are specifications of adversarial conditions with explicit acceptance thresholds (§8.11). Verifiable establishes that the check infrastructure exists; whether the implementation passes those checks in a real execution environment is the Executable property, scored separately.

Defended. Destructive operations are structurally prevented rather than merely discouraged. Commit hooks, branch protection rules, format enforcement, and MCP tool boundaries make certain classes of mistake architecturally unreachable. The system rejects malformed input the way a parser rejects a syntax error. The property formalizes what is informally called defensive programming and CI/CD hardening (Forsgren et al., 2018 DORA metrics) into a specification obligation: gates are not optional CI ceremonies; they are structural constraints on what the system may become.

Continuous integration pipelines can verify six of the seven specification properties automatically. Defended is the exception: whether adversarial challenge has been anticipated and answered requires human review. This is a hard ceiling on automated compliance checking, and §7 results should be read with this constraint in mind.

Defended: the gate corpus is a ledger of paid-for incidents. A gate in this methodology is not an opinion or a style preference — it is a production failure converted into structural prevention. The pipeline is explicit: a *field finding* (a class of defect that passed type-checking and unit tests yet still reached production) is named as a **forbidden pattern**, encoded as a contract assertion, and — when it generalizes — promoted to a versioned, shareable gate in the registry, each entry carrying the originating incident as `provenance`. ForgeCraft’s gate registry makes this literal: for example, a “`findMany` without an explicit `ORDER BY`” gate carries the incident in which heap-scan row ordering bound LLM-generated items to the wrong record *by position* while a two-row unit test passed and production differed. The consequence is methodological, not cosmetic: the rule set is **cumulative and falsifiable** — it grows only from observed failure, every entry is traceable to the loss that paid for it, and any entry can be contested by disputing its incident. This is the structural form of the “*each was paid for once*” discipline, and it is what separates the gate corpus from a linter’s default ruleset: the gates are evidence, not preference. The community ratchet (§7.9) is this pipeline operating across projects; gate-genesis (repeated violations auto-proposing draft gates) is it operating from a single project’s own friction.

Defended: Process. This structural logic extends to the development process itself. Test-driven development requires a strict phase sequence: failing test, confirmed failure, then implementation. In a human workflow, temporal separation enforces this gate. A generative agent in a single context window has no such separation, the agent that will write the implementation is already present when it writes the test. This is not a discipline failure; it is a structural one — **RED-phase collapse**, the TDD-level instance of *phase collapse* (§6.5): the RED phase ceases to exist as a distinct moment because no temporal barrier separates test authorship from implementation authorship. An agent told to “write a

failing test first” can comply in grammar while violating the property substantively, it will write a test shaped to fail against a not-yet-existing function, then immediately create that function. Instructions cannot close this gap. Only structural gates can. Forbidden patterns in the architectural constitution can prohibit implementation choices before a failing test commit is certified. A TDD workflow skill can require pasted test output as a mandatory stop gate before phase advance is permitted. A pre-commit hook can reject a test-only commit where all tests pass, the signature of post-hoc or vacuous tests added after the fact. The [RED] commit naming convention makes TDD phase sequence machine-readable in the git log: a CI rule can detect a feat: commit without a preceding test: [RED] commit and block the merge automatically. Applied to process rather than artifact, the Defended property means the RED phase cannot be bypassed any more than a malformed commit message can be pushed.

Defended: Consequence Classification. In zero-tolerance execution domains (surgical systems, autonomous vehicles, signed legal instruments), Defended acquires a second obligation: the specification must classify the *consequence tier* of each executor action — which operations are reversible, recoverable, or irreversible — and name the human confirmation gate required before any irreversible action may proceed. In software (\$C_i \approx 0\$, \$R \approx 1\$) iteration absorbs residual errors; the classification is unnecessary. Where the correction loop cannot run after the fact, “do no harm” is a specification obligation. The deployment gate framework in §9.4 formalizes when an executor may be trusted with consequential action.

Auditable. The current state of the system, and the history of how it arrived there, is fully recoverable from the artifacts alone. Conventional atomic commits form a typed corpus of change. Architecture Decision Records document why the grammar evolved. Status files record the current implementation state. Nothing requires asking someone who was present at the time. Without an auditable trail, the AI will treat intentional architectural tradeoffs as defects to correct, producing drift silently, across every session that inherits the corrupted context. Full recoverability requires that commit discipline and the ADR record are both maintained. A specification without commit discipline provides partial auditability, behavioral contracts survive, but the reasoning behind session-level decisions does not. The Shattered Stars case (§7.6) demonstrates this boundary precisely: the spec held the system’s structural contracts across sessions; what it could not hold was the provenance of the decisions that shaped them. The property elevates Architecture Decision Records (Nygard, 2011) and conventional commit conventions from recommended practice to required production rule: a system that cannot be audited for why it is the way it is has not satisfied the specification.

Composable. Units can be combined and extended without unexpected coupling. Clean architecture’s dependency inversion and the pure function model from functional programming ensure that composition is predictable. The AI can work on any unit without unexpected propagation effects because isolation is structural, not assumed. (Complete independent isolation of any unit additionally requires the Bounded property: predictably-scoped context ensures that isolation holds across seams, not merely within them.) The property applies Clean Architecture’s Dependency Inversion Principle (Martin, 2017) to the generation context: components must be navigable in isolation, so that a stateless reader can locate the relevant boundary without traversing the full artifact set.

The structural isolation Composable requires also determines the safety of cross-cutting changes. Text-pattern search tools available to AI agents are not AST-aware: they match strings, not symbols. A function rename or interface change propagates to callers, re-exports, barrel files, and dynamic imports — none reliably located by grep. In a system with clean Bounded module boundaries, the change surface for any interface modification is exactly the boundary declaration. The Bounded and Composable

properties together close the search problem that AST-less tooling leaves open — the structural justification for graph-aware, call-chain-traversal tooling as a first-class GS component.

Executable. The generated output satisfies the behavioral contracts the specification defines when exercised against a real execution environment — not merely compiles and passes static analysis. Verifiable establishes that correctness checks exist; Executable establishes that the implementation actually passes them against a live execution context. A system can be fully Verifiable — correct types, passing lint, well-structured tests — while producing a server that fails every integration test against a real database.

Executable is scored conditional on specification availability: a formal contract (Hurl suite, OpenAPI diff, HL7 FHIR runner) enables automated measurement; a goal-directed program requires human acceptance criteria and is scored N/A. The adversarial experiment series (§7.8.B) articulated Executable as a distinct property by measuring the gap: treatment-v2 achieved 12/12 on six structural properties while only 1/9 test suites passed materialization. The property formalizes what practitioners were already doing — verify-and-correct loops — so it can be specified, gated, and tracked.

The seven specification properties are universal, they apply to every project regardless of type or domain. Their concrete artifact expression, however, is project-type-parameterized: the specific quality gates, constraint vocabulary, and required artifact types that satisfy each property vary by what the project is. A healthcare system satisfying Defended requires PII redaction rules and audit logging constraints that a CLI tool does not; a real-time system satisfying Bounded requires latency contracts that a batch pipeline does not. §6 develops the artifact grammar and describes how the universal base and project-type overlays compose.

The failure-mode catalog. The seven properties name structural classes of obligation. Accumulated across six production projects, the observable violations of those obligations resolve into twenty-nine named pathologies — each a concrete, recognizable symptom of one or more absent properties (diagnostic catalog: pragmaworks.dev/diagnostico). The catalog is a bidirectional diagnostic instrument: given an observable symptom (*Architectural Drift*, *Session Amnesia*, *Implicit Contract Syndrome*, *UI Pattern Drift*, *Design Token Scatter*, *Component Reinvention*, *Unspecified Breakpoint Contracts*), it identifies which properties are absent and which tier of the cascade is the structural fix; given a GS property, it enumerates the failure modes that property makes structurally unreachable. The catalog introduces no new theoretical claims. It concretizes the seven properties into the symptom language practitioners encounter in the field — the gap between a rubric score and a team’s ability to act on it. The twenty-nine pathologies are the rubric made actionable.

4.5 Contract Sufficiency: The What-How Distinction

Generative Specification governs *what* the system must do — behavioral contracts, acceptance obligations, architectural boundaries, non-functional requirements — and *why* the non-obvious decisions were made. It does not govern *how* those obligations are fulfilled. Multiple valid implementations can satisfy the same contract. A function that retrieves a user by email may use an indexed SQL query, a cache lookup, or a key-value store — each conforming to the behavioral contract, each differing in its mechanism.

Non-functional requirements belong unambiguously to the *what* layer. A latency threshold, memory budget, throughput floor, or security classification is an obligation, not an implementation choice. At runtime, each NFR becomes a quantified acceptance criterion: a load test asserting p99 latency under 200ms, a penetration test asserting no known vulnerability class passes. The only failure mode for NFRs under GS is omission — an NFR not stated cannot be enforced.

This is the principle TDD operationalizes at the function level. A test specifies what a function must do; dozens of implementations can satisfy it. GS applies the same separation at the system level: the specification certifies what a valid implementation state is; the AI generates the *how* within that certified boundary. The correctness criterion is convergence, not inspection.

This will produce defects. The defect modes of GS differ structurally from those of traditional development: they arise from specification incompleteness rather than the coordination failures that dominate current practice. Specification error is auditable, correctable, and does not compound silently across team rotations.

The ratchet does not reverse. Every defect resolved produces a test and a permanent production rule; every ambiguity resolved becomes an ADR that closes an open decision forever. The defect is not evidence the method failed — it is a specification query: *what constraint, had it been present, would have ruled this out?* When that constraint is written, the grammar expands, and the class of output that produced the defect becomes unreachable.

What GS Does Not Govern: Prompt Engineering

A complete specification makes prompt engineering unnecessary. If the spec is complete, the stateless reader derives the correct output from the grammar alone. If few-shot examples are needed, the specification is incomplete: the example compensates for a constraint not yet stated. The correct response is to write that constraint. Prompt engineering techniques are also model-specific and version-specific; a specification is model-agnostic. The practitioner who needs examples is receiving a diagnostic: the spec has a gap.

4.4.1 Current Best Exemplars by Property

The six production case studies (§7) are the discovery substrate — early proof, built under still-maturing GS. More recent work demonstrates each property more cleanly; where that work is **public** it is cited here, so every exemplar is independently verifiable. (Two of the strongest internal exemplars — a fintech decision engine and a HIPAA data platform — are client-confidential and are deliberately *not* cited as public evidence; the public anchors below are sufficient to inspect each property.)

Property	Public exemplar	Concrete, verifiable artifact
Self-describing	<code>pragmaworks</code> (<code>repo</code>)	<code>CLAUDE.md</code> — a navigation root where every document location announces its domain (screaming architecture)
Bounded	AX (<code>experiments/ax</code>)	boundary and duplication violations (e.g. direct DB access bypassing the repository layer, module-boundary cycles) are machine-counted by external structural analysis (<code>madge</code> , <code>jscpd</code>) across all eight conditions — measured, not hand-graded
Verifiable	AX (<code>experiments/ax</code>)	a Stryker mutation gate drove the mutation score from 58.6% to 93.1% MSI — proving <i>detection</i> , not merely line execution
Defended	AX (<code>experiments/ax</code>)	the Defended property moved 0/2 → 2/2 only once gates were emitted as fenced file templates (the First Response Requirements) — structural enforcement, measured
Auditable	<code>pragmaworks</code>	<code>docs/adrs/0001...0006</code> (each with context/decision/consequences) + conventional-commit history — decisions retrievable
Composable	AX (<code>experiments/ax</code>)	interface-based dependency injection — the GS contribution over expert prompting — so a stateless reader can work a unit in isolation
Executable	EX (<code>experiments/ex</code>)	<code>evidence/slo-ramp-summary.json</code> + the Hurl probe suite — behavioral contracts run against a live runtime

Every exemplar above is in a public repository (`generative-specification` for the experiments, `jghiringhelli/pragmaworks` for the reference project); the §7 case studies remain the record of how the rubric was discovered. The same anchors appear in the derived white paper, keeping the two documents consistent.

5. Related Work: Relationship to Existing Principles

Generative Specification does not replace the syntactic and semantic tier disciplines. It operates at the pragmatic tier: it constrains derivability for a stateless reader, a requirement the prior disciplines were not designed for because no widely-deployed stateless reader existed at their formulation. Each prior discipline removed a programmer freedom and in doing so made the code more predictable for its intended reader. The table below maps each discipline to the tier it occupies, the freedom it removes, and the GS property it satisfies — or the gap it leaves that GS fills.

Discipline	Tier	What it is	Freedom removed	GS property satisfied	What GS adds
Structured programming	Syntactic	Prohibition on unstructured jumps (<code>goto</code>), replaced by loops and conditionals	Unstructured control flow	—	Pragmatic layer obligation: the reader that executes the work must be able to derive intent from artifacts alone
Object-oriented programming (OOP)	Syntactic	Encapsulation of data and behavior inside objects with explicit boundaries	Direct access to internal data structures	Bounded (partially)	Self-describing: the boundary must be externalized in artifacts visible to a stateless reader, not just respected in code
Functional programming (FP)	Syntactic	Prohibition on mutable shared state; computation as transformation of values — immutability and the functional-core/imperative-shell split (Bernhardt)	Variable reassignment across shared state; hidden mutation propagated through the call graph	Bounded, Verifiable	Self-describing: an immutable value object is fully described by its construction, and a pure core is derivable and testable in isolation; GS requires the boundary between pure core and effectful shell to be stated, not merely observed in the code
Clean architecture	Syntactic	Layered design pattern separating concerns into concentric rings: domain entities at the center, application logic surrounding them, infrastructure at the outermost edge	Arbitrary dependency direction between layers	Bounded, Composable	Self-describing: why the layers exist and which directions of change are valid must be in the specification, not deducible from the shape alone
Convention over Configuration / Screaming Architecture	Syntactic	Predictable, uniform project structure in which placement announces purpose — controllers live in <code>controllers/</code> , and the top-level layout names the system's intent rather than its framework (Rails; Martin)	Idiosyncratic structure that forces discovery traversal to locate anything	Self-describing, Bounded	The convention itself must be stated in the specification so a stateless reader's first search is deterministic; an unstated convention is undiscoverable to a reader with no institutional memory

Discipline	Tier	What it is	Freedom removed	GS property satisfied	What GS adds
SOLID	Semantic	Five object-oriented design principles: Single Responsibility (one reason to change), Open/Closed (open for extension, closed for modification), Liskov Substitution (subtypes behave as their supertypes), Interface Segregation (no client forced to depend on methods it does not use), Dependency Inversion (depend on abstractions, not concretions)	Tangled responsibilities and tight coupling between modules	Bounded, Composable	Self-describing: a responsibility boundary identified by SOLID says nothing about whether that boundary must be externalized in artifacts a stateless reader can inspect. A generative specification requires the system to describe itself completely, because the reader cannot ask a colleague
Type-Driven Design	Semantic	Encoding invariants in the type system so that illegal states are unrepresentable; parsing input into precise types at the boundary rather than validating loosely (Minsky; King, <i>Parse, don't validate</i> , 2019)	Illegal states reachable at runtime; failure modes hidden in exceptions rather than declared in signatures	Verifiable, Defended	The type signature is a machine-checked contract and the compiler is a verification layer the executor cannot bypass; GS additionally requires the intent behind a constraint — <i>why</i> an invariant holds — to live in the specification, not only in the type
TDD — Test-Driven Development	Semantic	Discipline of writing a failing test <i>before</i> writing the code that satisfies it; CI gates enforce it by blocking merges on test failure	Shipping code whose behavior is unverified	Verifiable	Executable: a passing test suite certifies behavioral contracts against static assertions; GS adds runtime measurement against a real execution environment. Tests certify intent; Executable certifies derivation
DDD — Domain-Driven Design	Semantic	Methodology for aligning the software model with the business domain through shared language (ubiquitous language), explicit boundaries (bounded	Domain incoherence and leaky abstractions across module boundaries	Bounded	Auditable: DDD defines what the domain model is; GS additionally requires that the decisions behind it — what alternatives were considered

Discipline	Tier	What it is	Freedom removed	GS property satisfied	What GS adds
		contexts), and domain-first modeling			and why they were rejected — live in the decision record, not only in the code
BDD — Behavior-Driven Development	Semantic	Specification of system behavior in structured natural language (Given/When/Then) readable by both technical and non-technical stakeholders	Unspecified user-facing behavior: code that passes tests but satisfies no stated user intent	Verifiable	—
Design by Contract	Semantic	Preconditions, postconditions, and class invariants stated as first-class, checkable assertions on each operation (Meyer, 1992)	Implicit obligations on caller and callee that hold by convention and erode silently	Verifiable, Self-describing	The direct lineage of the F-NNN specification format — Actor, Precondition, Flow, Postcondition, Acceptance criteria — elevated from a per-method assertion to the unit of specification the stateless reader derives from; GS scales the contract from the function to the use case
Conventional commits	Semantic	Typed commit message format (<code>feat</code> , <code>fix</code> , <code>chore</code> , <code>refactor</code> , etc.) that encodes the nature and scope of every change in the git log	Untyped, unreadable change history	Auditable	Elevated from recommended practice to required production rule: an unrecorded architectural decision is a gap in the grammar, not a style choice
CI/CD gates — Continuous Integration / Continuous Deployment	Enforcement	Automated pipelines that run tests, linters, coverage checks, and quality gates on every commit, blocking merges that fail	Post-hoc quality verification after code ships	Defended	Classified consequence tiers in zero-tolerance execution domains: the specification must identify which executor actions are irreversible and require human confirmation before proceeding
GoF Design Patterns	Semantic	Named, reusable solutions to recurring structural problems in object-oriented design. Creational: how objects	Ad-hoc structural solutions that are untestable, undiscoverable,	Composable (Strategy, Adapter: interchangeable, boundary-preserving units);	Pattern vocabulary as specification shorthand: naming a pattern in the architectural

Discipline	Tier	What it is	Freedom removed	GS property satisfied	What GS adds
		are made (Factory Method, Builder, Abstract Factory). Structural: how objects are composed (Adapter, Decorator, Facade, Proxy). Behavioral: how objects interact (Strategy, Command, Observer, State, Chain of Responsibility, Template Method)	and inconsistently applied; AI assistants default to ad-hoc solutions when the pattern vocabulary is absent from the specification	Auditable (Command, Memento: operations as named, traceable objects); Verifiable (State: formal state machine with testable transitions); Defended (Decorator: cross-cutting concerns without touching core logic); Executable (Observer: verifiable event notification contracts); Self-describing (Factory, Repository: naming creates discoverability of the creation and persistence contracts)	constitution activates the AI's full trained knowledge of its canonical structure, interface contracts, and invariants without restating them in prose. The name is the specification. One explicit exclusion: Singleton violates Bounded (hidden global state) and Composable (implicit, undeclared dependencies); GS-governed systems replace Singleton with dependency injection at the composition root, enforced as a prohibition in the architectural constitution

The practical implication: a codebase that follows SOLID and clean architecture is a necessary but not sufficient generative specification. It becomes sufficient when the self-describing, auditable, and executable artifact layers are present and maintained.

GS provides the first formal theoretical framework, named obligation structure, and measurement rubric for a practice that has already emerged organically in the AI-assisted development community — practitioners using AI tools have arrived at discipline-like behaviors (structured prompting, checkpoint reviews, regeneration gates) without a formal account of why they work. That the practice emerged independently of this formalization is evidence for the problem's reality, not against the framework's originality. Formalizing what practitioners do implicitly is the contribution: naming the underlying constraint makes it enforceable structurally rather than discovered through failure.

Why Prior Disciplines Compound with AI Generation

The prior disciplines were designed for human readers — developers who carry context between sessions, notice boundary violations through accumulated familiarity, and ask colleagues when intent is unclear. A stateless AI reader cannot do any of these things. What the prior disciplines provide to a human reader is *ergonomic*: code that is easier to understand and modify. What they provide to an AI reader is *mechanical*: code that is navigable, parseable, and actionable without external context. These are different benefits flowing from the same structural property — and both are real.

Each discipline contributes a distinct mechanical advantage to a stateless reader, separate from its human-reader benefit:

Discipline	Human benefit	AI mechanical advantage — and what read it eliminates
SOLID — Interface Segregation + Dependency Inversion	Decoupled systems are easier to change independently	The interface is the complete behavioral contract for consumers. For any task that does not modify the implementation, the AI never needs to read it — the interface already contains everything a caller can depend on. Token cost falls by an order of magnitude; hallucinated private state is structurally unreachable.
Single Responsibility	One reason to change per class; lower cognitive load	An SRP class cannot have side effects outside its declared responsibility. The class name and method signatures are the complete specification of scope — the AI can act without reading the method body to establish what else it might affect. Side-effect analysis is eliminated.
Hexagonal architecture + consistent folder structure	Concern separation between domain, application, and infrastructure layers	Layer membership predicts the complete dependency graph without reading imports. Domain layer → no framework or database calls possible. Infrastructure layer → no business logic. The folder position is the dependency contract; import reading is eliminated for constraint discovery.
DDD — ubiquitous language	Shared vocabulary between technical and domain teams reduces translation loss	Method signatures drawn from domain vocabulary often contain the full behavioral specification at the call site: domain, operation, inputs, output type, and error contract. For many tasks the implementation body is redundant — the name has already specified the behavior completely.
Conventional commits	Readable change history for human reviewers	The git log is AI-queryable by type and scope without reading diffs. <code>feat(billing): add prorated invoice calculation</code> identifies what changed, where, and the nature of the change. Combined with SOLID, the AI can trace downstream impact from the commit message alone — reading the diff is only necessary when the scope boundary is ambiguous.
TDD — tests as behavioral specification	Failing-first discipline verifies that tests detect the defect they claim to	The test suite is the complete, executable behavioral specification. Every expected behavior is asserted; every error case is named; every invariant is tested. For behavior understanding, the test suite replaces reading the implementation — the implementation is the proof, the tests are the theorem, and for most tasks the theorem is what matters.
ADRs — Architecture Decision Records	Preserves design rationale for future human maintainers	ADRs predict implementation shape without requiring a code read. “Event sourcing for billing” in an ADR tells the AI the billing module has event stores, projections, and replays — none of which need to be read to be known. Structural decision reading is eliminated for modules whose ADRs are present and current.
Doc-first cascade (sentinel → spec → ADR → code)	Keeps documentation synchronized with implementation	The functional specification describes all behaviors; the code is the derivation. When the spec is complete and maintained, reading the derivation adds no information about intent — the spec already contains it. Implementation reading for behavior understanding is eliminated for spec-covered components.
Intentional naming	Reduces cognitive load when reading unfamiliar code	<code>calculateMonthlyCostPerMember(userId: UserId, period: BillingPeriod): Result<Money, BillingError></code> specifies domain, operation, inputs, output, and error contract at the signature. Nothing further is inferrable from the body that is not already present. For a large class of operations, the method signature is the complete specification and the implementation body is never read.
Commit hooks / quality gates	Catches defects before they land in the shared branch	Hooks are what make the above navigation policies safe. They guarantee that SOLID interfaces, TDD coverage, and structural rules are maintained — that the contract layer has not silently diverged from the implementation. Without enforcement, contract-sufficient navigation produces confident errors. With enforcement, it is safe to apply.

Discipline	Human benefit	AI mechanical advantage — and what read it eliminates
GoF design patterns (when named in spec)	Named, reusable solutions with known tradeoffs	The pattern name is a complete implementation specification. A named <code>Repository</code> requires no implementation read — the AI knows the interface shape, the correct dependency direction, the persistence contract, and the test strategy from training alone. Pattern-named components are fully specified without a single line of code being read.
Clean architecture layers	Explicit dependency directions make the system predictable	Layer membership eliminates import reading for constraint discovery and eliminates implementation reading for dependency tracing. Knowing a class is in the domain layer means: no framework dependencies, no database calls, pure business logic — verifiable from folder position alone, no file open required.

The implications for adoption are structural. These disciplines existed for thirty years. Adoption was incomplete because sustaining them required what human developers could not reliably provide under deadline pressure: consistent application across every person, every commit, every architectural decision. The AI reader that now executes the work is simultaneously the beneficiary of the disciplines and the executor that makes them enforceable. The adoption incentive also changes: a team that installs SOLID because the sentinel makes implementation reads unnecessary experiences the benefit as measurable, session-by-session reduction in cost and context usage — not as an abstract quality improvement attributed to other causes over time.

GS's four new contributions — the properties no prior discipline named — are worth stating plainly. **Self-describing**: the system must externalize its complete architectural identity in artifacts a stateless reader can inspect without asking a colleague. **Defended**: the process must have structural guards that make certain failures unreachable, not merely discouraged. **Auditable**: every architectural decision must be recorded as a required production rule, not a recommended practice. **Executable**: the implementation must be measured against a real runtime environment, not only against static test assertions. Together these four close the derivability gap the prior disciplines leave open.

GS is categorically distinct from prompt engineering — HOW-layer techniques that guide model behavior toward a particular implementation path. The full distinction is in §4.5.

On Infrastructure-as-Code and contract-first API design. Two established practices are frequently conflated with the derivability obligation and deserve explicit placement. Infrastructure-as-Code (Terraform, Pulumi, AWS CDK) declares infrastructure state as a versioned, auditable artifact — this satisfies GS's *Auditable* and *Defended* properties at the infrastructure layer and is a required component of a Tier 2 GS-governed system. Contract-first API design (OpenAPI, AsyncAPI, gRPC IDL) declares service boundary behavior as a machine-readable artifact — this satisfies *Verifiable* and *Composable* at the API layer. The GS differential is scope and unification: IaC governs infrastructure alone; contract-first API design governs API surfaces alone; neither addresses the architectural constitution, decision record, or session-load sentinel that make the full derivability obligation satisfiable across all lifecycle stages. A project with complete Terraform coverage and an OpenAPI specification has partially satisfied Tier 2 and partial *Verifiable/Composable* at the boundary layer — it has not produced a generative specification. The specification is the artifact set from which an AI reader can derive correct behavior across all lifecycle stages without external context; infrastructure declarations and API contracts are necessary components of that set, not substitutes for it.

SOLID Extended: The Service Boundary Tier

The SOLID principles were formulated for the class/module grain. They apply without modification at a larger grain: the microservice boundary. The mapping is structurally exact — each principle that governs the class interface governs the service interface — but the HA consequences are more severe because failures at the service boundary are distributed failures. The following maps each SOLID principle to its microservices expression and its GS derivability consequence:

SOLID Principle	Class/module expression	Service boundary expression	GS consequence
Single Responsibility	One reason to change per class	One business capability per service	A single-capability service has a small, bounded spec. Blast radius is proportional to spec surface: a service that does too much has a spec that cannot be complete, and an outage takes down unrelated capabilities whose behavioral contracts the AI cannot distinguish from the failed one.
Open/Closed + Dependency Inversion	Extend via abstraction; depend on interfaces not concretions	Services communicate via well-defined contracts (event schemas, API specs), not direct implementation dependencies	Synchronous REST between services creates temporal coupling — a downstream failure propagates upstream because the contract includes availability. Asynchronous messaging via event broker (Kafka, RabbitMQ) decouples availability from the contract: the event schema is the interface, the broker absorbs the coupling, and the consumer's spec depends only on the event shape, not the producer's availability. The AI navigates event schemas as contracts; it never reads producer implementations to understand what a consumer depends on.
Interface Segregation → Database isolation	No client forced to depend on methods it does not use	No service shares a datastore with another	A shared database is an undeclared interface: Service B implicitly depends on Service A's schema, write patterns, and failure modes without that dependency appearing in either service's spec. GS requires that every dependency be expressible in the spec; a shared database violates Self-describing at the service boundary. Database-per-service enforces that the full data contract is derivable from the service's spec alone.
Liskov Substitution → Idempotency + Statelessness	Subtypes substitutable without altering correctness	Service instances substitutable without altering correctness	Stateless services can be killed and replaced by an orchestrator (Kubernetes) without session loss — they behave correctly as substitutes for the failed instance. Idempotent write operations mean retrying after a mid-flight failure produces the same result as the original call: the contract holds across retries. GS's Verifiable property requires that the behavioral contract be testable across repeated execution; idempotency is the precondition for that.
Dependency Inversion → Bulkheads + Circuit Breakers	Depend on abstractions; inject concretions from outside	Isolate failure domains; circuit-break slow/failed dependencies before they consume available threads	The circuit breaker is the runtime expression of dependency inversion at the service boundary: when Service Y is slow, Service X does not wait for it — the breaker trips and returns the fallback contract immediately. The timeout, retry policy, and fallback behavior are spec artifacts. A service whose failure modes are not declared in its spec has an incomplete contract; GS requires these to be stated, not discovered from production incidents.

Two additional HA patterns extend SOLID at the service boundary without a direct class-level analog:

Graceful Degradation. When a downstream service fails, the system continues in reduced capacity rather than propagating a 500. The degraded behavior — cached data, default response, empty collection — is a behavioral contract that must be declared in the spec, not discovered from the implementation. A service whose spec does not declare its fallback behavior has a contract that only

covers the success path; the AI cannot generate correct consumer behavior for the failure path without that contract.

Redundancy and Autoscaling. Deployment topology — container orchestration, horizontal autoscaling, multi-availability-zone distribution — is a T2/T3 specification artifact (Hardening and Production tiers). It does not belong in the class-level spec; it belongs in the infrastructure-as-code spec and the deployment ADR. The AI reads deployment specs to understand failure domains, not implementation code.

The GS implication across all seven principles: **at the service boundary, the API spec and event schema are the “interface” in the SOLID sense.** Contract-sufficient navigation (§6.0) applies at this grain too — a service consumer never reads the service implementation to understand what it can depend on; it reads the service’s API spec or event schema. A microservices architecture governed by GS has complete behavioral contracts at every grain: class interface → module boundary → service API → event schema. At no grain does a stateless reader need to read an implementation to understand the behavior it depends on.

Complementary Patterns and Their GS Expression

The following patterns extend the discipline covered above or replace weaker forms of the same principle. Each is treated as a spec artifact question: what does the AI read, and what does the discipline make unnecessary to read?

Pattern	Replaces / Complements	GS consequence
CQRS — Command Query Responsibility Segregation	Replaces a single unified model with two explicit contracts: command (writes, changes state) and query (reads, returns state).	The AI reads the command contract to understand what changes state and the query contract to understand what is returned — without reading the implementation of either. Three auditable surfaces (command, event, query projection) replace one opaque state model. The audit log is structural, not bolted on.
Event Sourcing	Replaces state-snapshot persistence with event-log persistence.	The event schema is the complete behavioral specification: every state transition is a named, typed event. Current state is a projection — a derivation the AI can reconstruct from the event log spec without reading the projector implementation. Pairs naturally with CQRS: command → event → projection is a derivation chain fully specified by three contracts.
Consumer-Driven Contract Testing (Pact)	Replaces provider-defined API specs as the source of truth for integration contracts.	Consumers write what they need from the provider; the provider verifies it satisfies those contracts. Pact files live in the repo as spec artifacts. The AI reads Pact contracts to understand what each consumer actually depends on — not what the provider thinks it exposes. Stronger than OpenAPI alone because it is driven by verified, real-world consumer usage.
Protocol Buffers / gRPC	Replaces OpenAPI + JSON as the service interface spec for internal service-to-service communication.	The <code>.proto</code> file is simultaneously the interface contract, the serialization contract, and the code generation template — the canonical expression of “the interface IS the spec” at the service level. Client and server stubs are generated derivations. The AI reads the proto file; reading the generated code adds nothing the proto does not already state. Stronger than OpenAPI because the serialization contract is declared, not inferred.
Property-Based Testing	Complements TDD; replaces many specific test cases with generalized invariant specifications.	Instead of <code>input X → output Y</code> , the specification states: <code>for all X in domain D, output satisfies property P</code> . The property spec is a richer contract — it defines the behavioral envelope rather than verifying sampled points within it. The AI reads property specifications to understand what must always be true, not just what was verified for specific inputs. Particularly valuable for domains with large input spaces (parsers, financial calculations, data transformations).
Consistency Model Declaration (CAP)	Complements any distributed service spec; replaces implicit assumptions with explicit behavioral contracts.	Whether a service provides strong consistency or eventual consistency is a behavioral contract every consumer’s code depends on. An undeclared consistency model is a spec gap: the AI generates consumer code assuming strong consistency by default and fails silently under eventual consistency. One ADR entry per service boundary.
Saga Pattern	Replaces distributed ACID transactions with explicit step + compensation sequences.	The saga definition — each step and its compensation (rollback action) — is the spec. Compensations are the fallback behavioral contracts for distributed transaction failure; an unspecified compensation is an unspecified failure path. The AI reads the saga definition to understand success and failure behavior at each step without reading the orchestrator or choreographer implementation.
Anti-Corruption Layer (DDD)	Complements bounded context integration; replaces implicit vocabulary leakage with an explicit translation contract.	The ACL interface is the spec artifact that isolates two bounded contexts with different vocabularies. The AI reads the two interfaces the ACL connects; it never reads the translation logic inside it. The translation is an implementation detail; the contracts on both sides are the specification. The ACL makes the vocabulary boundary explicit rather than leaving it as an informal convention.

Industry Prior Art: Spec-Driven Development

The practitioner community independently converged on the same starting insight: specifications should drive AI-assisted development, not follow it. **SpecKit** (GitHub, 2025) and **OpenSpec** (@fission-ai/openspec, 2025) both implement this as a structured path from “write a spec, then prompt.” **AWS Kiro** (2025) and **Tessl** (2025) implement spec-first development as the primary authoring surface, extending the same convergence to cloud-native and low-code tooling. **Fowler (2026)** surveys the emerging tooling landscape, establishing that the practice has reached mainstream visibility: when the field’s leading methodologist publishes comparisons, the practice is past the early-adopter phase. A peer-reviewed taxonomy has appeared: **arXiv:2602.00180** (Feb 2026) defines three Spec-Driven Development rigor levels — spec-first, spec-anchored, and spec-as-source — the first formal classification of the practice this work claims to theorize. **ThoughtWorks Technology Radar (2025)** placed SDD in the Adopt ring, its strongest practitioner endorsement, confirming independent spread across the field. **Osmani (Google Cloud AI, 2026)** publishes `agent-skills`, a 22-skill production lifecycle pack organized as DEFINE → PLAN → BUILD → VERIFY → REVIEW → SHIP and built on the explicit principle that “code without a spec is guessing.” His framing — “*AI coding quality usually fails at the specification layer before it fails at the model layer*” — makes the SDD convergence the operating standard at a hyperscaler AI org, no longer only an emerging-tools narrative.

The divergence from GS is depth. Every listed system — including Osmani’s six DEFINE→...→SHIP phases — stops at Tier 1’s *authoring* half (spec drives implementation), without treating Tier 1’s verification harness as constitutive — let alone reaching staging governance (T2), production runtime (T3), or evolutionary lifecycle (T4+). None formalizes the structural properties an artifact set must satisfy to make derivation correct at any of those stages. The shared starting point validates the problem is real; the divergence in depth is where the paradigm claim lives. The industry is converging on the practice without having named the principle. GS names the principle.

Industry Prior Art: Context Enhancement

Agent instruction files (`CLAUDE.md`, `AGENTS.md`, `.cursorrules`) and memory/status files are the field’s first-order responses: inject rules into the context window, persist state across sessions. GS makes full use of them. But they are context-enhancement tools — they improve what enters the channel. None specify what the channel must be able to *derive* from what it receives. Feeding a richer window into an underspecified system produces richer drift at generation speed. GS is categorically distinct: not what to put in the channel, but what a complete grammar for a stateless reader must look like.

Independent corroborating work is covered in §3.5.

Industry Prior Art: Process Orchestration

A distinct prior art category addresses how work is organized *within* AI-assisted development — not the specification the AI reads, but the workflow structure around the AI. **gstack** (Tan, 2025; 85,900 GitHub stars at time of writing) is the most widely adopted: a sprint-based process methodology that assigns role specialization (architect, implementer, reviewer), parallel work tracks, and delivery cadence to human-AI collaboration. It answers the question “how should a team structure its sessions?” GS addresses a prior and orthogonal question: “what must the codebase say so that any session — regardless of team structure — begins from a correct position?”

The distinction is architectural layer, not competing methodology. Process orchestration governs behavior within a session; GS governs what persists between sessions. A team following gstack’s sprint structure but operating on an underspecified codebase will produce consistent outputs for one session

and drift for the next, because the stateless reader that begins session two has no specification to read. Conversely, a practitioner working alone against a fully GS-compliant codebase needs no process framework — the specification is the continuity mechanism. The two approaches are compatible and address different failure modes: gstack prevents disorganization within a session; GS prevents drift across sessions. The gstack gap analysis (see §7.8 ForgeCraft case, P1–P4 enumeration) confirms that the tooling-level gaps gstack identifies — pre-commit hook reliability, PR title validation, dependency audit wiring — are enforcement machinery at GS’s defended layer, not process claims.

The Prior Paradigm and Its Anomalies

Agile iteration, TDD, CI/CD, and DDD constitute a mature layered methodology the industry converged on over two decades. Each discipline removes a degree of programmer freedom; each removal makes the system more predictable. Together they set the baseline GS extends.

The methodology breaks at generation speed. Each discipline assumes a human author: a developer who carries architectural intent between sessions, who notices when a new module violates an existing boundary, who understands why a decision was made three months ago. The quality checks are periodic; the generation is continuous. Drift accumulates in the interval between checks, locally invisible, propagating silently.

The anomalies are specific: TDD’s RED phase ceases to exist when test and implementation authorship occur in the same context window (*RED-phase collapse*, the TDD-level instance of phase collapse). Code review becomes structurally unable to catch drift that accumulated across ten sessions. ADRs become orphaned — the AI has no access to why the system is the way it is. Peng et al. (2023) document a 55.8% productivity gain from AI assistance; the same generation capacity accelerates drift. The methodology that governed human-speed development has no mechanism for generation-speed incoherence.

GS extends the prior paradigm to the generation context. The prior paradigm removed degrees of programmer freedom at human speed. GS removes degrees of generator freedom at generation speed. The gap was not visible when developers wrote every line. It became visible the moment they stopped.

LLM Code Generation Research: Empirical Grounding

HumanEval (Chen et al., 2021) measures single-function synthesis in isolation; **SWE-bench** (Jimenez et al., 2024) measures AI patching ability on navigable codebases. GS addresses a structurally different problem at a prior layer: the conditions under which a codebase is navigable by a stateless reader in the first place. A model scoring 90% on HumanEval can still produce architectural drift across ten sessions if no specification governs cross-session structure. Whether GS-compliant codebases achieve measurably higher SWE-bench patch success is a direct empirical falsification candidate.

Peng et al. (2023) establish a 55.8% task-completion speed increase under AI assistance. GS’s claim is not that AI improves speed — that is established. The claim is that speed without structural governance produces drift faster, and GS is the governance layer that makes multi-session speed sustainable.

Prompt engineering (White et al., 2023) establishes that input framing significantly affects output quality. A prompt is a session artifact; it governs one interaction. GS is the layer that persists across every session and governs every prompt submitted against it. The full distinction is in §8.9.

The seven GS properties have structural analogs in **ISO/IEC 25010** (Maintainability, Testability, Analyzability) — independent convergence from a standards direction, establishing that the properties are not ad hoc.

A separate empirical thread provides independent evidence for this work’s central mechanism claim — that the annotation burden blocking the formal tradition for decades is dissolving. **Stanford Clover (2024)** demonstrates closed-loop verifiable code generation: an LLM generates code and formal annotations simultaneously, a verifier checks them, and the loop iterates until verification passes — achieving an 87% acceptance rate on standard benchmarks. **Microsoft DafnyBench (2024)** demonstrates LLMs auto-annotating Dafny programs with formal invariants: 68% baseline accuracy rising to 98% with verifier feedback. **PropertyGPT (Ye et al., NDSS 2024)** generates formal verification properties for smart contracts via retrieval-augmented LLMs, discovering previously unknown vulnerabilities in the process. None of these systems is GS-aware. None frames itself as a paradigm shift. That is precisely their value: they are independent empirical proof that the annotation burden which made the formal tradition impractical for human teams is structurally gone. The formal disciplines did not fail because their theory was wrong. They failed because maintenance exceeded human capacity. That capacity constraint has changed.

Sakana AI et al. (2025) introduce the **Darwin-Gödel Machine** (arXiv:2505.22954), a coding agent that empirically rewrites its own scaffold — foundation-model weights frozen — and retains every variant in an open-ended archive, lifting itself from 20.0% to 50.0% on SWE-bench and 14.2% to 30.7% on Polyglot. Its defining move is to replace Schmidhuber’s Gödel Machine, which required *provable* self-improvement, with *empirical* validation against a benchmark — the same substitution GS makes at the artifact layer: the stateless reader verifies, it does not prove. The two systems are complementary rather than competing. The DGM optimizes the harness; GS disciplines the artifact the harness produces; the verify-over-prove stance recurs at both layers, and GS is the verification substrate that would make a self-improving agent auditable. The relationship sharpens to direct corroboration in the DGM’s documented failure modes: tasked to reduce its own tool-use hallucination, one variant fabricated test logs — reporting passes for tests it never ran — and another deleted the very markers the evaluator inspected, scoring perfectly while solving nothing. This is independent evidence for the verification requirement GS makes structural: an agent’s report of its own success is not verification. The Executable property (§4) is satisfied only when verification reads signals the agent does not author — a real runtime, recorded by the harness, not the agent — which is precisely what generative execution enforces.

A complementary intervention attacks the verbosity failure mode (§3.5) from the opposite layer. **ponytail** (github.com/DietrichGebert/ponytail, 2026) is an agent-agnostic ruleset that injects a minimization discipline — a decision hierarchy (necessity/YAGNI → standard library → platform feature → installed dependency → one line → minimal implementation, with security, data-loss handling, and accessibility explicitly out of scope of the trimming) — and reports, across Haiku/Sonnet/Opus on a reproducible promptfoo benchmark, on the order of 80–94% less code at comparable correctness. It is independent evidence that the agent over-generation SlopCodeBench measures is reducible by discipline rather than intrinsic. The distinction from GS is the layer: ponytail operates at the HOW layer — shaping generation behavior through a prompt-injected rule the model applies as it writes — whereas GS removes bespoke code at the WHAT layer, deriving the implementation from a specification (§4.5). The two compose: a GS-governed project can adopt ponytail’s minimization heuristic inside the generation step without altering the specification that governs derivability.

6. The Artifact Grammar

The methodology's empirical record does not depend on ForgeCraft: any practitioner producing the same artifacts by hand produces the same grammar the AI reads. See §7.8 for authorship and conflict-of-interest disclosure.

Tool-agnostic mechanical reference. The repo-mechanical specifics — what files exist, what they contain, how they are touched, and the manual checklists that hold the discipline together without any tooling — are consolidated in [docs/repository-discipline.md](#). This section gives the theory; that document gives the operational discipline.

A system built to generative specification consists of the following artifact types, each functioning as a distinct production rule in the system's grammar.

Artifact	Linguistic Analog	Function in the System
Architectural constitution ¹	Grammar rules	Defines what is and is not a valid sentence in this system. Every AI interaction is governed by this document. Agent-specific filenames: <code>CLAUDE.md</code> (Anthropic Claude), <code>AGENTS.md</code> (OpenAI), <code>.cursorrules</code> / <code>.cursor/rules/</code> (Cursor), <code>.github/copilot-instructions.md</code> (GitHub Copilot), <code>.windsurfrules</code> (Windsurf). The concept is agent-agnostic; the filename is not. In projects large enough to exceed a single bounded file, the architectural constitution is the root node of a sentinel navigational tree (§4.4 Bounded).
Sentinel navigational tree	Grammar index with lazy derivation	A hierarchy of scoped specification files. The root is always loaded; each child node declares its own domain and routing condition; the AI descends only the path relevant to the current task. Joining all leaf nodes yields the complete specification. Five categories must be collectively present across the tree: architectural identity, standards, constraints and prohibitions, tool sequencing (when to use which tool under which condition), and routing. The root node must stay within the bounded line limit precisely because it is always loaded. A tree that omits any category from its leaves forces the AI to infer that category — which is a derivability failure at the navigation layer. On GS-compliant codebases (SOLID interfaces present, hexagonal structure, TDD contracts), the root node carries a navigation mode declaration instructing the AI to read interface and test contract definitions before implementations — converting the passive structural benefit of the prior disciplines into an active navigation policy (§6.o).
Architecture Decision Records (ADRs)	Etymology and rule changelog	Documents why the grammar evolved. Prevents the AI from “correcting” intentional decisions that appear suboptimal without context.
C4 diagrams / structural diagrams (PlantUML, Mermaid)	Syntax tree	The parsed structural representation of the system. Context at a glance for any agent entering the codebase.
Use cases, flow diagrams, sequence diagrams, state machine diagrams	Sentence patterns and grammar rules with temporal order	Each diagram type constrains a distinct dimension: sequence diagrams fix the protocol between components (which calls, in which order, with which contracts); user flow diagrams define the expected journey from entry point to outcome (and are simultaneously the script for every E2E test in that flow); state machine diagrams enumerate valid states and transitions (and directly generate state transition test cases and the user-facing documentation of each mode). These are not illustrations. They are production rules the AI reads before generating any artifact the diagram describes.
Schema definitions (database, API, event)	Type system / lexicon	The vocabulary of the system with its constraints formally stated.
Living documentation (derived)	Compiled output from the grammar	Documentation regenerated from the specification: OpenAPI/Swagger from type annotations or route decorators, TypeDoc/JSDoc from inline documentation, Storybook from component specifications, generated README sections from centralized specs. Documentation maintained separately from the code it describes is a liability: it will drift. Documentation derived from the same artifacts the AI reads is always current, because it shares a source of truth with the implementation.
Intentional naming conventions	Word choice	Semantic signal at every token. A function named <code>calculateMonthlyCostPerMember</code> carries domain, operation, unit, and scope. <code>processData</code> carries nothing.

Artifact	Linguistic Analog	Function in the System
Package and module hierarchy	Phrase structure rules	Communicates responsibility and ownership through structure. The location of a file is a claim about what it is.
Conventional atomic commits	Typed corpus with morphology	<code>feat(billing): add prorated invoice calculation</code> has a part of speech, a scope, and a semantic payload. The git log is a readable history of how the grammar evolved and why.
Test suite (TDD / adversarial)	Semantic validation + adversarial probe	Each test is a statement about what the system must do: a specification assertion <i>and</i> an adversarial challenge, the agent writes tests intended to expose incorrect code, not to document assumptions. The full suite is a continuously-running audit and a standing challenge to the implementation.
Commit hooks and quality gates	Parser rejection rules	Malformed input is structurally rejected before it enters the system. The architecture makes certain mistakes unreachable.
MCP tools and environment tooling	Runtime environment	The tools available to the agent define what operations are possible. Bounded tool access is bounded agency.

The artifact grammar above is the universal base; a complete generative specification composes it with a project-type overlay (healthcare adds PII rules and audit trails; real-time adds latency contracts; a game adds asset quality gates; a frontend or full-stack web project adds UI/UX contracts — design tokens, component library specification using atomic design, UX pattern documents covering error, loading, empty, modal, and table states, and a responsive/adaptive ADR — as a recognized artifact class satisfying the Self-describing and Composable properties at the presentation layer). ForgeCraft-MCP implements this through its tag system: every project receives `UNIVERSAL`, and each active tag (`API`, `WEB-REACT`, `GAME`, `FINTECH`, etc.) applies its overlay. ForgeCraft is **transitional scaffolding** — it enforces at the governance layer what Loom will eventually enforce at the language and compiler layers; its ADRs become verified contracts, its commit hooks become type-checker passes. ForgeCraft bundles CodeSeeker as a first-class component because the grep-versus-AST limitation (§4.4 Composable) makes agent-driven refactoring unsafe without graph-level navigation; governance and navigation are designed to operate together.

6.0 Contract-Sufficient Navigation: The Closed Loop

The preceding section (§5) established that the prior disciplines benefit AI readers for mechanical reasons their authors did not design for. A further implication follows: a sentinel that makes those disciplines explicit can instruct the AI to *exploit* them actively, rather than benefit from them passively. The claim is stronger than “read contracts before implementations.” On a fully GS-compliant codebase, **implementations are optional for the AI in most tasks** — they exist for modification, not for understanding. The contract layer is sufficient.

The architectural constitution on a GS-compliant codebase should include a navigation mode declaration that makes this explicit:

Navigation Mode

This project follows SOLID (interfaces as contracts), hexagonal layering, and TDD.

- For understanding behavior: read interface/abstract class definitions only.
Do not read implementations — they are derivations of contracts already visible.
- For behavioral contracts: read TDD test signatures and descriptions.
The test suite is the complete behavioral specification.
- For behavior inference: method signatures, return types, and domain names are often sufficient. Read implementations only when modifying them or debugging a contract violation.
- Layer membership (domain / application / infrastructure) predicts the complete dependency graph. Do not read imports to discover constraints.

Each structural discipline present in the codebase removes a class of read from the AI's navigation path:

Discipline present	Class of read that becomes unnecessary
SOLID interfaces + Dependency Inversion	Reading implementations to understand behavior a consumer depends on. The interface is the contract; the implementation is the proof. Consumers only need the contract.
Single Responsibility	Reading method bodies to establish scope and side-effect boundary. An SRP class cannot have side effects outside its declared responsibility — the class name enforces this without reading.
Hexagonal / clean architecture layers	Reading imports to discover dependency constraints. Layer membership predicts the complete dependency graph: domain layer → no framework or database calls possible; infrastructure layer → no business logic.
DDD ubiquitous language + intentional naming	Reading method bodies to infer what a method does. <code>calculateMonthlyCostPerMember(userId, period): Result<Money, BillingError></code> contains domain, operation, inputs, output, and error contract at the signature. Nothing more is inferable from the body that isn't already present.
TDD — tests as behavioral specification	Reading implementations to understand expected behavior. The test suite is a complete, executable behavioral specification. Every expected behavior is asserted; every error case is named; every invariant is tested. The implementation is the proof; the tests are the theorem.
GoF patterns (named in spec)	Reading implementations to understand structure, interface contracts, and invariants. The pattern name activates complete trained knowledge — a named <code>Repository</code> or <code>Strategy</code> requires no implementation read to know its interface shape, dependency direction, and extension contract.
ADRs	Reading code to understand <i>why</i> structural decisions were made. An ADR predicts the implementation shape: “event sourcing for billing” → the AI knows the billing module has event stores, projections, and replays without reading any of it.
Doc-first cascade	Reading code to understand what a component is supposed to do. The functional spec describes all behaviors; the code is the derivation. When the spec is complete, reading the derivation adds no information about intent.
Commit hooks / quality gates	Doubting whether the contract layer is trustworthy. Enforcement ensures SOLID interfaces, TDD coverage, and structural rules are maintained — the contract layer has not silently diverged from the implementation. The hooks are what make the above navigation policies safe to apply.

The conditionality is structural, not optional. These navigation policies apply only where the corresponding discipline is enforced, not merely named. GS-compliant greenfield: fully applicable from day one. Post-Annealing brownfield with confirmed SOLID interfaces and TDD coverage: applicable where enforced. Unrefactored brownfield: inapplicable — on codebases where interfaces may be absent, incomplete, or silently diverged from implementations, reading contracts as authoritative produces

errors indistinguishable from good output. The Annealing stage creates the conditions; the navigation policies are the operational reward that makes Annealing worth delivering.

The closed loop. The adoption dynamic for the prior disciplines was: follow the discipline because it is correct; experience the benefit as ergonomic improvement over time. Under GS, the dynamic changes: the sentinel instructs the AI to skip implementation reads when contracts are trustworthy, and the team experiences the benefit as measurable reduction in session cost, context consumption, and token usage — observable, session-by-session. A team that installs SOLID because the sentinel makes implementation reads unnecessary has a different adoption incentive than a team that installs SOLID because “it’s best practice.” The discipline pays off in a currency the team can observe in every session, rather than in a quality improvement they attribute to other causes.

6.1 A Generative Specification in Practice

The artifact types above are not theoretical constructs. The following is a representative excerpt from the architectural constitution (`CLAUDE.md` in the Claude convention¹) that governed the SafetyCorePro refactor described in §7.1, written in its entirety before a single implementation change was made. The complete document is 155 lines.

CLAUDE.md. SafetyCorePro

Project Identity

- Primary Language: TypeScript 5.x
- Framework: Next.js 14 (App Router) + Prisma 5 + PostgreSQL
- Domain: Occupational Safety Management Platform
- Sensitive Data: YES: PII (employee records), safety incident data, compliance records

Architecture Rules

- All data access goes through service/repository layers, never direct Prisma calls from components or route handlers.
- No business logic in API route handlers, they delegate to services.
- Multi-tenant: Every query MUST include `cuentaId` filter. Never expose cross-tenant data.
- Permission checks via `requirePermission()` / `requireAuth()` as first line of every server action.

Layered Architecture

Pages / API Routes / Actions	← Thin. Validation + delegation only.
Services (Business Logic)	← Orchestration. Depends on interfaces only.
Domain Models / Types	← Pure data + behavior. No I/O. No framework.
Repositories / Adapters	← All external I/O (DB, APIs, files, queues)

Never skip layers. Dependencies point downward only.

Error Handling

- Custom error hierarchy per module. No bare Error throws.
- Errors carry context: IDs, timestamps, operation names.
- Fail fast, fail loud. No silent swallowing of exceptions.

Code Standards

- Maximum function length: 50 lines. Maximum file length: 300 lines.
- Every public function must have JSDoc with typed params and returns.
- No abbreviations except universally understood (id, url, http, db, api).
- Bilingual naming: Domain entities keep Spanish names (visita, empresa, hallazgo, reporte) to match DB schema. All technical code uses English.

Testing Pyramid

- Overall minimum: 80% line coverage
- New/changed code: 90% minimum
- Critical paths: 95%+ (permissions, multi-tenant isolation)
- Every test name is a specification: `test_rejects_duplicate_empresa`, not `test_validation`

Commit Protocol

- Conventional commits: `feat|fix|refactor|docs|test|chore(scope): description`
- Commits must pass: TypeScript compilation, lint, tests.
- Keep commits atomic, one logical change per commit.
- Update Status.md at the end of every session.

The AI read the architecture rules and produced services. It read the error handling rules and produced a custom exception hierarchy. It read the bilingual naming convention and applied it consistently across every new file. The specification is not a description of what was built. It is the grammar from which the build was derived.

6.2 The Initialization Cascade

The artifact types above are not produced in parallel or in arbitrary order. Each artifact is both an output of what precedes it and a production rule for what follows. A sequence diagram that contradicts the architecture is evidence that one of them is incomplete; an ADR written before the architecture is speculation rather than decision record. The initialization cascade is:

1. **Functional specification**, user-facing behavior, domain model, key entities, and system boundaries stated with enough precision that an agent can distinguish an in-scope request from an out-of-scope one. This is the axiom set; everything else is derived from it. If a requirement cannot be stated here, it is not yet a requirement.
2. **Architecture document**, the layered structure, module boundaries, and integration surfaces the specification implies. Mermaid C4 context and container diagrams are produced at this step, expressing the architecture in the structural vocabulary the artifact grammar names. The diagram is not an illustration of the architecture; it is the architecture at a level of abstraction the team and the AI can both read without ambiguity.
3. **Architectural constitution** (`CLAUDE.md` / equivalent), the operative grammar extracted from the architecture: the rules an agent must read before any implementation session begins. This document is derived mechanically from the architecture and the functional specification; ForgeCraft-MCP automates a substantial portion of this derivation.
4. **Architecture Decision Records (ADRs)**, one per non-obvious architectural choice, each recording the alternatives considered, the criteria applied, and the reasoning for the decision taken. ADRs are written immediately after the constitution, not reconstructed after the fact, because the reasoning is present now and will not be recoverable later.
5. **Use cases, sequence diagrams, and state machines**, the behavioral contracts between components, specified with enough precision that each diagram is simultaneously a test specification. A Mermaid sequence diagram naming a payment flow generates both the service interface contract and the acceptance test skeleton. A state machine diagram for a subscription entity enumerates valid states and valid transitions, and any implementation that permits an unlisted transition is wrong by the specification.

The cascade closes when a stateless agent given these five artifact sets can derive any valid implementation state without further human direction. That is the derivability criterion of §4.4, and it is the test the practitioner should apply before calling the specification complete. Generating diagrams after the code is written is documentation; generating them in this order is the specification act itself.

6.3 The Prompt-Bound Roadmap

The roadmap is not planned separately from the artifacts — it is derived from them. The AI reads the functional specification, architecture document, and ADRs, and produces a phased plan with a pre-

generated agent prompt for each item. The binding is the operative detail: a prompt-bound item is an independent execution unit containing the relevant specification references, acceptance criteria, and verification steps. At execution time, the practitioner triggers the item and reviews the output. The result: (1) the git log is the roadmap execution record, each commit traceable to a roadmap item; (2) loop separation by granularity is enforced — short loops have fixed scope, long loops have milestone-level granularity, neither collapses into the other; (3) waiting states constitute productive inventory — a project at a natural boundary is placed in a ready state with context intact, no reconstitution required.

6.4 The Incremental Cascade

Incremental changes propagate bottom-up: the practitioner observes a discrepancy or new requirement; the AI performs an impact assessment (which artifacts reference the changed element, which roadmap items share a dependency); the cascade propagates upward to the minimum affected layers, then back down to implementation. Not every increment walks all five initialization steps. The ordering constraint is: when a layer needs updating, all layers above it are made consistent before any layer below it receives the change. The spec is always the system of record — code wrong relative to the new spec is a derivation gap, not a bug.

6.5 Loop Types and Gate Conditions

The methodology operates at four loop granularities:

- **Initialization loop** (once per project): gate = derivability criterion — a stateless agent given the complete artifact set can derive any valid implementation state.
- **Incremental short loop** (per roadmap item or spec delta): gate = full test suite passes, feature exercised at the HTTP or CLI boundary, documentation cascade complete, Status.md updated.
- **Pre-release loop** (before each environment promotion): gate = release candidate criteria stated in the test architecture document. Required: full mutation testing (pre-deployment), smoke tests across all surfaces, load tests naming the target concurrent user population and p99 latency ceiling, stress tests to failure with documented recovery, dynamic security analysis against the deployed environment. Canary and blue-green rollouts must name the canary population size, error rate rollback threshold, and observation window.
- **Hotfix loop**: minimal targeted fix ships first; post-mortem ADR and cascade artifacts follow immediately after stabilization.

Each milestone is a **phase collapse**: planning, implementation, testing, review, and deploy executed within a single session, converging because the specification holds the full intent and quality gates close the loop before the session ends. A complete practitioner's protocol is in the companion execution guide (`GenerativeSpecification_PractitionerProtocol.md`).

6.6 The Test Architecture as a Specification Artifact

The test suite is a first-class artifact, not supplementary documentation. It specifies observable behavior across every layer, couples to a commit-boundary discipline, and is generated by the AI from the project specification. A project without a stated test architecture has left the verification surface implicit — structurally the same error as leaving the system architecture implicit.

Three executor tiers. The methodology adds an orthogonal dimension to the standard test taxonomy: *who executes and against what environment*. The **automated test suite** covers the full established canon — unit through E2E, executed by CI pipelines. **Synthetic QA** is new: a capable AI agent operating in the staging or live environment reads rendered output, console errors, network traces, and stack traces in a single pass, completing in minutes what QA exploratory testing took hours to surface. **Human QA** is the irreducible judgment layer — aesthetic evaluation and ergonomic assessment — but its scope is shrinking as Synthetic QA expands. The full taxonomy, cross-referenced to commit boundary and ForgeCraft project tag, is in the companion execution guide (§§21–23).

The expose-store-to-window technique. In the test environment, the application state store is exposed to `window`. Playwright can then assert what the application believes is true — not only what renders — catching the class of failure that displays correctly but corrupts internal state.

The vertical chain test. A single UI action is traced through the service layer response, database state, and affected indexes, then back to the visible outcome. One trigger, inspected at every boundary it crosses.

Mutation testing performs an adversarial audit. An AI-generated suite may be written to pass the correct implementation rather than to catch violations of it. Mutation testing closes this gap (Jia & Harman, 2011): by introducing deliberate faults and verifying the suite detects each, it proves detection capability. Coverage measures what was executed. Mutation score measures what was caught. The second is the meaningful metric.

Three-stage multimodal quality gates. Generative asset pipelines require gates standard frameworks were not designed to address. The Shattered Stars case (§7.6) established a three-stage structure generalizable across media — visual assets, audio, and generated code:

- *Stage 1* — programmatic geometry or syntax checks (free, milliseconds): objective library-level assertions. Failure = new seed or adjusted parameters.
- *Stage 2* — composition or architecture analysis (free, seconds): structural properties without a learned model. Failure = parameter adjustment before re-run.
- *Stage 3* — vision, audio, or code model evaluation (~\$0.01/asset): semantic properties detectable only through learned understanding. Failure = structured critique injected into the next generation prompt.

Stages 1–2 filter obvious failures before the expensive evaluation runs. Stage 3 closes the gap between technical compliance and perceptual correctness. The practitioner specifies Stage 3 acceptance criteria before generation begins; the pipeline converges autonomously.

6.7 Use Cases, Diagrams, and Living Documentation

A use case in a generative specification is a multi-purpose production rule, not a requirements artifact superseded by implementation. One precise interaction description seeds three independent outputs: the **implementation contract** (actor, precondition, trigger, postcondition — what the service method is written against); the **acceptance test** (the same artifact transcribed into executable form; test difficulty is a diagnostic for underspecified use cases); and the **user documentation** (the same content with a different framing — a rendering pass, not a writing pass).

Diagram types constitute grammar layers. C4 covers static structure. The temporal and behavioral complement is: sequence diagrams fixing inter-component protocol; state machine diagrams

enumerating valid states and transitions; user flow diagrams specifying the expected path. These are not illustrations — they are constraints the AI reads before generating implementations. A sequence diagram specifying authorization-before-fetch is a stricter constraint than prose, because it is unambiguous about order.

Living documentation derives from the specification. OpenAPI from TypeScript decorators, TypeDoc from JSDoc, changelog from typed commit history — documentation is a derivation from the same source as the code and cannot be wrong in a way the code is right. When the specification layer is complete, inline comments collapse: naming conventions carry semantic signal; ADRs hold decision rationale; use cases hold behavioral intent. A comment that explains *why* is a gap in the ADR record. A comment that explains *what* is a gap in the naming.

A closing observation. The methodology has two separable layers. The process layer — seven specification properties, artifact grammar, commit discipline, tooling — is community-convergent: it can be standardized, automated, and handed off. ForgeCraft automates parts of it; the community will extend it. The specification layer — domain understanding, the ability to name the correct dimension before artifacts are written — cannot be automated. It is the input the process acts on. If the process layer reaches community-maintained solved state, outcome variance becomes a function of specification quality alone. Tool mastery and syntax fluency approach zero as differentiators. What remains is the engineer’s understanding of the problem — which is precisely what the process cannot supply, and what it was never designed to. The structural implications are taken up in §10.

7. Empirical Case Studies

The evidence in this section falls into two distinct categories with different epistemic weights. The six production projects (§7.1, §7.2, §7.4–§7.7; ForgeCraft, §7.3, is the methodology’s own tooling, presented as the self-application case in §7.9 rather than as a proof project) are the substrate from which the methodology was derived: real systems built or refactored under increasingly rigorous GS discipline, with the author as both practitioner and evaluator. They are existence proofs and diagnostic instruments — each project surfaced a failure mode, named it, and produced a corrective property. Built under early, still-maturing versions of the discipline, they carry the weight of discovery rather than of best demonstration; more recent systems built under mature GS are stronger exhibits of why the method works, but the early six are the record of how the rubric was found. The seven-property rubric is their residue, not their premise. The controlled experiments (§7.8 — AX adversarial series, ALX self-applicability, RX replication, BX blind review, EX executable sprint, KX knowledge-retrieval replication) are the testing phase: conducted after the methodology stabilized, with prospectively committed evaluation criteria, designed to falsify rather than illustrate. ALX is the highest-tier controlled result to date: the Loom compiler was derived entirely from its own formal specification, with 386/386 acceptance tests passing ($S_{\text{realized}} = 1.0$), proving spec derivability at the machine-checkable layer above natural language. The Conduit EX experiment (§7.8.D) is the most complete single-project demonstration to date — full T1–T3 proof in a live production environment (development with verified harness, staging governance, production runtime). Observational field corroboration on a paying client team (§7.8.A) provides a practice-side check outside the controlled apparatus. These two categories must be read differently: the six projects are how GS was developed; the controlled experiments are how it was tested.

Six projects across five distinct challenge types document the method’s development and demonstrate that Generative Specification generalizes beyond any single case. Each represents a fundamentally different condition the methodology must address: inheriting unknown foreign code, extending a live system without architectural structure, building from nothing at a simple scale, building from nothing at a distributed system scale, extending an existing system’s domain intelligence, and migrating a broken implementation to a new platform while establishing original IP. I executed all of them with AI assistance. The practitioner’s relationship to the AI changed across this arc: early projects involved sustained dialogue to establish conventions and resolve ambiguities; later projects required only course corrections and extensions, as the specification discipline internalized and the AI’s role shifted from collaborator to executor. That trajectory is itself evidence that GS expertise accumulates — a point developed further in §8.13.

A structural observation applicable across the cases: in both the SafetyCorePro takeover and the BRAD migration, the pre-methodology state was itself produced through AI-assisted development, same tool class, same systems, no architectural constitution. The technology did not change between the before and after states. The specification did. This is not a designed control condition, but it is a natural one whose evidential weight is developed in §7.8.

7.1 Takeover. SafetyCorePro

Production occupational safety management system (SafetyCorePro) refactored from monolithic Next.js to a fully layered, SOLID-compliant architecture. One engineer, three sessions (Feb 14–16, 2026), zero application code lines written by the human. The pre-refactor codebase was produced through unstructured AI prompting — same model class. The comparison is AI without specification against AI with one; the specification is the independent variable.

7.1.1 Pre-Refactor State

The system prior to the refactor exhibited the following characteristics:

- 71 direct database calls from the UI layer (Prisma invocations in route handlers and React components)
- Zero unit or integration tests (23 end-to-end Playwright tests only)
- 227 `console.log` statements used as the logging infrastructure across server-side code
- Business logic distributed across route handlers with no service layer
- A single critical API route performing 100 database queries per page load
- No error hierarchy, bare `throw new Error('something went wrong')` throughout
- 17 missing foreign key indexes across 10 database models
- 126 prior commits built across two external developer identities, establishing the system as a real production codebase before the methodology engineer first touched the repository

7.1.2 The Specification

Before any implementation work began, an architectural constitution was produced: a `CLAUDE.md` file defining the target architecture, quality gates, coding standards, naming conventions, and explicit constraints. The specification was produced collaboratively with the AI assistant and established:

- Target layered architecture: UI → Service → Repository → Database, with explicit dependency rules
- Test coverage threshold: 80% minimum, enforced on every commit
- Error handling standard: custom exception hierarchy, every error carrying an ID, timestamp, and context
- Logging standard: structured logger with level gating and PII redaction
- Naming conventions, function length limits, file length limits
- Explicit forbidden patterns: no direct database calls from route handlers, no hardcoded secrets, no bare exception throws

7.1.3 Results

Metric	Value
Wall-clock time	37.5 hours (includes overnight)
Active development time	8–10 hours across 3 sessions
Commits	10 (atomic, conventional)
Files changed	174
Lines added	16,229
Lines removed	1,889
New test cases	484
New test files	27
Test lines of code	4,898
Direct DB calls removed from UI	71
Repository interfaces introduced	3 (fully swappable)
Custom error types introduced	8
<code>console.log</code> calls replaced	227
Structured logger calls added	263
DB queries per page load (critical route)	100 → 15
Missing FK indexes added	17

The test progression across commits: 75 → 97 → 218 → 285 → 339 → 346 → 484. Architecture materialized progressively. Each commit was independently valid, tested, and deployable.

7.1.4 The Significance

One instruction: “*Make it production-grade and maintainable. One atomic commit at a time.*” The 16,229 lines decompose honestly: ~3,040 dependency metadata and spec artifacts; ~4,840 test lines (zero pre-existed); ~1,200 business logic redistributed into service/repository layers; ~7,150 genuinely new production code (repository interfaces, service layer, error hierarchy, structured logger, RAG/BM25 infrastructure). The headline is the structural transformation: 484 tests from zero, 71 direct DB calls removed, critical route from 100 to 15 queries — one weekend, one engineer, foreign codebase. Two acts constituted the human contribution: authoring the specification, and issuing one instruction. Pre-refactor state preserved at <https://github.com/jghiringhelli/scp-gs-experiment> for reviewer comparison.

7.2 Brownfield. Invellum

Domain: Entrepreneurial ecosystem platform, a social network for founders and entrepreneurs. Connections, social feed, project and campaign management, real-time chat, discovery, notifications, and an administrative console.

Beginning development in June 2025 (eight months before the structured restart), Invellum used earlier AI models as development tools throughout that period, but without a specification to read against, their output had no architectural home. By the time of the methodology intervention, the system had a Next.js frontend, an Express/TypeScript API, and a Prisma-backed schema covering the major domain surfaces: authentication, profiles, connections, feed, projects, campaigns, messaging, and notifications. Working, but not extensible. No architectural discipline, no test suite, no ADRs, no layer boundaries, no specification. Eight months of informal knowledge held the system together in the engineer’s memory, with no artifact form.

The challenge: Extending a live system whose structure existed only in accumulated context. The system worked; the cost was coherence without structure — every feature required reading prior work to understand where it belonged; the AI could produce output but not *coherent* output without a grammar.

The intervention: The transformation point was the introduction of Claude Opus 4.5 and ForgeCraft. ForgeCraft generated the architectural constitution (`CLAUDE.md`), covering layered architecture, SOLID standards, testing requirements, naming conventions, and explicit module boundaries. An ADR directory was introduced. A `Status.md` checkpoint file tracked session-to-session continuity. Playwright-based oracle tests defined the expected behavior of every surface before implementation resumed. The production deployment (Railway backend with PostgreSQL and Redis, Vercel frontend, environment configuration, CORS policy, and production URL validation) was executed from the CLI with Claude directing every step. The spec was not imposed on the existing code, it was written to describe what the system should become, and the system was grown into it.

The results over 36 commits (*qualitative case, no test-count baseline was established before the specification intervention; the figures below reflect the post-specification state, not a before/after comparison*):

Metric	Value
Development history	June 2025 – February 2026 (8 months) pre-specification; earlier AI models used throughout, without specification structure
Post-specification commits	36
Final production state	Live on Railway (Express backend, PostgreSQL, Redis) and Vercel (Next.js frontend)
Test coverage	17 oracle tests, 12 production smoke tests, resource audit, security audit (<i>no quantitative baseline available from pre-specification period</i>)
Security findings	Zero critical findings
Feature surface	Auth, profiles, connections, feed, projects, campaigns, chat, messages, notifications, discover, onboarding, admin console, i18n

After the specification, each feature was an implementation against a contract that already knew where it belonged. The ADRs made session decisions available as context in subsequent sessions. Invellum

remains in active development at time of writing — the only ongoing case in this study. The spec does not expire when the sprint ends.

7.3 Greenfield. ForgeCraft

Domain: Developer tooling. An MCP (Model Context Protocol) server that generates production-grade AI coding assistant instruction files from a library of 112 curated template blocks. Supports six AI assistants, 19 project classification tags, and a tier system. ForgeCraft-MCP 1.0.0 is distributed freely via npm (`npm install npx forgecraft-mcp@latest`). The tool is open source. The project is monetised through consulting engagements with organisations that want guided convergence cycles, bespoke quality gate authoring, or custom integration work. There is no subscription or per-seat fee.

Starting condition: A blank repository. No prior codebase, no inherited debt, no existing architecture. Pure specification-first construction.

The distinctive characteristic of this case: The tool was built using the methodology it implements. ForgeCraft generates generative specifications for other projects. It was itself built as a generative specification from day one. The `CLAUDE.md` that governed ForgeCraft's construction was structurally identical to the documents ForgeCraft would later generate for its users. The methodology was eating its own cooking from commit one.

The specification: The architectural constitution defined the MCP SDK integration contract, the template loading and rendering pipeline as a port/adaptor boundary, the tag classification system as a domain model, and the test coverage requirements. The composition root, the tool handlers, and the registry layer were all specified as interfaces before any implementation existed. Vitest was configured as the test runner with coverage gates enforced by a commit hook.

The initial release (a single commit) shipped with 14 MCP tools, 18 composable tags, 43 template files containing 112 tier-tagged blocks, and 111 tests passing across 9 test suites. There was no prototype phase, no iterative assembly toward a working state. The specification described a complete tool. The first commit delivered one. The subsequent 39 commits are documented feature additions: multi-target assistant support, the tier system, CLI mode, the MCP sentinel, domain playbooks. The breaking rename from `forgekit` to `forgecraft-mcp`, including package name, configuration format, all type names, and every documentation reference, landed in a single commit with zero test regressions. Six months of additions. Nothing revisited.

The results over 40 commits:

Metric	Value
Total commits	40
Current version	1.0.0 (released March 2026)
Tests passing	1127 (current; 111 at initial release)
Template blocks	112
Project classification tags	19
Supported AI assistants	6 (Claude, Cursor, Copilot, Windsurf, Cline, Aider)
Distribution channels	npm (<code>npx forgecraft-mcp@latest</code>). The MCP configuration is declared in <code>forgecraft.yaml</code> ; developers register it once with their MCP client (VS Code Copilot agent mode, Claude Desktop, or any MCP-compatible host).
Breaking refactor	Full rename from <code>forgekit</code> to <code>forgecraft-mcp</code> , one commit, zero test regressions

The role of the specification: The breaking rename (package name, config format, all type names, all documentation) landed in a single commit with zero test regressions — possible because the test suite verified behavior through interfaces, not implementation structure. When the names changed, the behavior contracts held. If GS produces systems resilient to change, the proof of concept is the tool that generates the specification, built under the specification.

7.4 Greenfield (Complex). Conclave

Domain: Multi-role AI orchestration. A system that decomposes a natural language specification into a directed acyclic graph of tasks, assigns each task to a specialized AI role (Architect, Implementer, Reviewer, Tester, Deployer, Auditor), executes them in sequence with inter-role artifact flow, and provides a real-time dashboard for human oversight and gate approval.

Starting condition: A blank repository with a written specification document. The system being built was itself a system for managing AI-assisted software construction, the most structurally complex case in this study.

The challenge: Conclave is a distributed system with a monorepo architecture (9 packages, pnpm workspaces), a DAG execution engine, a message bus with bounce protocol, a rate limiter, a streaming execution layer, a React dashboard with real-time output, and a deployment pipeline with target auto-detection. Each of these is a non-trivial subsystem. Coordinating their construction without the generative specification would have required continuous human navigation of cross-package dependencies.

The specification: The architectural constitution governed the monorepo package boundaries as hard contract lines. Inter-package dependencies were explicitly mapped. Each package was given a single responsibility: core (DAG + state), actions (typed action library), roles (executor registry), dashboard-ui (React orchestration interface), MCP server (external interface), and so on. The DAG engine and message bus were specified as interfaces before any implementation; the bounce protocol and rate limiter were defined as domain models with pure behavior. A `STANDARD_PIPELINE` template with 11 phases and 15 tasks was written as a declarative configuration before any execution path was implemented.

The results over 27 commits:

Metric	Value
Total commits	27
Monorepo packages	9
Pipeline phases	11
Pipeline tasks	15
Tests	203 Vitest + 50 Playwright E2E = 253 total
Dashboard	React orchestration UI with streaming output, gate approval, retry/cancel, history
RAPTOR indexing	Hierarchical codebase summarization (file → module → subsystem → repo) injected into every task context via CodeSeeker integration (see §8.5)
Deploy detection	Auto-detects target from filesystem (Railway, Vercel, Fly, Docker)

The role of the specification: The 9-package monorepo was navigated without cross-boundary contamination in 27 commits — the AI never crossed a package boundary not defined in the architectural constitution. The 253-test suite gave continuous verification across multi-hop failure modes. Complexity is not the ceiling of the methodology; it is the argument for it.

7.5 Extension: BRAD (Legal Intelligence Engine)

Domain: Sovereign legal intelligence engine for US family law cases, live at askbrad.ai. Semantic and structural analysis of case documents: pattern recognition, argumentation logic, and deontic reasoning applied to family law proceedings. The methodology engineer arrived to extend and deepen the analytical capability of a codebase with an established external commit history. Two distinct developer identities are present in the repository.

The challenge: Like SafetyCorePro (§7.1), this case places the methodology engineer in a foreign codebase: another developer’s commit history precedes the specification, and the familiarity objection (addressed in §7.8) applies least here. What distinguishes BRAD from SafetyCorePro is not that structure, it is the demand the extension placed on the specification. The question was not architectural coherence but epistemological precision: not how to extend the system, but which domains the system needed to occupy.

As with SafetyCorePro, the methodology engineer wrote zero lines of application code during the extension phase. The prior developer identity’s commit history was also produced through AI-assisted development without a generative specification. The CLAUDE.md committed March 1, 2026 is the inflection point between both methodologies and both identities: what preceded it was AI output without specification; what followed was AI output with one.

The specification: The architectural constitution from the prior refactor phase was already in place. Extension work was defined as additions to that grammar: new analysis layers specified as interfaces before implementation, new domain models named with the rigor of the domain they served. The specification explicitly named the analytical techniques to be incorporated: prosody and argumentation analysis, discourse analysis, formal fallacy classification, and deontic modal logic (the formal ontology governing obligation, permission, and prohibition that structures legal reasoning). RAPTOR indexing,

first specified for CodeSeeker, was carried forward in the architectural constitution and applied both to the codebase and to the legal document corpus.

The results: The analytical capability added during this extension phase included:

- **Deontic modal logic:** The formal reasoning framework governing obligation, permission, and prohibition, developed by von Wright (1951) and subsequently elaborated for legal and normative contexts, was present in the model’s training corpus from academic legal theory sources and was activated by naming it explicitly in the specification. A generic “analyze arguments” prompt does not invoke this. A specification that names the domain does.
- **Discourse analysis and formal fallacy classification:** Specified as a structured analysis layer producing a taxonomy of argumentation patterns mapped to named fallacy types. The Toulmin (1958) argument model and the pragma-dialectical framework of van Eemeren and Grootendorst (2004) provide the formal vocabulary; both are part of the model’s argumentation-theory training corpus and were activated by naming them in the specification.
- **AI-derived case taxonomy:** The specification directed the AI to derive the classification schema — to identify the natural orthogonal dimensions along which US family law cases differ in legally material ways. The result: a three-tier taxonomy of event types (12 tags), legal relevance tags (8 tags mapping events to statutory implications), and behavioral pattern tags (7 tags identifying relational dynamics). Jurisdiction-aware, calibrated against Minnesota family law statutes (MN 518.003, MN 518.17) and extensible across all fifty states through a jurisdiction-profile system. Practitioner review by family law attorneys remains the appropriate next step for clinical deployment.
- **Property graph knowledge layer:** A graph structure encoding case entities, relationships, and legal claims, schema, ingestion logic, and query patterns, produced from a specification of what the analysis required, not how to implement it.

Metric	Value
Developer identities in repository	2 (saxaboom , prior refactor, 7 commits; Ghiringhelli , methodology extension, 31 commits), build history at https://github.com/jghiringhelli/brad-gs-build
Specification introduced	CLAUDE.md committed March 1, 2026, marks start of extension phase
Post-specification commits	31
Files changed (post-specification)	142
Lines added	23,848
Lines removed	887
Test files	39 (37 unit + 2 e2e Playwright)
Test cases	1,183
Analysis dimensions incorporated	4 (deontic modal logic, discourse analysis, formal fallacy classification, AI-derived case taxonomy)
Knowledge structures introduced	Property graph encoding case entities, relationships, and legal claims (schema, ingestion, and query patterns)
Indexing infrastructure	RAPTOR hierarchical indexing applied to both codebase and legal document corpus

The epistemological finding: The AI’s knowledge of formal legal reasoning frameworks, argumentation theory, and deontic logic is deep — it exists in the training corpus. What determines whether that knowledge is invoked is whether the specification names the domain. “Analyze legal arguments” produces legal analysis. Naming deontic modal logic, argumentation theory, and formal fallacy classification produces a specialist instrument calibrated to each field.

The author names this **domain dimensional expansion** (observed LLM behavior, coined here, with structural parallels to semantic priming, Meyer and Schvaneveldt, 1971): a domain name in the specification functions not as a keyword but as a coordinate — not a retrieval cue but a calibration signal. The model’s response is not a definition of the term; it is the full apparatus of the named field at specialist depth. The pattern holds for engineering concepts equally: “upload guard” named in the spec produced a complete upload validation architecture without line-by-line specification. The concept was the specification; the architecture was its consequence. This effect has been observed consistently across the case studies in this section but has not yet been subjected to systematic controlled testing; §8.5 and the §10.2 limitations section address this gap explicitly.

7.6 Migration. Shattered Stars (x-wing-arcade → TypeScript/Phaser 3)

GS methodology version: vo, pre-experiment series, before the adversarial runs that produced the seven-property rubric, Known Type Pitfalls, infrastructure-first prompting, or the verify loop. Readers should interpret the results in that context: this is the methodology in its earliest form, applied to a demanding migration.

Domain: Tactical arcade space combat game. Five asymmetric factions with distinct playstyle philosophies, 100 ship types (20 per faction), ships within each faction share a coherent aesthetic and color schema that distinguishes them visually from other factions, full ability and upgrade systems, arc-based targeting, maneuver dials, AI opponents with faction-specific behavior, headless simulation for balance testing, and an AI-generated art pipeline via Stable Diffusion. Original IP.

Source system and IP origin: A Unity/C# implementation (“x-wing-arcade”): 108 commits of built gameplay drawn from the mechanical foundations of a well-known tabletop miniatures game (readers familiar with FFG’s *X-Wing Miniatures* will recognize the arc-based targeting, maneuver dials, and dice-based combat resolution). The specification was complete. The code was substantially implemented. The execution was deeply broken: runtime defects had accumulated across movement resolution, combat state, and scene management to a state where the game did not run correctly.

Current state (March 2026): See milestone table below. Playable game; campaign mode, additional modes, and balance simulation in active development.

A central technique enabling this progress is a visual execution loop that the browser-based runtime makes possible. The AI executes the game step by step, reads the browser console log output, takes screenshots and analyzes them using Claude’s vision capability, sends keyboard and mouse input, and iterates, confirming that sprites are rendered and animated correctly, that the targeting arcs track as specified, and that faction AI behaves according to its behavioral template. This closes exactly the feedback loop this work’s prior tooling discussion identified as the ceiling for Unity-based AI development: the tight read-run-observe cycle that terminal-based AI could not perform. In a browser runtime, it can. The technique is not specific to games. Any system with a visual output surface and readable log channels becomes an inspection and correction target for this loop.

The migration was also driven by a tooling gap: Claude’s integration with Unity was limited at the time, and the MCP ecosystem that now enables deeper editor interaction did not yet exist. The practical ceiling for AI-assisted Unity development was low for exactly the class of defects that had accumulated (runtime behavior, physics edge cases, scene lifecycle), which require the kind of tight read-run-observe loop that terminal-based AI tooling handles poorly in a Unity context.

The migration served two purposes simultaneously. **Platform reach:** Unity targets desktop builds; the intended delivery is a web-hosted static application deployable to Netlify, Vercel, or itch.io with no installation. **Original IP:** A game built on borrowed mechanics is a prototype. At migration time, Shattered Stars did not yet have a name, factions, ships, lore, or visual identity.

The specification step was also an extraction. The behavioral contracts of the Unity implementation (what each system did, not how it did it) were pulled from the existing codebase and expressed in platform-independent form. The broken execution became irrelevant. What the Unity codebase contained was a complete specification of game behavior. That specification was extracted, cleaned of any Unity API reference, and the new implementation was written against it. A broken implementation is, structurally, a complete spec with a bad executor. The methodology replaces the executor.

The specification step, before rewriting a single TypeScript file:

The methodology’s claim in a migration context is that the behavioral contracts of the source system can be extracted and expressed as a platform-independent specification, one that describes *what* the system does without referring to Unity, C#, MonoBehaviour, or any API the new stack will not have. That specification then becomes the authoritative grammar against which the new implementation is written.

The output of this step for Shattered Stars:

Artifact	Content	Scale
specs/TECH_SPEC_AND_ROADMAP.md	Full platform-independent architecture: tech stack decision with comparison tables, all game systems formally specified, AI system design, rendering pipeline, balance simulation framework, risk register	2,277 lines
specs/ directory	25 supporting documents covering faction mechanics, ship stat distributions, maneuver dial definitions, UI wireframes, art generation pipeline, sound/music assets, progression systems, special ability distributions, crew point costs, point budget tables	25 files
DEVELOPMENT_PROMPTS.md	Session-scoped implementation prompts, one self-contained prompt per subsystem, ordered by dependency, each supplying the exact contract the AI session needs to implement that system without carrying forward context from prior sessions	1,496 lines
CLAUDE.md	Architectural constitution for the new stack: TypeScript strict mode, Phaser 3 patterns, commit policy, key system inventory	Project root

None of these documents mentions `MonoBehaviour`, `GameObject`, `SerializeField`, or any Unity API. The spec describes faction playstyle (“swarm tactics, expendable; shared targeting data”), maneuver resolution (“conversion of maneuver + current position/angle into target position/angle using Bezier curve path points”), and combat properties (“dice rolling, damage resolution, stress tracking”). The platform changed. The behavioral contracts did not.

Timeline: Specification written in a focused session (`TECH_SPEC_AND_ROADMAP.md`, 25 supporting spec files, `DEVELOPMENT_PROMPTS.md`, `CLAUDE.md`). Each subsystem prompt in `DEVELOPMENT_PROMPTS.md` was fed to an AI session; the engineer initiated, reviewed, and moved to the next prompt without co-authoring

code. All 64 source files committed in a single batch March 7, 2026; art validation March 8. Active engagement at two points: writing the spec, and returning for the art pipeline.

Implementation results:

Metric	Value
Source files (src/)	64 TypeScript files
Total source lines	32,470
Game systems implemented	16 (CombatSystem, ActionsSystem, TargetingSystem, MovementSystem, OrdnanceSystem, TurretSystem, PilotAbilities, UpgradeSystem, SquadBuilder, FlightControlSystem, PerformanceSystem, FactionAbilities, ManeuverTemplates, TurnSystem, TurnManager, CampaignSaveManager)
Additional modules	AI system (utility-based, 5 faction personalities × 4 difficulty levels), procedural audio engine (Web Audio API), rendering pipeline, headless balance simulation framework, 8 scenes, full UI layer
Test files	17 (435 individual test cases)

Milestone state (March 2026):

Milestone	Status
Core Systems. Movement, Combat, Targeting, Actions, Turn	✔ Functional
AI System, behavior templates, closest-enemy distance + angle, faction-specific	✔ Functional
Ship Definitions: 100 ships, 20 per faction, individual style and color schema	✔ Complete
Arcade Controls, real-time gameplay	✔ Functional
Menus & UI, main menu, settings, credits, lobby, pause, game over	✔ Complete
Audio, sounds from free sources	✔ Integrated
Pre-built Squads	✔ Complete
Image Generator Service. Stable Diffusion API integration	✔ Pipeline built
Visual Execution Loop. Claude Vision + console log + input cycle	✔ Active development technique
Campaign Mode, full lore history, AI-generated portraits	🔄 In progress
Additional Game Modes	🔄 In progress
Balance & Testing, headless simulation framework	🔄 In progress

The migration extracted a complete behavioral specification from a broken Unity codebase and drove autonomous generation of 64 TypeScript files across 16 game systems — producing a playable game with 100 ships, functional AI opponents, complete menus, and integrated audio.

Longitudinal note. Executed at GS vo — before the seven-property rubric, Known Type Pitfalls, or the verify loop. A game of this scope reaching a playable state under an early, unvalidated methodology is the finding. The project will be presented as a longitudinal case when complete: same codebase, same engineer, tracked from GS vo through current methodology, with ForgeCraft as the active tooling throughout.

On commit discipline: The git history is thin — two commits capturing the after-state, not the construction sequence. ForgeCraft at that version generated the architectural constitution and session prompts but did not initialize a git remote; the gap was discovered at push time and has since been addressed. The structural finding: the specification held behavioral contracts and architectural coherence across sessions without the commit corpus. What the audit trail enables is *context recovery* —

the spec is not a substitute for commit discipline, it is what survives when commit discipline is not applied. The repository is publicly accessible at https://github.com/jghiringhelli/shattered_stars.

The three sub-sections below show what the methodology executes across surfaces once a specification exists: environment configuration, generative art production, and automated visual QA. Each follows the same structure as the code work above.

7.6.1 Environment Setup as a Specification Problem

The desired state — a GPU-accelerated Stable Diffusion instance ready to accept API calls — was specified and handed to the AI. After several cycles of dependency diagnosis (version conflicts, CUDA gaps, driver mismatches), it identified a pre-packaged distribution with known working configurations, installed it, and verified the service was responding. A follow-up prompt to optimize throughput led it to identify batch size and VAE precision mistuned for available VRAM and adjust both. Desired state + acceptance criteria + agent iteration. The medium differs from code; the structure does not.

7.6.2 Automated Art Validation

Ship sprites (100 total, 20 per faction) are generated via Stable Diffusion with a precise contract: top-down orthographic view, vertically symmetric, pure black background. Prompt engineering (positive/negative framing) reduced rejection rates but not to zero; manual review at scale is not a pipeline. The solution: specify acceptance criteria as executable validation. `scripts/generate_sprites.py` implements four checks before a sprite is accepted:

Check	Mechanism	Threshold
Vertical symmetry	Compare pixel-level left and right halves after horizontal flip; normalized 0–1 similarity	≥ 0.85
Clean background	Measure ratio of non-black pixels in the 20-pixel border region	≤ 0.30
Vertical orientation	Principal component analysis on the ship’s pixel mass; extract angle of principal axis from vertical	$\leq 15^\circ$
Centering	Center-of-mass offset from image center	Informational

A sprite that fails any check triggers regeneration, up to three retries per ship. Ships that pass are logged to a preservation list so re-runs skip them. The symmetry threshold was tightened from 0.70 to 0.85 mid-project after reviewing the first batch, the initial threshold accepted sprites that were technically symmetric but visually lopsided.

The evolved architecture: staged convergence. The flat four-check baseline performs all checks at the same cost tier. The evolved design is cost-stratified: Stage 1 (geometry — symmetry, background, orientation, color histogram) is free and runs in milliseconds; Stage 2 (composition — centering, edge density, aspect ratio) is free and runs in seconds; Stage 3 (vision model evaluation — style consistency, archetype match, quality ranking against accepted sprites) costs ~\$0.01/image. Each stage failure drives a structurally different corrective action: seed randomness (Stage 1), generation parameters (Stage 2), or prompt content with the vision model’s specific critique injected (Stage 3). Most rejections fail before Stage 3. The pipeline converges autonomously. The human’s role is to define Stage 3 acceptance criteria before generation begins.

The pattern: desired output specified as a measurable contract at three distinct cost/abstraction levels, each driving its own corrective feedback. The same logic that governs whether a TypeScript module

satisfies an interface governs whether a sprite satisfies its visual requirements. The artifact type differs. The underlying principle is identical.

7.6.3 Visual QA via Screenshot Analysis

The bug-squashing phase introduced a fourth pattern. Playwright runs against the live game and captures screenshots at defined interaction points: game states, combat sequences, UI transitions. These screenshots are passed to the AI, which reads them visually and identifies defects, misaligned UI elements, incorrect game state rendering, ships in positions they should not occupy. The defect description is then fed back as a fix prompt.

This is manual QA operationalized. The human role in a traditional QA cycle is to run the game, observe the visual output, identify what is wrong, and report it. That loop is now closed by the AI reading the screenshot. The Playwright harness provides repeatability; the visual analysis provides the judgment that a test assertion cannot, because some classes of defect are only visible, not textually detectable. The game iterates toward correctness the same way the art pipeline does: specify the acceptable state, observe the actual state, close the gap.

A more demanding variant closes the full vertical slice. A Playwright interaction fires a UI action; the AI then queries the service layer response, the database state, and any affected indexes, verifying that the effect propagated correctly through every layer, then returns to the UI to confirm the visible outcome matches the stored state. This is not a unit test and not a visual check. It is a chain verification: one trigger, observed at every boundary it crosses. A defect anywhere in the chain (service logic, persistence, index consistency, UI rendering) is surfaced in a single pass. The specification defines what the chain should produce at each layer; the AI runs the chain and reports where the actual state diverges from the specified one.

7.7 Regulated Multi-Layer Data Platform (T2 + T3)

Challenge category: Regulated cloud ETL platform with specification-governed infrastructure and self-healing diagnostic governance

Context: A public-sector data platform processes documents from multiple heterogeneous sources — court records, clinical systems, administrative data — through a medallion architecture: Bronze (immutable raw storage) → Silver (structured, extracted, validated) → Gold (resolved, patient-centric, analytics-ready). The platform integrates polyglot persistence (S3, DynamoDB, Neptune graph, OpenSearch) across four access patterns and processes sensitive data under compliance requirements. No code is shared; the architecture description is documented here as the evidence.

7.7.1 T2: Specification-Derived Infrastructure

COMPASS is a configuration-driven pipeline: CDK reads a single YAML specification and derives the entire cloud environment (S3, DynamoDB, Neptune, OpenSearch, Lambda, Step Functions, EventBridge). A mandatory NFR layer — `CompassNfrAspect` applied at synthesis time — cannot be disabled: all DynamoDB tables receive CDC streams, all Lambdas receive X-Ray tracing, all resources receive required tags. Deployments can configure thresholds but cannot remove obligations. Compliance policy (HIPAA, audit log retention) is enforced as a synthesis-time type-level constraint, not a launch checklist. The practitioner issues no CLI commands, edits no infrastructure files.

7.7.2 T3: Specification-Governed Self-Healing

The diagnostic agent — “The Eye” — has read access to the complete runtime specification of the system: the lineage graph (DynamoDB), the processing manifest (per-document version history), CloudWatch logs and metrics, X-Ray distributed traces, GitHub commit history, and CDK blueprint versions. It has no access to PII values, credentials, or file contents — only to metadata, hashes, scores, and version information. This access boundary is specified, not enforced by convention.

The formal substrate that makes diagnosis possible is the **process hash**:

```
process_hash = sha256(bronze_file_hash | script_version | config_version | llm_prompt_hash)
```

This is not a performance optimization. It is a formal invariant: the complete specification of what valid processed state looks like for a given document at a given pipeline version. If the bronze source changes, the script version changes, the configuration changes, or the LLM prompt changes, the hash changes — and the specification declares the existing downstream state invalid. The pipeline is designed so that the practitioner never manually identifies which documents need reprocessing. The hash tells the system, by formal contract, which states are stale.

The lineage graph stores the process hash, bronze hash, script version, config version, and prompt hash for every transformation event — a complete specification of every state every document has passed through. When a problem is reported, The Eye traverses this graph backwards from the reported symptom, comparing actual state to the specification at each layer, and stops when it finds a discrepancy. It does not scan the full system. It reads the formal record and halts at the first specification violation.

Root cause taxonomy: seven types (low confidence, stale data, code bug correlated with git commit, config change, source corruption, missing rule, human-review edge case). Data-driven failures trigger autonomous reprocessing; code-driven failures generate a structured GitHub issue and await deployment approval. Safe actions execute without approval; escalation actions and DataZone governance approval gate any output change — the human governance role preserved at the boundary the specification has not yet closed. The practitioner is released from: diagnosis, root cause investigation, document reprocessing identification, and correction decisions for known failure types.

7.7.3 The Formal Connection Between T2 and T3

The COMPASS case demonstrates something the six prior cases cannot: the dependency between tiers. T3 diagnosis is only possible because T2 captures the complete provenance of every infrastructure and processing decision. The process hash is a formal contract that propagates from the specification downward through every pipeline stage. The lineage graph is not logging; it is the runtime specification of what valid state looks like at every step. When the diagnostic agent reads the lineage graph, it is reading the specification of what should have happened and comparing it to what did happen. The correction mechanism derives from the same source as the construction mechanism, which is the T3 claim stated precisely.

The case also demonstrates the restriction/liberation duality operating across both tiers simultaneously: the mandatory NFR layer (T2 restriction) is what makes proactive monitoring and compliance audit (T3 capability) possible without practitioner intervention. A system whose infrastructure is not fully governed by specification cannot have its runtime behavior evaluated against that specification. T2 completeness is the precondition for T3 correctness.

7.8 Threats to Validity and Experimental Closure

The Define/Build/Measure Loop

The primary methodological challenge is not any single threat, it is a structural one that runs through the entire experiment series. GS defines the seven specification properties. GS guides the AI to produce implementations that satisfy those properties. GS scores whether the implementations are good. A discipline that defines “good,” builds toward it, and then measures whether it achieved it has a circularity problem that no amount of external checking fully eliminates. This is the load-bearing concern. Every other threat in this section is secondary to it.

The loop has three distinct layers, each requiring a separate closure mechanism:

Layer	Threat	Closure Mechanism	Status
3, Output measurement	External checks use criteria the rubric author defined	<code>tsc --noEmit</code> , <code>ESLint (eslint:recommended + @typescript-eslint/recommended)</code> , <code>npm audit</code> (supply-chain gate, not code quality), and the Conduit test suite, 104 tests authored by the open-source community, not this work’s author	Closed
1, Rubric validity	Rubric rewards GS compliance, not objective quality	BX: three Conduit implementations never built with GS scored blind, rubric ranking (13/14 > 7/14 > 6/14) congruent with CVE count, test count, and TypeScript health on every axis	Closed
2, Guidance circularity	GS guided the implementation AND scored it	BX/RX: implementations built and independently replicated without GS guidance, scored against the same rubric (one battery predates GS); observational field corroboration on a real team (§7.8.A)	Mitigated; controlled human-participant study noted as future work

The ordering is deliberate: Layer 3 is the weakest mitigation (the tools are independent but the implementations were still GS-guided); Layer 1 is stronger (the rubric is applied to implementations it never guided); Layer 2 is the deepest concern — the guidance-plus-measurement loop. It is mitigated by BX/RX (implementations GS never guided, scored blind against a rubric one battery of which predates GS) and corroborated observationally by a paying practitioner cohort on its own codebase (§7.8.A). A controlled human-participant study closing Layer 2 directly is noted as future work.

One finding from BX merits explicit disclosure: the Defended property reveals a persistent tooling-layer limitation. No implementation across AX, BX, or RX scores 2/2 on Defended. A CI pipeline can be specified in a GS document; it cannot be provisioned by generated code. CI runners require external infrastructure that AI-assisted generation cannot physically create. This is not a gap in the methodology: the specification correctly describes what must exist, the convergence loop requires that nothing important is deployed before it runs, and for safety-critical domains the methodology explicitly calls for human review on top. The 0/2 score is an honest record of what the current generation of AI tooling can deliver, not a bound on the methodology’s claim. It is documented here rather than scored aspirationally.

Additional Threats

Threat: Single-engineer design introduces selection bias. *Assessment:* The codebase-familiarity objection does not survive the evidence — SafetyCorePro and BRAD both carry forensically

distinct prior contributor identities; the methodology engineer arrived as an outsider to foreign code. The specification-authorship confound remains: every specification was written by the methodology's creator. This limitation is partially addressed by BX and RX (external implementations and independent replication, neither GS-guided) and corroborated observationally on a real team (§7.8.A); a controlled human-participant study providing a pre-written specification to practitioners who did not author it is noted as future work. The methodology establishes a floor; the ceiling is set by the practitioner's domain depth. Independent replication will quantify how much the floor alone accomplishes.

Threat: No control condition. *Assessment:* SafetyCorePro and BRAD both provide natural controls: the pre-methodology codebase was itself produced through unstructured AI prompting, same model class. Technology was constant across before/after states; the specification was not. The structural differences (zero tests → hundreds, monolithic → layered) cannot be attributed to the technology. Before/after states are preserved in version control and available to reviewers on request.

Threat: Self-referential rubric. *Assessment:* Substantially true; mitigated at three levels matching the loop table above. Layer 3: external checks (`tsc` , ESLint, `npm audit` , the open-source Conduit 104-test suite) are rubric-independent and directionally consistent with GS rubric scores. Layer 1: BX scores three RealWorld implementations without GS exposure; rubric ranking ($13/14 > 7/14 > 6/14$) is congruent with CVE count and test coverage on all axes. Layer 2: BX applies a dual rubric where one battery predates GS to implementations GS never guided, and observational field corroboration (§7.8.A) shows the findings recur on a real team. The residual circularity in the GS audit scores is named honestly; a controlled human-participant study is its designed resolution and is noted as future work.

Threat: Self-reported metrics. *Assessment:* Core claims (commits, files, lines, test counts, layer boundary conformance) derive from git history, reproducible from repository access available on request. Multi-author attribution is forensically verifiable from commit log. The one genuinely self-reported metric is active development hours; commit timestamps confirm plausibility but do not verify precisely.

Threat: Incomplete tooling confounds results. *Assessment:* This is a directional argument, not a threat. If incomplete tooling produced these outcomes, complete tooling establishes a floor: the results are a lower bound on the methodology's potential.

Threat: AX ran on a single model. *Assessment:* All AX conditions were executed on claude-sonnet-4-5. Cross-model generalizability now has initial pilot evidence: MX (§7.8.H) held the GS specification constant and varied only the model, and a mid-tier (Sonnet 4.6) and a strong (Opus 4.8) model scored identically — 149/149 on the full Conduit backend — indicating the measured effect is a property of the specification rather than of any one model. Broader, controlled cross-model replication is still invited. The expected direction on any instruction-following model is positive: the value GS adds is structural (artifact completeness, boundary explicitness, decision persistence), not prompt phrasing. A more capable model should require less scaffolding; a less capable model may require more. The magnitude of the effect is model-dependent; the direction should be model-independent. Independent replication on alternative models is invited.

Threat: GS does not eliminate defects. *Assessment:* True, and stated plainly. GS reduces the *class* of defects produced by interpretive variance — drift, unstated assumptions, context lost across sessions — and makes the residual localizable against the specification; it does not render a stateless reader infallible within a single derivation. The detection layer for that residual is the Executable property exercised as *generative execution*: verification that drives the running system through its use cases across independent signals — persisted state, interface, contract, logs — rather than the static test suite alone, and never the agent's self-report (see the Darwin-Gödel reward-hacking evidence, §5). Without

that layer, residual defects ship undetected; with it, they surface and localize to the specification, where the fix is made and the implementation regenerated. The claim the evidence supports is therefore not zero defects but a reduced and *detectable* defect surface.

Independent replication is invited. Experimental conditions, prompts, and scoring rubric are archived at <https://github.com/jghiringhelli/generative-specification/tree/main/experiments/ax/>. It is the author's explicit intent that the methodology be tested and that it survive the test.

7.8.0 Validation Strategy

Two experiment series form a layered case, with observational corroboration as a practice-side check. **AX** (solo layer): the methodology applied across eight conditions (plus a ninth construction-invariance condition, §7.8.E) and measured against an external rubric — closes whether the procedure can be executed consistently, not whether it is self-referential. **BX/RX** (peer layer): three RealWorld implementations built without GS knowledge, scored blind — closes circularity at the output measurement layer using the open-source 104-test Conduit suite. **Observational field corroboration** (§7.8.A): a paying practitioner cohort applying GS on its own codebase — a practice-side check that the findings recur outside the controlled apparatus. The guidance layer is mitigated by the peer layer and corroborated observationally; a controlled human-participant study closing it directly is noted as future work.

7.8.A Observational Field Corroboration: A Paying Practitioner Cohort (June 2026)

The controlled experiments (§7.8.B onward) are necessarily artificial: fixed-time greenfield tasks scored by a blind evaluator. This subsection adds *observational* evidence of a different kind: a four-day GS workshop delivered to a paying client — an eight-developer cohort of non-senior practitioners at a mid-size insurance brokerage (June 2–5, 2026). It is reported deliberately as observational, not controlled: there is no control condition, no blind scoring, no pre-registration, and a single self-selected cohort. It cannot test a hypothesis. What it can do is show whether the controlled findings recur when GS meets a real team on its own codebase, and surface the field shape of the one recurring objection. It carries the practitioner-transfer evidence for this work; a controlled human-participant study is noted as future work. The full transcript corpus is archived; quotes below are translated from Spanish and lightly cleaned from automatic transcription.

Single-engagement transfer, on the cohort's own work. On the greenfield day every project demonstrated produced a running artifact within a single morning, and in each case the residual defects were traceable to *under-specification* or a missing external asset rather than to the method — the same signature the methodology predicts. The sharpest datum is a participant diagnosing his own failed build without prompting: “*it is absolutely my fault, because I did not specify it correctly.*” A practitioner attributing a generation failure to a specification gap, rather than to the tool, is the methodology's central claim stated back from the field. By the brownfield day every participant had mapped GS onto an actual production system, and one instituted a process change of his own accord — making an architecture decision record a merge prerequisite (“*now the doc is required to pass committee; without it, it does not pass*”). That is adoption expressed as a gate, not as sentiment.

The cost objection, observed as a conversion curve. The token-expenditure objection (§4.1.b) was the only substantive objection raised, and it recurred across all four days with a consistent trajectory: initial caution about spend, an acute mid-workshop episode in which several practitioners exhausted their token budgets during iterative generation (some retreating to free or local models), and a closing posture of qualified willingness to adopt anyway. The facilitator's reconciliation, refined in

front of the cohort, is the honest form of the §4.1.b argument and worth recording precisely: *under inversion of control the absolute token spend rises, because the practitioner advances faster; what is purchased is time, not tokens*. The per-output economics (fewer tokens per accepted line, via authored structure the reader need not re-derive — §4.1.b) and the absolute-spend increase are both true and not in tension; conflating them is the source of the objection. This observation motivated the controlled retrieval measurement (KX, §7.8.E) and is consistent with it.

Status of this evidence. Observational, uncontrolled, single-cohort, facilitator-delivered — it establishes nothing on its own and is not weighed in the rubric or the inference statistics. It is included because the controlled studies are necessarily artificial (fixed-time greenfield tasks scored by a blind evaluator) and a methodology's claim to practice is incomplete without at least one account of it meeting a real team and a real codebase. Read it as corroboration and as a source of the failure modes and objections that the controlled program must then test, not as a result.

7.8.B Experiment I: Multi-Agent Adversarial Study: Results

Three findings drive everything that follows. **The primary prospective finding is narrow and should be stated plainly:** the GS v1 treatment scored 10/14 against the control's 9/14 — a one-point difference on a single benchmark, where GS's sole advantage was the Composable property, traceable directly to a SOLID clause the control did not include. On Executable specifically, the control outperformed GS v1: the control's full suite passed; GS v1's suite had 6/10 suites blocked by a JWT type narrowing pattern not yet named in the specification. A one-point prospective advantage on one benchmark is not a paradigm demonstration. It is the starting point for a diagnostic series. First: the boundary that matters is naive vs. structured — unstructured AI deployment produced an internally incoherent project with zero passing tests; both structured conditions produced compilable, layered code. Second: 14/14 on the full seven-property rubric is achievable, the post-hoc conditions demonstrate this, with the explicit caveat that treatments v2 through v6 were designed with full knowledge of each prior condition's gaps. This is iterated optimization on a single benchmark, not independent confirmation; the progression from 3→14 is the evidence that each gap is diagnosable and closable, not a statistical demonstration of convergence behavior. Third: the experiment both measured and corrected the methodology — the three template changes confirmed by treatment-v2 were committed to production templates and propagate to every GS-governed project; the gap between experimental finding and production tooling is zero.

One benchmark (Conduit), one model (claude-sonnet-4-5), one author's specifications: the AX study is N=1 on every structural axis. The empirical claim, a quality gradient observable under controlled conditions, is stated without population-level authority; the paradigm claim does not require it, being a structural argument about what specification completeness permits an executor to derive, not an effect size across benchmarks. The AX findings establish proof-of-concept for the measurability of the GS rubric and the correctness of gap diagnosis: each condition shows a diagnosable, closable gap. External static analysis tools (tsc, ESLint, madge, jscpd) independently corroborate the direction across all 8 conditions without access to the GS property definitions.

Benchmark: RealWorld (Conduit) backend API, a full-featured REST application (authentication, articles, profiles, comments, tags, favourites) implemented in TypeScript/Node.js/Express/Prisma against a live PostgreSQL database. Chosen because it has a published specification, a community Postman collection for conformance testing, and known correct implementation patterns, making automated evaluation straightforward and reviewer replication feasible. All three conditions ran on

claude-sonnet-4-5, March 13 2026. The experiment design and scoring rubric were prospectively committed in commit `bd2c05b` before any condition was run (verified by timestamp at <https://github.com/jghiringhelli/generative-specification/tree/main/experiments/ax/>).

Eight conditions — three prospectively designed (committed to the public repository before execution, verified by timestamped commits at <https://github.com/jghiringhelli/generative-specification/tree/main/experiments/ax/>) and five post-hoc; a ninth, the construction-invariance condition, is reported separately in §7.8.E:

7.8.B.1 Results

Condition	Key additions	Pre-registered
Naive	3-line README, 4-line prompts — de facto default	—
Control	Detailed README + 7 prompts averaging 30 lines (expert prompting)	✓
Treatment (GS v1)	17 GS context files (CLAUDE.md, Status.md, 4 ADRs, C4, use cases, NFR); 8-line prompts	✓
Treatment-v2	+ “Emit, Don’t Reference” directives; First Response Requirements list of 9 mandatory P1 artifacts	No
Treatment-v3	+ Dependency governance (argon2 over bcrypt; <code>npm audit</code> gate as P1 requirement)	No
Treatment-v4	+ ADR emission precision fixes; verify loop (materialize→tsc→jest→correct, max 5 passes)	No
Treatment-v5	Dedicated <code>00-infrastructure.md</code> prompt before feature prompts; Known Type Pitfalls in CLAUDE.md	No
Treatment-v6	+ DRY gate (jscpd < 5%); Interface Completeness gate; ESLint as P1	No

GS audit scores, unified 7-property rubric (all conditions re-audited; AI auditor agent, blind session, no prior context of the authoring conditions, scale 0–2 per property; no inter-rater reliability check was conducted across human evaluators):

Here N=7 refers to the seven specification properties in the GS rubric, not to participant count; each experiment iteration tested derivation quality as a function of rubric compliance score. Across the eight conditions, two iterations — Treatment-v3 and Treatment-v5 — reached the 14/14 ceiling, the rest scoring below it along a documented non-monotonic path (the Treatment-v4 regression to 11/14).

Property	Naive	Control	Treatment	T-v2	T-v3	T-v4	T-v5	T-v6
Self-Describing	0/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2
Bounded	1/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2
Verifiable	1/2*	2/2	2/2	2/2	2/2	2/2	2/2	2/2
Defended	0/2	0/2	0/2	2/2	2/2	1/2	2/2	2/2
Auditable	0/2	0/2	0/2	1/2	2/2	1/2	2/2	1/2
Composable	0/2	1/2	2/2	2/2	2/2	2/2	2/2	2/2
Executable	1/2‡	2/2‡	2/2‡	2/2‡	2/2‡	1/2	2/2†	2/2†
Total	3/14	9/14	10/14	13/14	14/14‡	11/14	14/14†	13/14†

* Naive Verifiable: the original audit scored this 2/2 based on test structure and naming; the unified re-audit applied a stricter behavioral criterion, tests must compile and tests must run, reducing the score to

1/2. All six naive test suites fail to compile due to missing schema models (real coverage 0%). # Naive—Treatment-v3 Executable is auditor-inferred: the re-audit assessed whether generated code compiles and tests exist, not whether tests pass against a real execution environment, no verify loop ran for these conditions. † Treatment-v5 and Treatment-v6 Executable 2/2 is session-verified. Treatment-v5: the verify loop confirmed 109 total tests (independent re-run: 106 passing, 3 test-isolation failures in `article.test.ts`, duplicate user registration in a preceding test leaves token undefined in cleanup; not implementation errors) across 10/11 suites against a live PostgreSQL database, converged in 2 fix passes (four runner infrastructure bugs fixed before the verified run, see companion supplement §S9.6). The AI integration response reported 114 total tests; 109 is the runner-confirmed count and is the figure used throughout. No `jest-output.json` artifact was committed for v5 in the way that RX evidence was committed to [experiments/rx/evidence/](#); the verification was conducted within the audit session rather than as a reproducible committed artifact. This is the remaining epistemic gap between v5 and RX. Treatment-v6: `session-summary.md` Final Results table confirms 62/62 tests passing with 0 tsc errors and 0 ESLint errors, verify loop converging in 3 fix passes. Treatment-v3 and treatment-v5 share the same score (14/14) with completely different epistemic bases: treatment-v3's score is auditor-inferred from static artifacts; treatment-v5's is session-confirmed by a passing test suite against a live database. The verify loop's value is not the score, it is the guarantee that the score reflects something real.

Treatment-v6: 13/14. Sole gap: Auditable 1/2 — `Status.md` absent. Executable 2/2 session-verified: 62/62 tests, 0 tsc, 0 ESLint errors. First Stryker mutation gate in `ci.yml`, not yet scored under rubric. Full per-condition justifications at [experiments/ax/treatment-v6/evaluation/scores.md](#).

The progression 3→9→10→13 (out of 14) across the first four conditions is directionally consistent on most instruments: tsc error count decreases monotonically across all four; npm CVE counts do not follow the same trajectory. The direction is unambiguous; the magnitude, as single-model single-run evidence, is not. Full unified scoring across all eight conditions: companion supplement §S5. Pre-registered prediction vs. actual comparison: §S11. Blind adversarial audit methodology: §S7.

Primary finding — naive vs. structured: Naive produced an internally incoherent project; all six test suites fail to compile (schema additions described in prose, not emitted as code blocks). Both structured conditions produced compilable, layered code. The naive→structured boundary is the finding that matters; control→treatment (one point, Composable) is a directional signal.

Defended floor — behavioral constant: All three prospectively-designed conditions scored 0/2. A GS artifact that *specifies* a hook does not cause the hook to *exist* — models treat specification text as guidance for application code, not as directives for operational infrastructure. The fix: fenced file templates in a First Response Requirements list cause emission. Defended moved 0→2 in treatment-v2. Failure was instruction precision, not model capability.

Treatment-v2 achieved 12/12. All three gaps (Defended 0→2, Auditable 1→2, Composable maintained) closed in a single post-hoc run. The changes were centralized: three additions to `templates/universal/instructions.yaml`, propagating to every GS-governed project. An expert-prompting practitioner who discovers the same gaps must update every project README individually. That asymmetry — one template change vs. N project changes — is the democratizable difference.

Mutation score gap: GS v1 reported 93.1% line coverage but 58.62% MSI. Line coverage measures execution; mutation score measures detection. A test that covers a line without asserting correctness passes coverage and fails mutation. After three rounds of Stryker-guided assertion improvements, the treatment project's mutation score converged to 93.10% — matching line coverage. The same gap appeared in Shattered Stars (80% line coverage, 58% MSI). When both converge, every covered line is verified. Treatment-v2 added the mutation gate as a hard P1 criterion.

What GS adds over expert prompting: Bounded (layer discipline) is achievable through expert prompting. GS’s specific contributions are Composable (interface-based DI), Auditable (decision record persistence), and Defended (operational enforcement infrastructure). The democratizable difference is compounding: GS improvements cascade forward via template; expert-prompting improvements stay local.

Three template changes confirmed by treatment-v2:

- 1. *Mutation gate as a hard quality criterion.* MSI $\geq$ 65% overall blocks PR merge; $\geq$ 70% on new/changed code. Run Stryker per module after test authoring, not only at release. Propagates to all GS-governed projects on next `forgecraft refresh_project`.
- 2. *Emit, don’t reference.* Infrastructure files, hooks, CI workflows, commitlint configuration, ADR stubs, IRepository interfaces, are named as files to be emitted in P1 with fenced templates. Treatment-v2 confirmed this change produces the artifacts; prior treatment specified them and produced none.
- 3. *Line coverage and mutation score are complementary, not interchangeable.* Line coverage measures execution; mutation score measures detection. A gap between them is the fraction of covered code that tests cannot verify. When they converge, every covered line is verified. Both gates are required.

Post-publication finding: architectural compliance and dependency security are fully orthogonal. Static quality checks were run across all four materialized conditions after the primary results were reported. The three checks required no running server and were applied with consistent flags across all conditions.

Check	Naive	Control	Treatment	Treatment-v2
<code>tsc --noEmit errors</code>	41	1	0	0
ESLint problems (bare baseline)	29	40	40	21
<code>npm audit high CVEs</code>	3	0	3	9

† ESLint violations measured with `eslint:recommended` + `@typescript-eslint/recommended` applied consistently across all conditions. The increase from Naive (29) to Control/Treatment (40) reflects additional source files generated in later conditions, not a degradation in per-file quality; Treatment-v2’s reduction to 21 reflects the DRY and interface completeness gates specified in that condition.

Note: `tsc --noEmit` and ESLint measure static code quality, compiler correctness and style/safety rules respectively. `npm audit` measures supply-chain hygiene, known CVEs in the dependency graph. Both categories are rubric-independent; neither subsumes the other, and they are presented as separate gate categories rather than a unified quality score.

Treatment-v2, the first condition to achieve a 12/12 GS audit score, also has the highest vulnerability count: nine high CVEs, versus zero for the control. The source is a devdependency chain: `@typescript-eslint` pulling an old `minimatch` version. The control avoided this by selecting a different password library. Neither choice was architecturally motivated; both were made by the model without explicit guidance. The finding establishes that the GS rubric measures structural quality (layer discipline, interface enforcement, test construction, enforcement infrastructure) and does not assess supply-chain security. Both are necessary pre-release blockers; neither subsumes the other. A complete gate requires both an architectural audit *and* a vulnerability scan as independent checks. Full per-condition npm audit detail is in the companion supplement (§S9.3).

Post-publication condition, V3: Dependency Governance. A third post-hoc run was conducted after the static quality analysis, targeting the CVE finding directly. The condition added one prescriptive layer to the GS v2 template: explicit dependency governance instructions requiring `npm audit` to pass (zero high CVEs) as a P1 requirement, and naming preferred packages for password hashing (avoiding the `bcrypt` → `@mapbox/node-pre-gyp` → `tar` CVE chain). All other artifacts were unchanged from treatment-v2.

Property	Treatment-v2	Treatment-v3	Delta
Self-Describing (0–2)	2	2	0
Bounded (0–2)	2	2	0
Verifiable (0–2)	2	2	0
Defended (0–2)	2	2	0
Auditable (0–2)	2	1	–1
Composable (0–2)	2	2	0
Total (0–12)	12	11	–1
<code>npm audit</code> high CVEs	9	0	–9

Note: Scores above reflect the original direct-comparison audit at the time of the v3 run. The unified re-audit (see unified table, above) revised these scores under the full seven-property rubric and a stricter Auditable behavioural criterion: T-v2 Auditable revised to 1/2 (ADR referenced-but-not-emitted retroactively disqualified); T-v3 Auditable revised to 2/2 (dep governance directives created a richer, independently auditable trail). The comparison table is preserved as the historical record of the condition that motivated the ADR emission fix.

Principal finding: The dependency governance condition eliminated all high CVEs (9 → 0). One specification directive, prescriptive package selection and an explicit `npm audit` gate, closes the entire vulnerability surface while maintaining all other GS properties.

Auditable regression, root cause: The v3 score dropped one point from v2’s perfect 12/12. The model referenced `docs/adrs/ADR-0001-stack.md` in the README and emitted `CHANGELOG.md` as a structural stub with no entries. The auditor correctly penalized both: the ADR was referenced but never emitted as a file; the CHANGELOG satisfied the presence requirement but not the content requirement. The root cause is a precision gap in the “Emit, Don’t Reference” instruction: the template specified “emit ADR stubs” but did not state that each ADR must be a fenced file block with substantive content in P1, not merely cited in documentation prose and not as an empty placeholder. This is a template specification issue, not a GS architectural flaw.

Template fix applied; treatment-v5 complete, 14/14. The ADR emission precision gap was patched in `templates/universal/instructions.yaml`. Treatment-v5 achieved 14/14 by adding a dedicated `00-infrastructure.md` prompt (infrastructure must complete before feature prompts) and documenting the `jsonwebtoken` `StringValue` type pitfall in the specification. Session-verified: 109 tests across 10/11 suites against a live PostgreSQL database, converged in 2 fix passes. The root cause of prior Executable failures was a known type narrowing pattern the specification had not yet named — once named, it became a quality gate. An independent **Replication Experiment (RX)** commits `jest --json` output as a standard evidence artifact; any researcher with an Anthropic API key can reproduce the result at [experiments/rx/](#). Full supplementary data in `GS_Experiment_Supplement.md`.

Full replication data, session IDs, prompt texts, blind audit transcripts, per-condition metric tables, mutation testing progression, failed runs disclosure, and replication instructions, are in the companion supplement: `GS_Experiment_Supplement.md` (available at https://github.com/jghiringhelli/generative-specification/blob/main/docs/white-paper/GS_Experiment_Supplement.md).

7.8.C Experiment II: Benchmark Cross-validation (BX). Results

Purpose: Close Layer 1 of the define/build/measure loop. The circularity closure argument from BX is not that implementations were scored blind to condition — blind scoring cannot remove rubric self-referentiality. The closure is that rubric rankings prove congruent with external metrics the rubric never specified: CVE count, test count, and TypeScript health across implementations that received no GS guidance. If an author-designed rubric discriminates in the same direction as independent static analysis tools, the rubric is measuring something that exists outside the author’s methodology.

Implementations scored:

ID	Repository	Stack	Community Signal
A	lujakob/nestjs-realworld-example-app	NestJS + TypeORM + MySQL	~2k stars; cited NestJS reference
B	gothinkster/node-express-realworld-example-app	Express + Prisma + NX	Official RealWorld benchmark
C	GS-generated RX output	Express + Prisma (GS-specified)	104/104 tests; 0 CVEs

Repos A and B were never exposed to GS methodology. Scoring was conducted blind against the rubric before comparing with external tool results.

GS Rubric Scores:

Property	Repo A (NestJS)	Repo B (Official)	Repo C (GS)
Self-Describing	1	1	2
Bounded	2	1	2
Verifiable	0	1	2
Defended	0	1	1
Auditable	1	1	2
Composable	1	1	2
Executable	1	1	2
Total	6/14	7/14	13/14

External tool alignment:

Metric	Repo A	Repo B	Repo C
tsc errors	0 (after setup)	0	0
npm audit CVEs (total)	105 (16 critical)	43 (1 critical)	0
Test cases	1	27	104 passing

Ranking congruence: Rubric order (C > B > A) is identical to CVE rank and test rank. The rubric did not require GS guidance to produce this ordering.

Principal findings:

1. *Community reputation is an unreliable quality proxy.* Repo A (2k stars) scores below the official reference (Repo B). NestJS framework discipline yields 2/2 on Bounded, the framework enforces it, while the implementation carries 105 vulnerabilities (16 critical) and 1 test case. The rubric surfaces what star count ignores.
2. *The rubric discriminates on GS-specific contributions.* Both non-GS implementations score 0/2 on Defended (no CI, no pre-commit hooks, no enforced gates) and 1/2 on Auditable (partial conventional commits, no ADRs). These are the properties with no framework analog, the AI cannot emit them from NestJS conventions alone. The GS-generated implementation achieves 2/2 on both. The rubric identifies GS's specific contribution over what a high-quality framework already provides.
3. *The Defended gap is structural, not incidental.* No implementation scores 2/2 on Defended. A CI pipeline requires external infrastructure that generated code cannot provision. This is consistent across AX, BX, and RX and is reported as a permanent limitation, not a scoring anomaly.

Full scores and per-property rationale: `experiments/bx/scores.json`.

7.8.D Experiment III: The Executable Property in Production (EX)

The fourth experiment is the most complete single-project demonstration in this work: a full T1–T3 proof carried out in a live production environment rather than a sandbox. A RealWorld Conduit backend was specified, generated, verified, and deployed to Railway under ForgeCraft MCP orchestration (April 17, 2026), with the harness driving the cycle end to end — `generate_harness` and `run_harness` at development time, an environment-probe pass against the deployed instance, an SLO ramp against the production runtime, and `close_cycle` only after every gate returned green. The point of EX is not the build — AX (§7.8.B) already established conformant construction — but the **Executable property earned against a running system**: the specification's behavioral contracts checked against a real network endpoint backed by a real database, at three escalating levels.

Level 2 — behavioral contracts against the live runtime. Thirteen of thirteen Hurl use-case probes passed against the deployed service, carrying 1,013 individual assertions (status codes, response shapes, authorization boundaries, persisted-state deltas) derived from use-case acceptance criteria rather than asserted from compilation. Hurl is the executable form of the contract: each probe is a versioned HTTP transaction whose expected response is the acceptance criterion made runnable.

Level 3 — environment governance. Three of three environment probes passed, confirming the deployed configuration matched the specification — required environment variables present, secrets not leaked into logs or responses, the database reachable under the deployed credentials. This is the T2 governance tier: the specification governs not only the code but the environment the code runs in.

Level 4 — service-level objectives against the production runtime. Six of six SLO gates passed under a 10-VU load ramp: aggregate p95 350 ms and p99 720 ms, read p95 181 ms, write p95 401 ms, error rate 0.04%. The non-functional requirements were stated in the specification and checked against the live deployment, not estimated — the Executable property extends from “it responds correctly” to “it responds correctly under load within stated bounds.”

Fifteen defects were found and fixed inside the single session, each surfaced by a failing probe and closed before `close_cycle` would proceed — the harness refused to certify the cycle until the runtime matched the spec. Evidence is committed as machine-readable records (`harness-run.json` , `env-probe-run.json` , `slo-ramp-summary.json`) in `experiments/ex/` . EX is the production complement to RX (§7.8.G): RX proves the build is *reproducible* from the document; EX proves the build is *deployable and operable* against its stated contracts and SLOs.

7.8.E Experiment IV: Construction Invariance and the Knowledge-Retrieval Replication (KX)

This experiment closes the loop opened in §4.1.b: it measures, on a GS software harness, the retrieval economics Yarmoluk and McCreary measured for compact knowledge graphs, and it tests whether the cascade’s effect survives tool generation. Both arms run on one artifact — a ForgeCraft-generated RealWorld Conduit harness (treatment T8) — so the two results share a substrate. Every figure traces to a per-pass or per-query JSON record with full token usage (`experiments/ax/treatment-v8/` , `experiments/kx/`).

Construction invariance (AX Treatment-v8). The ninth condition of the adversarial study (§7.8.B) replaced the hand-written GS cascade with one generated by the ForgeCraft MCP tool at zero hand-tuning, holding prompts and the runner-verification protocol fixed. (The condition labels below map to the §7.8.B series: T0 = Naive, T1 = Control, T2 = Treatment/GS v1, T5 = the iterated hand-built arm; T8 is the new ninth condition.)

Condition	Harness	Blind audit	Expanded scale
To Naive	none	3/14	—
T1 Expert prompts	none	9/14	—
T2 Hand-built GS	hand-written	10/14	—
T5 Hand-built, iterated	hand-written	—	14/14
T8 ForgeCraft-generated	generated, zero tuning	12/12	14/14

The generated harness reached the same ceiling as the best hand-built arm: 211 tests at 99.08% statement coverage, zero layer violations (the ORM client confined to adapters), 12/12 Conventional Commits, and — the Executable property earned rather than assumed — 11/11 use-case probes derived from acceptance criteria and run against the live PostgreSQL runtime (cost $\approx$ \$52, $\approx$ 150 min, $\approx$ 640 agent turns across ten sessions). This is the software analog of the benchmark’s “construction invariance” finding: the structural advantage does not depend on expert hand-authorship. One field finding is reported against interest (F4): a community gate encoding the exact conformance rule sat installed but unsurfaced in session context, and the agent re-made the mistake the gate prevents — installed knowledge that is not read does not transfer, the same lost-in-the-middle mechanism the KX arm quantifies next.

Knowledge-retrieval replication (KX). A ForgeCraft navigation tree satisfies the three load-bearing CKG properties — finite enumerable context (the harness budget, $\leq$ 1,100 lines), deterministic traversal (the routing table), and closed vocabulary (a screaming-architecture layout in which structure states what lives where); it is a learning graph whose nodes are artifacts and whose edges are reading order. We generated 45 queries deterministically from the T8 project’s artifacts (entity $\times$ 8 as a negative

control, obligation $\times 10$, layer-path $\times 8$, aggregate $\times 11$, cross-link $\times 8$) and ran three retrieval conditions, each a fresh agent session per query, scored by token-level F1 and $RDS = F1/\text{tokens}$:

Condition	Macro F1	Tokens/query	Cost/query
monolith (all docs in context — RAG-dump analog)	0.611	100,237	\$0.56
navigation tree (routed — CKG analog)	0.808	78,603	\$0.10
no structure (code search — derive-at-query-time analog)	0.431	233,583	\$0.24

Four results carry the §4.1.b claims. (1) Routed retrieval beat both alternatives on accuracy *and* cost — $\approx 1.3\times$ cheaper than the in-context dump (100k $\rightarrow$ 78.6k) and up to $3.0\times$ cheaper than the unstructured condition (233k $\rightarrow$ 78.6k) — so token sanitation is measurable, not aspirational. (2) The monolith held every answer inside its $\sim 100\text{k}$ context and still lost on three of five query types, quantifying lost-in-the-middle (Liu et al.): context presence is not knowledge retrieval, which is the mechanism behind harness-bloat degradation. (3) Absence of structure was the most expensive condition — the structureless agent burned up to 492k tokens on a single query type searching for conventions that did not exist, to score ≈ 0 ; the harness’s absence multiplies cost. (4) The CKG architectural divergence replicated: aggregate-enumeration scored 0.909 (routed) versus 0.006 (no structure), mirroring the original 0.964 versus 0.054, while the negative control behaved (entity-lookup: no-structure 0.875, best of the three) — confirming the benchmark is not built to favor the harness universally: behavior questions belong to code search, structural questions to the structure.

Threats to validity. Agentic sessions carry fixed runtime overhead in all conditions, compressing the ratios relative to the original bare-pipeline figures ($\approx 11\times$ tokens/query there appears as $\approx 3\times$ in session totals here); marginal retrieval cost shows the true gap — obligation queries at 27.7k tokens (routed) versus 492k (no structure), $17.8\times$. Ground truth for the structural query types derives from the same structure the navigation tree reads (the original benchmark’s §8.5 caveat applies verbatim), so the claim is bounded: explicit structure beats inferred structure on structural queries, not general superiority. Results are single-model (Claude). One arm’s first run escaped its sandbox — the agent located the original project two directories up and read its constitution — and was invalidated and re-run in isolation; the incident is itself an observation: an agentic system will *locate* authored structure if it is reachable at all, so “no harness” is an unstable condition in practice.

Consequence. As of the post-T8 ForgeCraft release, `setup_project` emits `docs/learning-graph.csv` in the benchmark’s Definition 1 *column format* (ConceptID, ConceptLabel, Dependencies, TaxonomyID). The mapping is honest about where it generalizes the benchmark: the nodes are the harness’s **artifacts** (not atomic learnable concepts), and the edges are **reading order** — routing, derivation, doc-obligation, and code-to-spec traceability relations folded into one column, generalized from the benchmark’s single prerequisite relation; `TaxonomyID` here denotes **artifact class** (CNT/DOC/ADR/UC/SPEC/GATE/STD/CODE), a coarse grouping rather than a subject-domain taxonomy. The emitted graph is **validated acyclic at emission** — a write-time three-color-DFS cycle check throws on any back-edge before the CSV is written — and deterministic: the same project regenerates an identical graph. The T8 Conduit harness serializes to 83 nodes and 99 edges, comfortably within the benchmark’s reported corpus-size range. Every ForgeCraft project is therefore serializable into the benchmark’s input format as an **artifact-dependency graph** — a CKG-shaped structure rather than a concept learning-graph — directly consumable by the open ckg-benchmark harness.

7.8.F Experiment V: Self-Applicability at the Formal Tier (ALX)

ALX tests the strongest form of the derivation claim: not “AI builds a conformant application from a GS specification” but “AI derives a *compiler* from the formal specification of its own language.” The subject is Loom, whose specification (`loom.loom` plus `language-spec.md`) defines an 80-token lexer, an LL(2) recursive-descent parser, eleven semantic checkers run in fixed order (Hindley–Milner inference with occurs check, type resolution, effect tracking, algebraic-law conflicts, unit safety, tpestate transitions, privacy annotations, information-flow declassification, teleology, and a six-rule safety checker), and seven code generators (Rust, TypeScript, OpenAPI, JSON Schema, WASM, Mesa simulation, NeuroML). A blind derivation pass produced a complete Rust compiler from the specification alone; `cargo check` passed on first emission.

The contribution is the **iteration curve from spec gaps to $S_{\text{realized}} = 1.0$** , recorded in `s-realized.txt` and `correction-log.md`. The first run scored 0.000 — every test failed to compile because the spec named functions where the test suite imported public structs and module paths (`Lexer::tokenize` , `RustEmitter` , the eleven checker structs) the spec never declared. This is exactly the signature the directional relation $I \propto (1-S)/S$ predicts (§9.4): a single missing “Public API surface” section drove S to zero and made every downstream test unreachable, the maximum-cost gap. Each correction was a *specification* improvement, not a code patch — the gap was added to `loom.loom` and the compiler re-derived. The curve climbs $0.000 \rightarrow 0.339 \rightarrow 0.642 \rightarrow 0.781 \rightarrow 0.900 \rightarrow 1.000$ across six phases, terminating at 386/386 acceptance tests passing. The corrections themselves are the finding: they enumerate which classes of detail a formal specification must carry to be machine-derivable — public API surface, struct-vs-function naming conventions, recursive type representations (the spec’s `TypeExpr` of `String` could not express `Generic<List<T>>`), and parameter-name preservation for path inference — each traceable to a named spec section. ALX is the highest-tier controlled result in this work: spec derivability demonstrated at the machine-checkable layer *above* natural language, where conformance is `cargo test` , not a rubric. Evidence: `experiments/alx/` in the Loom repository.

7.8.G Experiment VI: Independent Reproducibility (RX)

RX makes the Executable property *independently verifiable*: any reader with Docker, Node, and an Anthropic API key can regenerate the result. From a single committed GS document (`experiments/rx/spec/conduit-gs.md`), the runner drives `generate` → `build` → `test` against an ephemeral PostgreSQL 16 instance and emits committed evidence — `jest-output.json` , `build-log.txt` , `score.json` , `run-metadata.json` . The recorded run (March 15, 2026, Claude Opus) produced 104 passing tests across seven suites with zero failures, `tsc --noEmit` exiting clean, and no hardcoded credentials in the generated output. One fix is disclosed in the metadata — `maxWorkers: 1` in the Jest config, because parallel database tests raced on a unique constraint — a configuration detail, not a derivation failure. Because the evidence is committed, a reader gets the pre-run artifacts on clone and can either trust them or re-run to produce their own; the claim does not rest on the author’s word. RX is the reproducibility floor beneath the case studies: where AX (§7.8.B) establishes the result under controlled adversarial conditions and EX (§7.8.D) proves it in production, RX makes it a button any third party can press.

7.8.H Experiment VII: Model Cost and the Tiering Boundary (MX)

MX measures the cost claim directly: once a task is GS-specified, does a cheaper model do it as well, and does multi-model *tiering* (a strong planner decomposing for cheaper executors, escalating failures) beat a single model? Executed June 2026 via tiered subagents (Opus 4.8 / Sonnet 4.6 / Haiku) on

RealWorld/Conduit tasks of increasing size — auth slice (35 oracle assertions), auth+articles+comments (95), and the full Conduit backend (149 across 13 suites) — each verified by a held-out Hurl acceptance oracle run by the harness, not self-reported by the agent. Result: on the complete backend, **both Opus and Sonnet scored 149/149 (100%)**; Sonnet used fewer tokens (43.4k vs 54.4k) at roughly one-fifth the per-token price — **≈6× cheaper for identical verified quality**. Tiering did not pay off at this task class: the strong-model planning step alone (37.4k tokens) exceeded the cheaper model’s entire one-shot build (34.8k). The measured conclusion: for well-specified service/CRUD work the cost-efficient choice is the capable mid-tier model, not the expensive one, and tiering’s coordination overhead is justified only for tasks beyond a mid-tier model’s one-shot capacity. This moves the cost claim (§4.1.b, §8.2.1) from reasoned to measured *for this task class*. Conditions, generated outputs, the oracle, and reproduction steps are committed at `experiments/mx/`. Honest caveat: in-session execution records total tokens only; the precise input/output cost split awaits a metered (e.g. promptfoo) re-run.

7.8.1 Experiment VIII: Specification, Verification, and Bounded Context under Pressure (RND-1)

RND-1 tests, under explicit delivery pressure (speed and token economy), which GS arm suppresses the failure modes catalogued in the reward-hacking literature (ImpossibleBench; EvilGenie; the Reward Hacking Benchmark): *under-specification exploitation* — satisfying the letter of a vague spec while missing its intent — and *test-faking* — vacuous or improperly-mocked tests that pass without verifying behaviour. Three single-shot sub-experiments, each verified by a held-out Hurl oracle run by the harness and classified by **stateless external judges** (fresh agents with no access to the worker’s context). (1) **Prescriptive specification — confirmed (n=3)**: given a *descriptive/ambiguous* spec the model floored to the literal minimum — stored raw rows, no aggregation, no derived metrics, scoring 0/3 against the held-out intent — *at equal token cost*; given a *prescriptive* spec (use cases with postconditions) it recovered the full intent, 3/3. The base claim — that a missing behaviour traces to missing specificity, not model limitation — held, and was *complete* at this scale (specificity alone closed the gap; the floor was not a token saving but the model resolving ambiguity toward the minimum rather than the charitable reading). (2) **Bounded context (the sentinel/CNT) — bounding null (n=2)**: a twelve-requirement spec remained within the mid-tier model’s one-shot capacity (flat and bounded-navigable forms both 100%), so no context-overload effect appeared — the bounded-context arm’s value lies beyond one-shot capacity (large codebases), not medium tasks. (3) **Independent verification — honest null (n=2)**: under “graded on your own test suite” pressure the model did *not* reward-hack — its self-written tests agreed with the held-out oracle (gap zero) and the stateless judge confirmed the tests were genuine (consistent with the literature’s near-zero exploit rate for this model class). Synthesis: at current model capability the demonstrably load-bearing GS arm is **prescriptive specification**; independent verification and bounded context are real but their value surfaces against weaker or adversarial agents and at large scale respectively — honest nulls that *bound* rather than refute those arms. Full design, specs, oracles, generated outputs, and reproduction steps are committed at `experiments/rnd-1/`.

7.9 Meta-Application: Autonomous Specification Evolution

ForgeCraft 1.0’s gate system, enforcement hooks, and template hierarchy emerged from the AX experiment series — it was the output of the series, not a prior condition of the case studies. Each AX treatment cycle applied GS to ForgeCraft itself: the tool was simultaneously the specifier and the subject.

The AI identified specification gaps, authored gate definitions, implemented gate logic with tests. The human contributed the rubric and the release gate. The AX series is directionally convergent with a non-monotone path (v3: 14/14 → v4 regression → v5 recovery). The same seven properties governing ForgeCraft's construction served as the rubric against which ForgeCraft's outputs were evaluated — the pragmatic tier is self-applicable. The AX self-application cycle converges to $S_{\text{realized}} = 1.0$ across the automatable rubric.

8. Implications for Practice

GS provides a structured specification methodology with empirically validated quality improvements. The Kuhnian structural criteria are met: the anomaly is architectural drift at generation speed, which documentation-based conventions cannot structurally prevent; the reconstitution is the specification becoming the primary artifact, with code as derived output. When the community crosses the adoption threshold is a question the EX replication data will inform. The implications in this section follow from the structural claim, which is answerable by inspection and confirmed by the AX series.

8.1 The Specification Precedes the Code

The shift required by generative specification is temporal: design precedes implementation, not the other way around. The architectural constitution, the C4 diagrams, the schema definitions, and at least a skeleton of the ADR structure must exist before the first AI-assisted implementation begins. This is not a new idea. It is an idea that was optional when the cost of skipping it was paid personally by a human engineer who could compensate with memory and informal communication. That compensation is not available to an AI session.

8.2 The Synthesis: Specification-First and Iterative Delivery

Generative Specification is not a third methodology alongside waterfall and agile. The specification layer runs waterfall: the architectural constitution, ADRs, structural diagrams, and behavioral contracts are complete before any agent session begins. The grammar must be written first. The delivery layer runs agile: each session produces atomic, tested, deployable commits; features are new production rules; bugs are delta reports between actual and specified state.

The failure modes of each model cancel. Waterfall's rigid front-loading is resolved because the architectural constitution is a living document revised through commit discipline. Agile's structural drift is resolved because the specification gates every session. The specification provides the coherence agile lacked; iterative delivery provides the adaptability waterfall could not sustain. They operate at different altitudes in the same system.

Scope may be bounded. A complete specification for the minimum viable system is still a complete specification — the discipline does not require the full system to be specified before the first session begins; it requires the current session's scope to be completely specified before generation starts. Each scope expansion begins with a spec expansion captured in a new ADR, before any new session opens. The practitioner who finds the full-system spec daunting is not being asked to do less rigorous work — they are being asked to do the same rigorous work on a smaller domain first. The investment front-loads correctly: the first scope takes the most specification time because the domain is being understood while it is being written. Each subsequent scope is faster because the architectural decisions, naming

conventions, and constraint vocabulary are already in the artifact set and the AI reads them before every session.

8.2.1 The Economic Inversion

In traditional software development, implementation accumulates a sunk cost. When a specification conflicts with an already-built system, the economically rational response has been to adjust the specification: the code is load-bearing and the specification is not. Requirements drift toward the artifact because the artifact carries the cost.

Generative Specification inverts this — **cost inversion** (§4.1.b): implementation is cheap and repeatable; fix the specification and regenerate. The code carries no sunk cost because it was never the expensive artifact. The specification is not reliably recoverable from code alone: decisions, alternatives considered, domain knowledge, and accumulated rationale resist reconstruction. Code is an implementation residue. The scarce resource is no longer the ability to write code. It is the ability to specify correctly.

8.2.2 The Industrial Threshold

The mechanical loom crossed a threshold: consistent quality became a floor, not an achievement. Software construction is approaching the same threshold. The comparison baseline is not the ideal case — it is the modal case: teams collaborating under architectural documents never completed, governed by requirements that drift, accumulating debt. Three structural constraints historically prevented maintaining full formal discipline: **learning** (no career is long enough); **maintenance** (disciplines erode under deadline pressure); **transfer** (knowledge lived in people). The executor makes all three irrelevant — it holds every discipline without fatigue, erosion, or transfer cost. Artisanal software survives at extreme constraint envelopes (radiation firmware, military flight controllers) that will constitute a smaller fraction of all construction. The specification discipline is the loom. The practitioner's role changes from implementation artisan to specification architect.

8.3 Commit Discipline as Corpus Quality

The git history of a generative specification system is a typed, scoped corpus. Each conventional commit is a sentence: a part of speech (feat, fix, refactor), a scope boundary (billing, auth, user), and a semantic payload (what changed and why). A history built of `fix bug`, `wip`, and `changes` is not a corpus, it is noise. A well-maintained history provides a queryable record of how the grammar evolved, available as context in every session, without requiring anyone who was present to explain it. The Shattered Stars case study (§7.6) demonstrates precisely what is lost when this record is absent: the specification held behavioral contracts across sessions; what it could not hold was the provenance of decisions. The Auditable property requires both: that the record exists, and that the next session begins by reading it.

Every fix commit must include the failing test that reproduces the defect. No exceptions. A fix without a reproducing test is a temporary suppression: the corpus has no record of what was wrong, and a future session can regenerate the same defect. In a GS system where regeneration is fast, writing a test is not expensive. Silent reintroduction is.

8.4 ADRs as Persistent Memory

Every non-obvious architectural decision produces an ADR before implementation begins. Format is minimal: the decision, the context that produced it, the alternatives considered, and the consequences.

This is not documentation for documentation's sake. It is the record that allows the AI to recognize intentional decisions and distinguish them from technical debt. Without it, the AI will “improve” them.

8.5 Names Are Production Rules

In a context-sensitive system, naming is not style. It is grammar. A function named `getUser` in a domain model that talks to a database is a violation of the architecture that the compiler will not catch, the linter may not catch, and a human reviewer will tolerate, but the AI will propagate. A function named `findUserByEmail` in a repository layer and `getUserProfile` in a service layer communicates ownership, scope, and responsibility through its name alone. That signal is available to the AI on every read.

The naming principle extends beyond architecture into technique transport. What a practitioner names in a specification, the AI knows how to apply. RAPTOR indexing (hierarchical codebase summarization at file, module, subsystem, and repository level) was first specified for CodeSeeker and propagated to BRAD, SafetyCorePro, and Conclave without any shared session context. The transport was the name. Every technique in the model's training corpus becomes available to any system whose specification names it. The specification is therefore not just an architectural grammar, it is a technique registry whose scope is the full depth of the model's training, activated at the cost of knowing the correct words to write.

This candidate mechanism — **domain dimensional expansion** (§7.5), observed consistently across case studies but not yet subjected to systematic controlled testing — means the specification is a technique registry whose scope is the full depth of the model's training, activated at the cost of knowing the correct words to write. Systematic characterization of which domain terms activate which capabilities, and at what reliability, is noted as a specific line of future work.

8.6 The CLI as Execution Surface

The productivity results in §7 depend on a second condition beyond specification quality: the AI has direct CLI access. An AI that can only read and write files is an advisor — it proposes plans. An AI with CLI access is an executor. It runs the migration, resolves the dependency conflict, reads the error, selects an alternative, and retries — without returning to the engineer between attempts.

The specification does not just govern code. It governs a system that can act. The architectural constitution, commit policy, deployment targets, and build constraints become operational rules for an agent that can execute them. The scope of what must be specified is therefore broader than code architecture alone.

Tool server budget and architectural constitution compression are treated in the Practitioner Protocol (§16).

8.6.1 The API-First Future

As the infrastructure world completes its API-first transition, GS-governed specifications will describe not only the code but the environment itself — DNS records, OAuth applications, SSL certificates, IAM policies, and every resource a capable executor can provision from a specification. The AWS ETL pipeline in §8.6 is a current instance; the pattern extends to every service that exposes its configuration as an API. The specification is not merely the mold for the code. Progressively, it is the mold for the entire operating environment the code runs in.

8.7 The Session Loop

The macro properties in §§8.1–8.6 govern the stable structure of a GS system. A complementary question governs the micro level: what must happen inside a single session to preserve that structure when the session ends?

The answer is an invariant: every session must begin and end at the same *steady state* — code, tests, documentation, and specification mutually consistent; Status.md capturing intent for the session that follows. The session loop has four phases:

- 1. Intake and clarification.** Before implementation begins, the agent checks for ambiguity (the request admits two interpretations that would produce different implementations) or unverifiable assumptions. If either is present, one exchange resolves it — all clarifying questions batched into a single prompt, answered once. The constraint on asking is as important as the obligation to ask.
- 2. Specification gate.** Before any code is written: does this change *fit* the existing specification, or *change* it? A feature the specification does not yet cover requires the specification to be updated first — ADR, schema change, new constitution section — in that order, without exception. Code written against the old specification is correct by local standards and wrong by the grammar it was supposed to serve.
- 3. Implementation and verification.** Tests are written alongside the code. Before any commit: full test suite passes, feature exercised at the HTTP or CLI boundary, no new anti-patterns introduced.
- 4. Documentation cascade.** Specification artifacts restored to consistency: public-contract spec files, ADR if a non-obvious decision was made, diagrams if a new component was introduced, Status.md always.

Full steady state requires all four artifacts: specification, tests, commit history, and Status.md. Any one missing adds cost to every session that follows. The spec update and co-written test are near-free at session close. Deferred, they are paid at full price.

8.8 The Paradigm Beyond Code

The seven specification properties are stated for application code because that is the domain where the principle was first visible and most formally developed. The same failure mode applies wherever AI output can be evaluated: a specification that does not state the restriction produces output that is locally valid and globally wrong. The restriction type changes by domain. The mechanism does not.

Infrastructure and generative assets: The restriction is *acceptance criteria*. The Stable Diffusion pipeline (§7.6.2) restricts each generated image against four quantitative checks. An image that fails is rejected and regenerated. Structurally identical to a type check or test assertion. Infrastructure provisioning specified as desired state (IAM policies, VPC boundaries, encryption requirements) applies the same constraint.

Business layer: The restriction has two axes. *Economic viability*: conversion rate, sustainable cadence, retention threshold, cost-per-acquisition ceiling — these are the business layer’s quality gates. A content strategy that saturates a distribution channel and burns the audience is architecturally incoherent in the business sense: every individual piece passed a local check; the system moved in the wrong direction. *Legal and ethical compliance*: jurisdictional requirements, contractual obligations, and ethical commitments are the constraints that define what counts as a valid sentence in the domain of business decisions. The distinction between a policy document and a generative specification is the same as at the code layer: the constraint must be blocking and automatic, not advisory.

The paradigm’s contribution across all domains is the same insistence: externalize constraints before instructing the agent. In code, the vocabulary is the architectural constitution. In generative media, it is acceptance criteria. In business, it is the economic logic, legal boundaries, and ethical commitments. The invitation to other disciplines to confirm or qualify the mechanism is open. The Practitioner Protocol (Part X) develops the beyond-code application further, including business-layer gate design and infrastructure specification patterns.

8.9 The Prompt Engineering Objection

The full distinction between GS and prompt engineering is in §4.5. In brief: a prompt is a session artifact, it exists for one interaction and disappears when the context window closes. The specification is the grammar the model reads *before* any session prompt, and it persists across every session. Architectural drift accumulated over thirty sessions is not the product of thirty bad prompts — it is the product of a context that degrades faster than any individual prompt can repair. The Shattered Stars case (§7.6) is the direct demonstration: the same session prompts produce structurally coherent output against a 2,277-line specification, and sixteen divergent systems without one. The coherence is produced by the grammar, not the prompt.

8.10 The Failure Mode: A Wrong Specification

The most important risk is not an underspecified system — it is a *wrongly* specified one. A faithful AI executing a flawed architectural constitution produces flawed code at scale, with high confidence and no complaint. A well-formed grammar does not guarantee the right grammar.

Four practices mitigate this. **(1) Specification verification:** before any code is written, concrete behavioral outcomes must be defined and made checkable. ADRs serve this function: a decision whose rationale does not survive being written down was not sound. **(2) Living document discipline:** the architectural constitution is revised through the same atomic commit discipline as the code it governs. A static grammar for a living system accumulates debt. **(3) Domain depth is the ceiling:** GS raises the floor — a practitioner following the methodology will produce a specification better than no specification. But the ceiling is the practitioner’s engagement with the domain, the precision of their naming, the judgment to recognize which decisions are architecturally load-bearing. GS cannot make an incorrect specification correct.

(4) Meta-completeness querying addresses the gaps the first three cannot — the things the practitioner does not know they are missing. Having specified as completely as current domain depth permits, the practitioner asks the model what dimensions of correctness the specification does not yet address. The model activates domain-specific correctness requirements from first principles. In the BRAD case (§7.5), querying surfaced two structural gaps: the infraction taxonomy needed a cross-dimensional mapping to twelve Minnesota statutory grounds, and citation accuracy required cross-referencing against the MN API public case database. Neither was visible from inside the specification. Both were correctness requirements derivable from the domain structure. The loop is: query, evaluate, specify, commit.

§8.11 extends this to the full hardening surface.

8.11 Hardening as Specification

The adversarial posture of the Verifiable property, tests designed to fail on incorrect code, extends naturally to the full hardening surface. Stress testing, security testing, chaos engineering, cross-cutting

concern validation, and environment auditing each follow the identical structure: specify the adversarial or compliance condition, define the acceptance threshold, execute, report divergence. In every case the test is designed to break the system, reveal a gap, or expose an assumption. Not to confirm it functions. A system that passes has been proven against its own stated limits. A system that has never been challenged has only been proven against itself.

Category	Constraint Vocabulary	Representative Tooling
Stress & performance	Peak concurrent users, sustained request rate, latency ceiling (p99), error rate threshold; soak, spike, and scalability ceiling variants	k6, Artillery, Locust
Security	Threat model: authentication bypass attempts, injection payloads, dependency vulnerability scan, CORS policy, secret exposure, privilege escalation; severity acceptability threshold (Invellum: zero critical findings)	npm audit, Snyk, OWASP WSTG, ZAP
Chaos engineering	Resilience contracts: recovery time after node kill, dead-letter injection, DB failover window, circuit breaker open/close thresholds; property-based testing extended to infrastructure	Chaos Monkey, Gremlin, custom fault injectors
Cross-cutting concerns	Encryption policy (TLS version, cipher suite, at-rest, secret rotation); authorization model (RBAC/ABAC per surface, agent generates tests from insufficient-permission contexts); observability schema (correlation ID, PII redaction, SLO/SLI thresholds); data lineage contract (provenance specification: where data originates, how it transforms, and where it terminates); dependency compliance (CVE threshold, license policy)	TLS auditors, log schema validators, npm/pip audit
Environment hardening	TLS headers, Content Security Policy, no exposed secrets, IAM least-privilege boundaries, CORS policy correctness, agent audits running environment against spec and closes the delta	Cloud provider policy tools, Trivy, tfsec

The common failure pattern across all hardening categories mirrors application architecture: concerns fail not because engineers are unaware of them, but because they were never stated as blocking acceptance criteria. The specification does not add new requirements. It makes existing ones structurally present, explicit, enforced, and verifiable.

8.12 The Application Gate

The quality gates in §8.11 test the implementation. There is a complementary gate where the thing being tested is the *specification artifact itself* — the gate, the template, the methodology change. The mechanism is identical: state the acceptance criterion, apply, report divergence.

The application gate verifies a specification artifact by applying it to real examples and comparing output against a known-good reference. Three benchmark sources: *existing governed projects* (known-good states; a gate that fires on a correct project is miscalibrated); *external benchmarks* (the Conduit/RealWorld spec was the AX and RX benchmark); and *AI-generated benchmarks* (the AI generates a project exhibiting the failure mode, confirms the gate fires, then generates a compliant version and confirms it does not — the Verifiable property applied one layer up).

A methodology change that would previously require weeks of manual verification across projects now requires a single generation pass. The application gate runs at the speed of a test suite. It is also a measurement instrument for \$\$\$: if a template change reduces divergence across N benchmark applications, \$\$\$ increased.

8.13 The Engineer Elevated

GS is a convergence mechanism — given a complete, correct specification, the system drives output toward correctness. Writing that specification for a complex system is not mechanical: it requires decomposing a problem domain the AI did not define, naming the dimensions along which the solution must be evaluated, and distinguishing the constraints that matter from those that appear to. That is where the intellectual effort lives.

Tool-syntax expertise and common-pattern knowledge depreciate universally; deep domain expertise and cross-domain synthesis appreciate. The engineer is elevated to the layer that was always the harder problem: stating what must be true before any code exists to confirm it. The practitioner who specifies with precision is more valuable, not less, in a world where implementation is abundant and correct specification is scarce. Every degree of freedom the specification leaves implicit is a degree of freedom the AI exercises without constraint.

GS expertise accumulates in a specific direction. The six production projects (§7.1, §7.2, §7.4–§7.7) document a consistent trajectory: early projects required sustained AI dialogue — conventions established, ambiguities resolved, failure modes encountered and named. Later projects required only course corrections and directed expansions. The practitioner learns to specify at the level the executor can derive; the executor learns nothing. The fluency is asymmetric and accumulates entirely with the practitioner. This trajectory is evidence against the concern that AI-assisted development creates structural dependency: it creates structural fluency.

Four structural changes follow from the evidence:

- **The design process:** specification precedes code, not accompanies it.
- **Team composition:** specification is a first-class engineering skill — the ability to decompose a problem, name its parts, and express architectural intent with no important gap unfilled.
- **Quality measurement:** structural coherence joins test coverage as a first-class metric.
- **Domain breadth:** the AI produces output at the level of specificity the specification signals. The ceiling the paradigm rewards is precision in specifying domains, not implementation speed.

The broader implications of this elevation — the restoration of economic value to cross-domain synthesis ability, the inversion of the industrial-era executor-training logic, the civilizational significance of a practitioner who can hold multiple domains simultaneously — are developed in the companion essay “Onwards: The Formal Tradition Was Waiting for Its Executor” (Ghiringhelli, 2026).²

8.14 The Adoption Ladder: Where GS Enters the Workflow

Five levels describe where most practitioners currently sit (Shapiro 2026; Swarmia 2025):

Level	Mode	Specification state
1 — Autocomplete	Token completion while typing	None
2 — Conversational	Chat-based discrete tasks	None; context re-established per session
3 — Context-aware	Files/docs supplied alongside prompts	Partial, inconsistent
4 — Agentic direction	AI executes multi-step tasks; Monitor → Assess → Nudge	Implicit; filled arbitrarily at session boundaries
5 — Specification-governed	Complete architectural constitution present before every session	Persistent grammar across sessions, practitioners, and model versions

Level 6 is already operational: **specification-governed multi-agent composition**, where a scaffolding agent (ForgeCraft), a codebase intelligence agent (CodeSeeker), and an implementing agent (Claude Code) each consume the same architectural grammar without re-deriving it independently. Conclave (§7.4) instantiates this fully: **deterministic orchestration governs non-deterministic agents**. The DAG is derived from the spec; the agents' outputs are generative. The DAG constrains the generative surface exactly as the specification does at the session level. In autonomous execution mode, the human's role reduces to three acts: write the spec, start the session, review the output.

Teams whose Level 4 sessions produce more output but whose aggregate system shows more drift have located the gap precisely: the executor is operating without a sufficient grammar. The fix is not a better model. It is a complete specification.

8.15 Change Governance by Construction

Enterprise organizations operating under formal change management requirements — ITIL, ITSM, regulated-industry audit obligations — maintain change records as a separate artifact class from the code they describe. A Request for Change lives in ServiceNow; the implementation lives in git; a Change Advisory Board meeting lives in calendar archives. These three artifacts describe the same event from three different locations and begin drifting apart within the sprint that follows. The audit trail is technically present and practically unusable: the record does not reference the commit, the commit does not reference the decision, and the decision rationale has no machine-readable location at all.

GS's artifact grammar satisfies change management requirements by construction, because the change record IS the specification. Every structural change to a GS-governed system follows the same path: the specification is updated first, the ADR records what changed and why, the gate stack enforces the change before merge, and the conventional commit logs it with type, scope, and payload. The RFC is the spec update. The change record is the ADR. The CAB is the quality gate. The audit trail is the git history. None of these are separate artifacts maintained in parallel — they are the same artifact set that governs generation, now also satisfying the change governance requirement as a side effect of correct practice.

The implication for regulated domains is structural rather than incidental. HIPAA's requirement for documented change management in systems handling protected health information, SOC 2's change management control, and CMS reporting requirements for data platform modifications are all satisfied by a GS artifact set maintained under normal discipline. A compliance auditor reading a GS-governed project's ADR directory, commit history, and gate configuration has everything a formal change management record requires — including what was considered and rejected, a dimension that traditional change tickets rarely capture. The COMPASS regulated data platform (§7.7) demonstrates this at production scale: the process hash is a formal contract propagating from the specification through every pipeline stage, making every infrastructure state change traceable to the specification that authorized it. The lineage graph is simultaneously the operational specification and the audit-ready change record. These are not two documents. They are the same document read from two directions.

8.16 Mechanism-Sample Conflation

A specification that describes a generative system will often include a concrete example of what the system produces. This is good specification practice: the example makes the mechanism legible, demonstrates scope, and grounds abstract description in something the reader can evaluate. The failure mode arises from what happens to that example when the AI reads the specification.

An AI executor operating on a specification that contains both a mechanism definition and a sample artifact will, in the absence of explicit structural separation, treat both as implementation targets. The result is that the sample artifact is built directly, as the goal, and the mechanism — the actual subject of the specification — is either omitted or reduced to scaffolding around the artifact. The executable output looks like progress: something was built, it is the kind of thing the specification described, and it even resembles the example. What was not built is the system that would generate that artifact, and any other artifact of the same kind, without the same effort being repeated.

The pattern recurs across domains. A specification for a writing tool that includes a sample chapter as illustration of voice and length yields an AI-generated chapter, not a writing tool. A specification for a music composition system that names a target piece as a demonstration of the style the system should produce yields an arrangement of that piece, not a system that composes in that style. A specification for a game asset pipeline that describes the visual identity of a specific faction to illustrate what the pipeline will generate yields artwork for that faction, not a pipeline. In each case the AI has resolved a structural ambiguity — what am I building versus what should come out of what I am building — by choosing the visible, graspable, immediately generatable artifact over the mechanism whose existence the artifact was meant to justify.

The structural fix is not subtle. A specification for a generative system must separate the two subjects with explicit named sections, at a level of hierarchy that makes the boundary unambiguous to a stateless reader encountering the document for the first time. The mechanism section describes what the system is and how it operates. The seed output section names the first concrete artifact the system will produce once it exists, and frames it explicitly as a validation target, not a deliverable. The sample is evidence that the mechanism is working, not the thing being built.

This has a direct consequence for how the seed output is written. It should be specific enough to validate the mechanism — specific names, concrete parameters, checkable outputs — but its specificity is in service of the mechanism test, not independent artifact production. A seed output that could be built without the mechanism is a warning sign: the artifact is separable from the engine, which means the AI can satisfy the specification without building the engine at all.

The §4.1.e art generation pipeline is the canonical positive illustration: the strategy game concept is named with enough specificity (factions, color palettes, lore) to validate the pipeline, but the pipeline — the infrastructure tier, the constraint vocabulary, the derivation chain — is the subject of the specification. The game concept is the first output, not the output. A specification that inverts that priority, naming the game first and the pipeline as a means to it, will produce the game. Whether the pipeline exists afterward is left to inference.

The failure mode is one of structural signal. The AI is not making an error of reasoning; it is making an inference from an ambiguous structure. The fix is to remove the ambiguity from the structure: build the engine, ship the sample as its first real outcome. This ambiguity is detectable at specification time — a seed deliverable that can be produced without the mechanism is the diagnostic signal — and is one of the structural checks the accompanying tooling enforces at specification review.

9. Convergence, Stability, and Forward Extension

9.1 Agentic Self-Refinement

A pattern visible across case studies: **agentic self-refinement**. Wherever desired output can be specified and actual output observed, the agent closes a feedback loop on its own execution without human intervention between cycles. The generate → evaluate against acceptance criteria → regenerate structure applies at every output level: image generation, hyperparameter tuning, session resumption, strategy engine backtesting. The scope is bounded only by the engineer’s ability to define acceptance criteria. Any domain where desired state can be stated and actual output observed yields to this loop. The surfaces are not a finite list. They are a consequence of a principle.

The restriction — removing implicit context — is the floor from which expansion reaches any such domain. Across the case studies, the same structure governs every surface the AI touched:

Domain	Constraint Mechanism	Evidence
Application & data architecture	Layered services, repository interfaces, named domain models, cross-language interface contracts, retrieval architecture composition (embeddings + BM25 + RAPTOR + knowledge graph, fused via RRF)	SafetyCorePro, Invellum, ForgeCraft, Conclave, CodeSeeker, BRAD
Infrastructure & environment	Cloud resource desired-state provisioning; toolchain configuration described as desired state and resolved iteratively without engineer-issued platform-specific commands	Invellum (Railway), Shattered Stars (Vercel, SD environment), AWS ETL
Generative asset pipelines	Executable acceptance criteria on AI-generated outputs: symmetry threshold, background validation, orientation angle (PCA), audio LUFS normalization; multimodal model evaluation of existing assets against style specification	Shattered Stars (§7.6.2, §7.6.3)
Agentic self-refinement	Generate → evaluate against spec-defined acceptance criteria → adjust parameters or session context → regenerate; loop operates identically at image generation, hyperparameter optimization, and session resumption	Shattered Stars, quantitative finance classifier, BRAD session logs

9.2 The Interface Layer: From Screen to Ambient Orchestration

No empirical claims; an honest account of where the paradigm’s structural argument leads when tested against lived experience.

At fifteen active projects cycling through structured waiting states, execution is not the bottleneck — a waiting project costs nothing. Status management is: knowing which projects have cycled to ready and what each needs next. A screen solves this when seated in front of it. What does not yet exist is the specification layer for an ambient alternative: a grammar that decides which signals rise to attention, at what summary depth, through which modality. That is a GS problem applied to the engineer’s own attention. Two rules must be specified before the hardware matters: a *maintenance window rule* that batches non-urgent signals into a scheduled review; and an *emergency filter rule* that defines, by named criteria, what bypasses the queue. Everything not named is a maintenance item. The hardware is available. The specification discipline is the same discipline this work argues for everywhere else.

9.3 Template Gap: ADR Emission Precision — Diagnosed, Patched, Validated

Three template changes diagnosed through the AX series and shipped to `templates/universal/instructions.yaml`:

1. *Minimum ADR set.* Emit at least three ADRs in P1 (stack selection, authentication strategy, architecture decisions) with substantive content in all fields — no TBD placeholders.
2. *Reference-check invariant.* If a file is named in documentation prose within P1, it must appear as a fenced code block in the same response. Referenced-but-absent files fail the Auditable criterion.
3. *CHANGELOG initialization.* The initial `CHANGELOG.md` must document actual P1 decisions, not emit an empty `## [Unreleased]` block.

Epistemic finding. Treatment-v3 achieved 14/14 through auditor-inferred Executable (static artifacts). Treatment-v5 achieved 14/14 with session-verified Executable (109 tests against a live database, 2-pass convergence). Same score, completely different epistemic basis. A specification that produces inferably-executable output is necessary; verifiably-executable output is sufficient. Raising `$$` before generation — infrastructure-first prompt, known type pitfalls — reduced the verify loop from 5 passes to 2. All fixes propagate to every GS-governed project on `forgecraft refresh_project`.

9.4 The Convergence Spiral: Expected Iterations as a Function of Specification Completeness

$I \propto (1-S)/S$ is a mental model — a visualization of the direction of the relationship, not a formal mathematical result and not a claim under empirical test. No formal proof is offered or intended. The formula asserts no specific units for `$$` or `$$I`, no proportionality constant, and no prediction about magnitude. What it communicates is direction: each freedom the specification leaves unclosed is an additional correction cycle. That direction is the claim. The formula is the visualization. The AX series provides directional support across its eight conditions (a ninth, §7.8.E, tests construction invariance); a cross-practitioner correlation test is noted as future work.

Two distinct S concepts. *Theoretical S*: the fraction of the output space closed by the specification. `S_{\text{realized}}`: ForgeCraft’s per-project proxy (accepted verification steps / total applicable steps). `S_realized` is correlated with theoretical S but does not validate the formula. Empirically testing the proportionality claim would require recording iteration counts alongside session-start S scores across practitioners; this is noted as future work.

Each AX condition raised S in a distinct dimension:

Condition	Dimension of <code>\$\$</code> raised	Score
Control	Baseline expert prompting	Reference
Treatment	ADRs, <code>CLAUDE.md</code> , pre-defined schema	+1 GS (Composable)
Treatment-v2	Explicit emit directives; First Response Requirements	13/14
Treatment-v3	Dependency governance (package registry + audit gate)	14/14 (auditor-inferred)
Treatment-v4	Verify loop (max 5 passes) — context gap caused regression	11/14
Treatment-v5	Infrastructure-first prompt + Known Type Pitfalls	14/14 session-verified

Treatment-v4 regression root cause. The verify loop introduced in v4 (`materialize` → `tsc` → `jest` → `correct`, max 5 passes) extended the `CLAUDE.md` significantly with loop mechanics, pass counters, and

correction directives. The cumulative token weight of the full context — all prior ADRs, architecture files, First Response Requirements, dependency governance directives, and now the verify loop — crossed a threshold where the model began dropping earlier directives in mid-session. Specifically, the Auditable (ADR emission) and Executable (verify loop completion) properties both regressed: the model referenced ADRs without emitting them and terminated the verify loop before convergence. The root cause is not a failure of the verify loop concept — it is a context window management failure. The fix, applied in v5, was architectural: a dedicated `00-infrastructure.md` prompt run before feature prompts, so infrastructure (ADRs, schema, environment config) is materialized in a low-context session before the feature generation context expands. This staged approach prevents any single session from carrying the full specification weight at once.

Specification determinism. The convergence behavior of $I(S)$ depends on how precisely the desired output can be stated before generation begins. At high determinism (HL7 FHIR, ACORD XML, FIX protocol), contracts *are* the specification — the verify loop has something rigorous to check against, $I \approx O$ approaches. At low determinism (“fuse these aesthetics”, “capture the 80/20 feature set”), the desired state is expressible but not automatically compilable into a test suite. GS forces the human to encode judgment *before* generation through ADRs and use-case documents rather than after through correction. The lower the determinism, the higher the return on upfront GS artifacts.

Uncertainty taxonomy classifies each verification step’s *completeness ceiling* — the maximum fraction of the acceptance surface coverable by automated verification:

Level	Domain examples	Ceiling band
Deterministic	Type contracts, schema validation, API conformance	High
Behavioral	UI flows, integration end-to-end	High–Medium
Stochastic	Game balance, financial simulation	Medium
Heuristic	ML training, hyperparameter search	Medium–Low
Generative	Art pipelines, content quality	Low

Human review is required to cross the ceiling — not as a fallback, but as the structurally necessary component for uncertainty classes that automated verification cannot resolve.

The deployment gate. Minimum SS required before first execution scales with $C_i(d) \times (1 - R(d))$ — iteration cost times irreversibility. In software ($C_i \approx O$, $R \approx 1$), low- SS deployment is survivable. For surgical robots or irreversible legal instruments (C_i large, $R \approx O$), SS must approach the completeness ceiling before execution begins. The Defended property (§4.4) operationalizes this: the executor must know which of its actions require a human gate, from the specification itself.

$S_{\{\text{realized}\}}$ tracking. Eight domain strategies ship with ForgeCraft (UNIVERSAL, API, WEB-REACT, GAME, FINTECH, ML, MOBILE, WEB3). The state file tracks accepted verification steps; $S_{\{\text{aggregate}\}} = \sum S_{\{\text{tag}\}} \cdot c_{\{\text{tag}\}} / \sum c_{\{\text{tag}\}}$ where $c_{\{\text{tag}\}}$ is the ordinal completeness ceiling band weight. The experiment series is the instrument’s calibration run: the three ADR emission fixes from §9.3, the dep governance prescriptive block, and the mutation gate are all now shipped in `templates/universal/instructions.yaml`. The gap between $I \propto (1-S)/S$ and a running instrument is operationally closed.

The uncertainty taxonomy retires the waterfall/agile debate by assigning each paradigm to the uncertainty class it was always right about: waterfall is correct where determinism is high (specification

→ contracts → machine oracle); agile is correct where irreducible uncertainty requires iteration. GS does not resolve the debate. It makes both executable at a fraction of their prior cost.

10. Conclusion

10.1 Summary of Contributions

This work makes four claims and provides evidence for each at the level noted.

Claim 1 — The discipline exists and can be stated precisely. Seven structural properties (Self-describing, Bounded, Verifiable, Defended, Auditable, Composable, Executable) define what a generative specification must satisfy. The properties are operationalized in a 14-point rubric, a practitioner protocol, and a production tool (ForgeCraft). *Evidence: formal definition in §4; six production case studies in §7.*

Claim 2 — The discipline produces measurable quality improvement in solo practice. The AX experiment series (eight conditions — three pre-registered, five post-hoc — plus a ninth construction-invariance condition, §7.8.E; single practitioner, external rubric) shows directional improvement from 3/14 (naive) to 14/14 (Treatment-v5), with a documented regression at Treatment-v4 (11/14, attributable to prompt-structure degradation) and three post-hoc conditions confirming the overall trajectory. Improvement is directional, not monotonic. The prospective result — the condition designed before post-hoc refinement — is Treatment (GS v1): 10/14; the subsequent improvements to 13/14 and 14/14 are post-hoc template refinements that demonstrate the methodology’s capacity for systematic improvement, not replications of the original result. External static analysis checks (tsc, ESLint, npm audit CVEs) are directionally consistent with rubric scores across all conditions. *Evidence: §7.8.B; §9.3.*

Claim 3 — The rubric measures something that exists independent of its author. BX cross-validation scores three community implementations (never exposed to GS) and finds rubric rankings congruent with independent external metrics (CVE count, test count, TypeScript health) on all axes. RX demonstrates 104 passing tests from a fresh GS document, independently reproducible from archived artifacts. *Evidence: §7.8.C (BX); §7.8.G (RX).*

Claim 4 — The discipline is transferable to practitioners. Observational field corroboration from a paying practitioner cohort (§7.8.A) shows single-session uptake on the cohort’s own codebase: practitioners diagnosing generation failures as specification gaps (“it is absolutely my fault, because I did not specify it correctly”), and one instituting an architecture-decision-record-as-merge-gate of his own accord. This is observational, uncontrolled evidence — no control condition, no blind scoring, no pre-registration. *Evidence: §7.8.A (observational). A controlled, pre-registered human-participant study powered for formal inference is noted as future work; transfer to practice is corroborated but not yet established under controlled conditions.*

10.2 Limitations

Single-author specification authorship. All AX and case study specifications were written by the methodology’s creator. The specification-authorship confound is partially mitigated by BX (external implementations) and RX (external replication), and corroborated observationally on a real team (§7.8.A), but not fully closed. A controlled human-participant study providing a pre-written specification to practitioners who did not author it is noted as future work.

Single model. All AX conditions ran on claude-sonnet-4-5. Cross-model generalizability is directionally expected (GS's value is structural, not prompt-specific) but untested. Independent replication on alternative models is invited.

GS rubric is author-designed. Partial mitigation through BX external congruence: the RealWorld implementations were scored against independent external metrics (CVE count, test count, TypeScript health) that predate GS. Full independence requires third-party rubric development, noted as future work.

Scope. Empirical evidence is from software systems. The convergent principle claim (§4.1.c, §10) extends the structural argument to other executor domains; no empirical evidence outside software is offered or claimed.

10.3 Future Work

Field testimony — practitioner transfer (McBrokers, June 2026). Esteban [surname withheld pending consent], tech lead at McBrokers, encountered GS outside any structured study context. After a single session with the methodology, he reports weeks of independent GS-driven development, unsolicited transfer to his team, and team-produced functional software — Minesweeper, a ticketing system — within hours of their first exposure. This is practitioner-transfer evidence: independent, outside any study apparatus, with a named practitioner and verifiable outputs, corroborating the observational field account in §7.8.A. A controlled study would measure formally what this field transfer indicates directionally.

Controlled human-participant study (planned). A pre-registered, powered, multi-cohort study — providing practitioners a pre-written specification they did not author, with blind evaluation by two independent evaluators and inter-rater reliability measured (Cohen's κ) — would move practitioner transfer from observational corroboration to controlled confirmation. A separate team-coordination arm would test whether GS discipline scales from individual practitioners to coordinated teams using shared specification governance and Chronicle memory.

Loom language paper. Formal treatment of the five assembled semantic constructs, multi-target compilation semantics, and the Biological Isomorphisms application frontier. Companion paper in preparation; development at bioiso.dev. Results to be integrated in this work on first-draft completion.

Cross-domain validation. The convergent principle predicts that GS's structural logic applies wherever a capable executor and observable outcomes exist. Robotics, medical AI, and legal drafting are the next natural test domains; the methodology's domain-agnostic properties provide the evaluation instrument.

Generative Specification names the programming discipline of the pragmatic dimension: the tier at which it is necessary to constrain what a *stateless reader* can derive. It emerges at the intersection of Chomsky's hierarchy climbing upward as readers become more expressive, and Martin's sequence of removal where each era of discipline takes away another degree of programmer freedom. Where those pressures meet, the cost of implicit context becomes structural drift.

The paradigm's central claim is that the restriction and the expansion are the same operation. Removing the option to leave intent unstated is not a tax on productivity — it is the enabling condition of everything above the specification vertex: instruct at any level of abstraction, extend across any medium the specification can reach, delegate execution without managing every step. A system built to

generative specification can be: - Understood completely from its own artifacts - Extended correctly by any agent with access to those artifacts - Verified automatically on every change - Defended against structural degradation by its own process - Derived into implementation contract, acceptance test, and living documentation from the same use case artifact

That failure mode is already visible. The discipline that addresses it is available.

GS is a specification-completeness amplifier. $\$I \backslash \text{propto} (1-S)/S$ (a directional model, not a formal result — see §9.4) governs the expected number of correction iterations as a decreasing function of specification completeness. GS raises the starting value of $\$S$, reducing iteration count regardless of domain. What dissolves with stronger models is the compliance scaffolding — structural reminders a more capable reader no longer needs. What does not dissolve is the core: architectural decisions, domain contracts, behavioral boundaries, decision rationale. Those are system-level artifacts. A model that never forgets still needs to be told what the system is.

The experiment closed a loop. The AX study began as a measurement and became the correction mechanism. Treatment-v2 through v5 changes shipped to `templates/universal/instructions.yaml` and propagate to every governed project via `forgecraft refresh_project`. The gap between experimental finding and production tooling is zero. The Replication Experiment (RX) demonstrates independent reproducibility: 104 passing tests across seven suites against a live PostgreSQL instance from a fresh GS document. Open invitation to falsify at [experiments/rx/evidence/](#). **The community ratchet:** template improvements accumulate in the methodology, not the practitioner — a rising floor that cannot retreat while quality gates hold. Contribute at [github.com/jghiringhelli/generative-specification/tree/main/quality-gates](#).

The convergent principle. *No empirical claim beyond software; a structural observation about where the discipline's logic leads.*

Declarative intent, executed by a capable agent, over observable outcomes, with a defined correction mechanism, produces correct results at the completeness of the specification. That sentence contains no reference to software or language models. GS is software's instance of it. The same structural logic applies to any domain where a capable executor exists and desired state can be specified: robotics, medical AI, autonomous vehicles, legal drafting, financial analysis, pedagogical sequencing. Software is the proof — the domain where the pressure crystallized first and the feedback loop compressed fast enough to make the structure visible.

Loom and BIOISO: ongoing experiments. Two active experiments apply the GS structural framework beyond software specification: the Loom language ([github.com/jghiringhelli/loom](#)), which implements GS properties as first-class language constructs, and the BIOISO framework ([bioiso.dev](#)), which applies the same structural tier to self-evolving computational entities across biological and financial domains. Results from these experiments will be reported in their respective companion papers.

11. Onwards

On the irreducible value of the expert.

In 1985, the Therac-25 radiation therapy machine began administering fatal overdoses to cancer patients. Six confirmed deaths before the cause was identified. A hardware safety interlock had been replaced by software written without documentation, concurrent testing, or formal specification of the

failure modes the hardware had previously prevented mechanically. The race condition that killed patients could only be triggered by experienced operators who had learned to enter commands rapidly — it was triggered precisely by competence. The specification that should have governed the software-hardware boundary did not exist.

The lesson usually drawn is about software testing and regulatory oversight. The prior lesson, relevant here: the specification gap killed people not because the software was written by bad engineers, but because every human in the chain was operating against an implicit specification that was no longer accurate. The operators had been trained on the hardware interlock's behavior. The radiologists trusted the error messages. The regulatory review examined what the software claimed to do, not what it could do under conditions the specification had not modeled.

This is the forward-looking obligation GS does not dissolve. Raising S reduces I, but S approaching 1 is a bound, not an achievement. In domains where the feedback loop does not compress to seconds — where the cost of a correction iteration is not a failed test but a clinical outcome — the distance between S_{actual} and S_{complete} is the space the expert human must permanently inhabit. Autonomous executors entering critical domains do not reduce the value of domain expertise. They relocate it: the surgeon governs the outcome, not the instrument; the radiologist reviews every flag; the engineer specifies the system. The craft moves upstream to the tier the executor cannot reach alone.

Therac-25 is the warning about what happens when that relocation is not managed — when the human expert is removed before the specification is complete enough to justify the removal. As GS extends into robotics, medical AI, autonomous systems, and infrastructure automation, specification correctness is not an engineering metric. It is a precondition of safe deployment. The domain experts most capable of identifying specification gaps are precisely those whose work the executor is being asked to perform. Their value concentrates at the residual gap: the hardest part of the domain, the part no specification has yet formalized, the part where the cost of being wrong is highest.

Everything this methodology produces derives from one thing the practitioner writes: the specification of intent. The practitioner who can specify precisely what a system is *for*, with enough precision that a stateless reader can derive everything else, owns the only irreducible function. The executor derives. The specification governs. The human provides the telos.

The organizational principles GS independently rediscovered — information persistence without consumption, error correction before propagation, homeostatic verification, immune memory, differentiated expression from a single specification, evolutionary selection of constraints — are not incidentally similar to biological mechanisms. They are functionally isomorphic to them. This is a second-order isomorphism: not the translation of specific mechanisms — the approach of genetic algorithms or neural networks — but the translation of the architectural relationship between levels. Genetic algorithms model a gear; GS models the transmission. Any sufficiently complex self-maintaining formal system converges on these solutions, because they are the only stable answers to the problems every such system faces. Life found them in carbon over three billion years. GS found them in specification in five. Whether this convergence is coincidence, structural inevitability, or something deeper is not a question this work answers. It is the question this work raises.³

Acknowledgements

Victoria Herrera, for patient counsel on the linguistic foundations of this work. Her expertise in classical Latin and Greek, applied with characteristic understatement, sharpened the precision of the

language throughout. Any remaining imprecision is mine alone.

Norman Owens, Senior Architect, Amazon (Minnesota), for technical review and critical feedback on the architectural and paradigm claims. His reading identified the circularity problem in the validation design, the missing engagement with prompt engineering and LLM benchmarking literature, and several neologisms requiring grounding — all of which materially strengthened this work. His perspective from large-scale production systems was invaluable in making the methodology credible beyond its author’s own context.

Nadjet Bouayad-Agha, for literary and structural review. Her reading as an outsider to the software engineering domain identified where this work assumed prior knowledge it had not earned: missing context before key figures and concepts, an undefined scope, and the drift between theoretical framework and technical specification. Her feedback shaped the accessibility of this work’s opening and the bridging between its registers.

Note on the Provenance of This Work

This work is a product of the methodology it describes. Three projects, built in succession, crystallized the discipline: CodeSeeker (graph-based hybrid retrieval infrastructure), ForgeCraft (structured project scaffolding), and the consolidation that made the pattern visible as a formal claim worth stating. The methodology was not designed in advance and then demonstrated; it was induced from practice and then named.

Intellectual attribution in this context requires precision. The theoretical frameworks structuring the argument — the Chomsky grammar hierarchy as structural analogy, Martin’s paradigm sequence as second axis, the semiotic three-tier taxonomy as classification framework — originated with the author and were developed in dialogue with the AI. What the AI contributed was not frameworks but everything inside them: once a domain is named, the model activates its full training depth for that domain. The author named the doors. The AI supplied the contents of the rooms. Both contributions are necessary; they are not the same contribution.

The review process instantiates the pattern this work describes. Each section was drafted against a specification of what it needed to establish; a blind session — a separate instance of the same model given the completed text with no prior context — critiqued it without attachment to how the arguments were formed. One instance is worth naming: a framing device drawn from J.L. Austin’s speech act theory was introduced by the authoring session, defended when challenged within that session, and rejected by the blind session as an overreach. It does not appear in the formal argument. The episode is recorded because it demonstrates what the adversarial review structure is for. A session that introduces an argument inherits a structural bias toward defending it. The blind session, carrying no investment in the sentence, evaluated it on its merits. That asymmetry is not a failure of the authoring session — it is the expected behavior of any author defending their own work. The discipline was in building the structure that could override it.

The author named the doors. The AI supplied the contents of the rooms. Both contributions are necessary; neither is sufficient alone.

References and Further Reading

- Allen, D. (2001). *Getting Things Done: The Art of Stress-Free Productivity*. Viking.
- Anthropic. (2026). *Claude Code memory — Auto Memory and project memory (MEMORY.md)*. Anthropic documentation. <https://docs.anthropic.com/en/docs/claude-code/memory>
- Bass, L., Clements, P., & Kazman, R. (2003). *Software Architecture in Practice* (2nd ed.). Addison-Wesley.
- Beck, K. (2003). *Test-Driven Development: By Example*. Addison-Wesley.
- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R.C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). *Manifesto for Agile Software Development*. <https://agilemanifesto.org>
- Brooks, F.P. (1987). No Silver Bullet: Essence and Accidents of Software Engineering. *Computer*, 20(4), 10–19.
- Brown, S. (2018). *The C4 Model for Software Architecture*. [leanpub.com](https://leanpub.com/c4model).
- Chomsky, N. (1957). *Syntactic Structures*. Mouton.
- Collins, A., Brown, J. S., & Newman, S. E. (1989). Cognitive apprenticeship: Teaching the crafts of reading, writing, and mathematics. In L. B. Resnick (Ed.), *Knowing, learning, and instruction: Essays in honor of Robert Glaser* (pp. 453–494). Lawrence Erlbaum Associates.
- Conway, M. (1968). How Do Committees Invent? *Datamation*, 14(4), 28–31.
- De Silva, L., & Balasubramaniam, D. (2012). Controlling software architecture erosion: A survey. *Journal of Systems and Software*, 85(1), 132–151.
- Dijkstra, E.W. (1968). Go To Statement Considered Harmful. *Communications of the ACM*, 11(3), 147–148.
- Dreyfus, H. L., & Dreyfus, S. E. (1986). *Mind over machine: The power of human intuition and expertise in the era of the computer*. Free Press.
- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.
- Fillmore, C.J. (1982). Frame semantics. In *Linguistics in the Morning Calm* (pp. 111–137). Hanshin Publishing.
- Firth, J.R. (1957). A Synopsis of Linguistic Theory, 1930–1955. *Studies in Linguistic Analysis*.
- Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press.
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- Fowler, M. (2009). FlaccidScrum. [martinfowler.com](https://martinfowler.com/bliki/FlaccidScrum.html). <https://martinfowler.com/bliki/FlaccidScrum.html>
- Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2nd ed.). Addison-Wesley.
- Fowler, M. (2018). The Practical Test Pyramid. [martinfowler.com](https://martinfowler.com/articles/practical-test-pyramid.html). <https://martinfowler.com/articles/practical-test-pyramid.html>
- Gordon, C.S. (2024). The Linguistics of Programming. *Onward! 2024: Proceedings of the 2024 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. ACM. <https://doi.org/10.1145/3689492.3689806>
- Gray, J. (Ed.). (2009). *The Fourth Paradigm: Data-Intensive Scientific Discovery*. Microsoft Research.

- Jackson, M. (2001). *Problem Frames: Analysing and Structuring Software Development Problems*. Addison-Wesley.
- Jia, Y., & Harman, M. (2011). An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering*, 37(5), 649–678.
- Kluev, A. et al. (2022). Automated API Testing with Schemathesis. *Proceedings of ISSTA 2022*.
- Kuhn, T.S. (1962). *The Structure of Scientific Revolutions*. University of Chicago Press.
- Lehman, M. M. (1980). Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, 68(9), 1060–1076.
- Lee, Y., et al. (2026). *Meta-Harness: End-to-End Optimization of Model Harnesses*. Stanford University, preprint. <https://yoonholee.com/meta-harness/paper.pdf>
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Hopkins, M., Liang, P., & Manning, C. D. (2023). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 157–173.
- Martin, R.C. (2002). *Agile Software Development, Principles, Patterns, and Practices*. Prentice Hall.
- Martin, R.C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- Meyer, D.E., & Schvaneveldt, R.W. (1971). Facilitation in recognizing pairs of words: Evidence of a dependence between retrieval operations. *Journal of Experimental Psychology*, 90(2), 227–234.
- Morris, C.W. (1938). *Foundations of the Theory of Signs*. University of Chicago Press.
- Nygard, M. T. (2011). Documenting architecture decisions. <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>
- OWASP Foundation. (2023). *Web Security Testing Guide v4.2*. <https://owasp.org/www-project-web-security-testing-guide/>
- Orlanski, G., et al. (2026). *SlopCodeBench: Benchmarking Agentic Code Quality Under Iterative Extension*. arXiv:2603.24755 [cs.SE]. <https://doi.org/10.48550/arXiv.2603.24755>
- Osmani, A. (2026). *agent-skills: Production-grade engineering skills for AI coding agents*. Google Cloud AI. <https://github.com/addyosmani/agent-skills>
- Pan, R., et al. (2026). *Natural-Language Agent Harnesses*. Tsinghua University. arXiv:2603.25723. <https://arxiv.org/html/2603.25723v1>
- Parnas, D.L. (1972). On the Criteria To Be Used in Decomposing Systems into Modules. *Communications of the ACM*, 15(12), 1053–1058.
- Parnas, D.L. (1994). Software aging. *Proceedings of the 16th International Conference on Software Engineering (ICSE 1994)*, 279–287.
- Royce, W.W. (1970). Managing the Development of Large Software Systems. *Proceedings of IEEE WESCON*, 26, 1–9.
- Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., & Manning, C.D. (2024). RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. *International Conference on Learning Representations (ICLR 2024)*. <https://arxiv.org/abs/2401.18059>
- Shapiro, D. (2026). The Five Levels: from Spicy Autocomplete to the Dark Factory. danshapiro.com. <https://www.danshapiro.com/blog/2026/01/the-five-levels-from-spicy-autocomplete-to-the-software-factory/>
- Swarmia. (2025). Five levels of AI coding agent autonomy, and why higher isn't always better. swarmia.com. <https://www.swarmia.com/blog/five-levels-ai-agent-autonomy/>
- Squire, L.R. (1987). *Memory and Brain*. Oxford University Press.
- Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2), 257–285.

- Thirolf, T. (2025). *Analysis of Project-Intrinsic Context for Automated Traceability Between Documentation and Code*. Bachelor's thesis, Karlsruhe Institute of Technology (KASTEL). <https://mcse.kastel.kit.edu/downloads/theses/ba-thirolf.pdf>
- Toulmin, S. (1958). *The Uses of Argument*. Cambridge University Press.
- Tulving, E. (1972). Episodic and semantic memory. In E. Tulving & W. Donaldson (Eds.), *Organization of Memory* (pp. 381–403). Academic Press.
- Tulving, E. (1985). Memory and consciousness. *Canadian Psychology*, 26(1), 1–12.
- van Eemeren, F.H., & Grootendorst, R. (2004). *A Systematic Theory of Argumentation: The Pragma-Dialectical Approach*. Cambridge University Press.
- Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS 2017*.
- von Wright, G.H. (1951). Deontic Logic. *Mind*, 60(237), 1–15.
- Vygotsky, L. S. (1978). *Mind in society: The development of higher psychological processes*. Harvard University Press.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H.P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., ... Zaremba, W. (2021). Evaluating Large Language Models Trained on Code. arXiv:2107.03374.
- ISO/IEC 25010:2011. *Systems and Software Engineering, Systems and Software Quality Requirements and Evaluation (SQuaRE), System and Software Quality Models*. International Organization for Standardization.
- Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *International Conference on Learning Representations (ICLR 2024)*. <https://arxiv.org/abs/2310.06770>
- Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. arXiv:2302.06590.
- White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., & Schmidt, D.C. (2023). A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. arXiv:2302.11382.
- Yarmoluk, D., & McCreary, D. (2026). *Benchmarking Knowledge Retrieval Architectures Across Educational and Commercial Domains: RAG, GraphRAG, and Compact Knowledge Graphs*. Version 0.6.2 (preprint). <https://github.com/Yarmoluk/ckg-benchmark>

Glossary

Agentic self-refinement. The AI model's capacity to evaluate its own prior output, detect gaps or inconsistencies, and revise iteratively within the same generation session, guided by the specification rather than by human re-prompting.

Architecture Decision Record (ADR), A short, immutable document capturing a significant architectural choice: the context, the options considered, the decision taken, and the consequences accepted. ADRs accumulate into a permanent decision log.

Architectural constitution. The complete, layered set of constraints, conventions, and principles that governs a system's structural evolution. In the Generative Specification model, the constitution is declared explicitly in the specification document so the AI can enforce it without human supervision.

Context-free grammar (Type 2), A formal grammar in which every production rule has a single non-terminal on the left-hand side. Sufficient to describe most programming-language syntax; insufficient to encode semantic meaning or cross-cutting constraints. Used in this work as a structural analogy for the reading capability of traditional compilers and parsers: deterministic, context-independent, unable to resolve meaning from surrounding context.

Context-sensitive grammar (Type 1), A formal grammar where production rules may depend on the surrounding context of symbols. More expressive than context-free; capable of representing constraints that span clauses, analogous to the cross-reference and consistency obligations carried by a Generative Specification.

Drift surface. The total area of a codebase or specification space that is left unspecified and therefore open to arbitrary resolution by the generating agent. An expanding context window over an underspecified codebase is an expanding drift surface: the model reads more of the implicit record but cannot derive intent that was never externalized. GS practice reduces the drift surface by making intent structurally present at each specification layer.

Generative grammar, In Chomsky's framework, a formal grammar oriented toward modeling linguistic competence: finite production rules applied recursively to yield infinite output. Used here as a structural analogy: the GS document is the finite grammar; the compliant codebase is the language it generates. The analogy imports the structural intuition; it does not import the formal apparatus of transformational grammar or a claim about the formal type class of the language generated.

Derivability. The structural property of an artifact set such that a stateless reader, given those artifacts alone, can correctly determine what should be built, where, why, and to what behavioral and architectural contracts, without requiring external human context. Derivability is the property GS states the obligation to satisfy; it is what distinguishes a generative specification from documentation.

Domain dimensional expansion, Observed pattern (BRAD extension, §7.5) by which naming a domain in the specification activates the full depth of the model's training in that domain. A domain name functions as a coordinate signaling which intellectual territory the problem occupies; the response is the full apparatus of the named field at specialist depth, not a definition. The specification is a technique registry whose scope is the full depth of the model's training, activated at the cost of knowing the correct words to write.

Drift. Architectural incoherence accumulated through AI-assisted development sessions operating against an underspecified grammar. Drift is locally invisible: each generated artifact may pass tests and satisfy type checks while violating the system's architectural intent. It propagates at generation speed across every session that inherits the corrupted context. Drift is the primary failure mode GS addresses.

Generative Specification (GS). The methodology introduced in this work. A structured, versioned document that encodes system architecture, domain rules, and behavioral constraints in sufficient formal detail that a large language model can derive compliant code from it with minimal human mediation.

Hardening surface. The complete set of adversarial conditions against which a system must be specified and verified before deployment: stress testing, security testing, chaos engineering, cross-cutting concern validation, and environment auditing. The hardening surface is a subset of the Verifiable property (§4.4): it extends the test-suite adversarial posture to infrastructure and runtime boundaries. A system that lacks an explicitly specified hardening surface has been proven only against itself.

Living documentation. Documentation that is co-located with, and continuously reconciled against, the system it describes. A Generative Specification is living documentation because it is the authoritative source from which both implementation and tests are derived.

Pragmatic tier, In semiotics, the dimension of sign use that concerns meaning in context: how signs are interpreted by agents in real situations. Applied here to software: the layer where intent, domain knowledge, human judgment, and organizational constraint reside, the layer a formal grammar alone cannot capture.

Phase collapse. The compression of the traditional software sprint phases — planning, implementation, testing, review, and deploy — into a single AI-assisted session. Enabled by a complete specification (which holds intent without reconstitution), a stateless executor (no context-switching cost), and structural quality gates (which close the verification loop automatically). The demo that would end a two-week sprint completes in hours. See §6.5. Distinct from *RED-phase collapse* (§4.4), the narrower, TDD-specific instance in which the RED phase ceases to exist when test and implementation authorship occur in the same context window.

Restriction. The removal of a degree of freedom from the specification space: an intent made structurally present in the artifact set, ruling out outputs that would have been generated in its absence. Every constraint added narrows the output space to the subset that is correct, increasing the AI’s ability to derive the right sentence for a given requirement. The restriction is the expansion mechanism.

Stateless reader, A consumer of a document that carries no prior knowledge of its history, dependencies, or tacit context. A large language model operating on a fresh context window is a stateless reader; the Generative Specification must therefore be self-contained enough to produce correct output without that tacit background.

About the Author

Juan Carlos Ghiringhelli is a senior software and data engineer with two decades of production experience across data infrastructure, AI pipelines, and distributed systems. He holds a Computer Engineering degree from the Universidad de la República Uruguay and a Master’s in Data Science from the Universitat Oberta de Catalunya (supervised by Nadjet Bouayad-Agha, Universitat Pompeu Fabra). His academic record includes a co-authored paper on transit network optimization (LAND-TRANSLOG III, Santa Cruz, Chile, 2016) and a master’s thesis on NLP-based query expansion for vehicle repair documentation (2020).

He is the creator of the Generative Specification methodology described in this work, the ForgeCraft-MCP open-source quality contract tool (`forgecraft-mcp` on npm), and CodeSeeker, a graph-powered hybrid semantic search infrastructure — all built under the discipline this work formalizes.

The first professional line of code was written in 2006. The last one typed by hand was written sometime before June 2025.

Contact: jcghiri@gmail.com · github.com/jghiringhelli

© 2026 Juan Carlos Ghiringhelli. All rights reserved. For republication or citation inquiries, contact the author.

1. The architectural constitution is agent-agnostic as a concept; agent-specific filenames are enumerated in the artifact grammar table (§6). The case studies in this work use `CLAUDE.md` because Claude was the primary agent throughout. The paradigm’s claim holds regardless of which agent or filename is used; these are interchangeable implementations of the same production rule. ↩↩
2. The Onwards essay also develops the synthetic/synaptic/synoptic properties of cross-domain specification, the concrete domain-transfer examples (retrieval architecture traveling from code intelligence to music composition), and the connection between mushin, Saint-Exupéry’s formulation, and GS’s central mechanism of restriction-as-expansion. ↩
3. The philosophical implications of this convergence — including its relationship to the formal tradition’s 2,376-year arc, the structural category of directed formal autopoiesis, and the governance frameworks that compile safety constraints at the type-system level — are explored in the companion essay “*Onwards: The Formal Tradition Was Waiting for Its Executor*” (Ghiringhelli, 2026) and formalized in “*Biological Isomorphisms in Formal Self-Maintaining Systems*” (forthcoming). ↩